
On the Expressive Power of GNNs to Solve Linear SDPs

Chendi Qian¹ Christopher Morris¹

Abstract

Semidefinite programs (SDPs) are a powerful framework for convex optimization and for constructing strong relaxations of hard combinatorial problems. However, solving large SDPs can be computationally expensive, motivating the use of machine learning models as fast computational surrogates. Graph neural networks (GNNs) are a natural candidate in this setting due to their sparsity-awareness and ability to model variable-constraint interactions. In this work, we study what expressive power is sufficient to recover optimal SDP solutions. We first prove negative results showing that standard GNN architectures fail on recovering linear SDP solutions. We then identify a more expressive architecture that captures the key structure of SDPs and can, in particular, emulate the updates of a standard first-order solver. Empirically, on both synthetic and SDPLIB benchmarks of various classes of SDPs, this more expressive architecture achieves consistently lower prediction error and objective gap than theoretically weaker baselines. Finally, using the learned high-quality predictions to warm-start the first-order solver yields practical speedups of up to 80%.

1. Introduction

Semidefinite programming (SDP) is a cornerstone of modern optimization, serving as a fundamental tool in both combinatorial and convex optimization (Boyd & Vandenberghe, 2004; Nocedal & Wright, 1999). In particular, it provides tight relaxations for NP-hard *combinatorial optimization* (CO) problems, such as max-cut (Goemans & Williamson, 1995), max clique (Galli & Letchford, 2017), and graph coloring (Charikar, 2002); and it subsumes important problem classes including *second-order cone programming* (SOCP),

quadratically constrained quadratic programming (QCQP), and *linear programming* (LP) (Dattorro, 2010), see Appendix D for more details. Despite their theoretical utility, solving large-scale SDPs remains computationally prohibitive; state-of-the-art *interior point method* (IPM) solvers typically scale super-cubically with the problem size, e.g., $\mathcal{O}(n^3)$ or even $\mathcal{O}(n^6)$ per-iteration, depending on the concrete implementation (Helmborg et al., 1996; Vandenberghe & Boyd, 1996; Jiang et al., 2020).

To address these scalability challenges, *learning-to-optimize* (L2O) has emerged as a promising approach that aims to replace computationally intensive optimization routines with lightweight machine learning proxies. In this context, *graph neural networks* (GNNs) (Scarselli et al., 2008; Gilmer et al., 2017) have shown strong potential, particularly for learning to solve *mixed-integer linear programs* (MILPs) by representing them as variable-constraint bipartite graphs (Gasse et al., 2019; Chen et al., 2023b). Recent works have further aligned GNNs with classical algorithms, such as the IPM or *primal-dual hybrid gradient* (PDHG) algorithm (Qian et al., 2024; Li et al., 2024a).

However, extending this success to SDPs poses a fundamental challenge, and there is currently a lack of neural architectures capable of effectively representing even *linear SDPs*. Unlike LPs, where variables are vector entries, SDP variables are matrix entries, where the underlying matrix is constrained to the *positive semidefinite* (PSD) cone. This introduces unique structural symmetries, such as equivariance under simultaneous row and column permutations, that standard bipartite representations fail to capture. While the expressivity of GNNs and their connection to the Weisfeiler-Leman (WL) hierarchy of graph isomorphism tests (Cai et al., 1992) are well-studied (Xu et al., 2019; Morris et al., 2019; 2020; Maron et al., 2019a), the specific expressive power required to represent the mapping from a linear SDP instance to its optimal matrix solution remains unclear.

Present work In this work, we bridge GNN expressivity theory and an important class of SDP. That is, we formally characterize the inductive bias needed to represent linear SDPs. Our analysis reveals that 1- and 2-WL-like methods, which we term VC-WL and VC-2-WL, are insufficient. In contrast, we show that a *folklore 2-dimensional Weisfeiler-Leman* (2-FWL) equivalent architecture, which we denote

¹Faculty of Computer Science, RWTH Aachen University, Germany. Correspondence to: Chendi Qian <chendi.qian@log.rwth-aachen.de>.

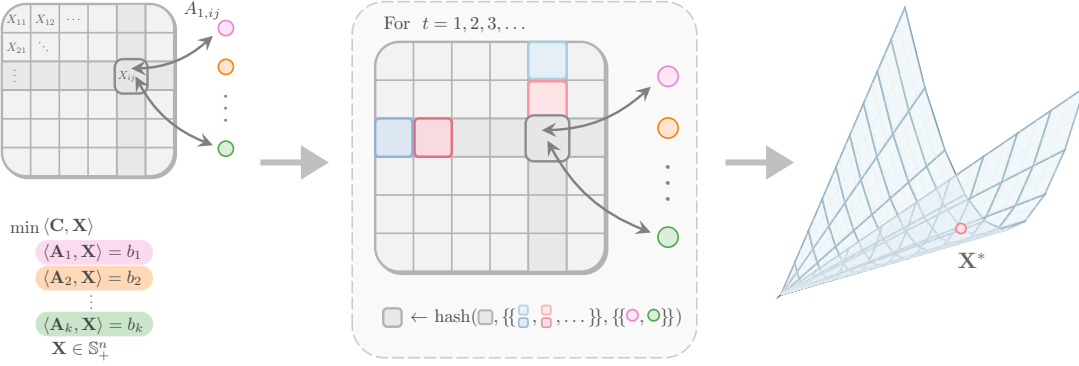


Figure 1. Our framework initializes variable and constraint embeddings directly from the SDP problem data. At each step t , the model iteratively updates these representations by aggregating messages from variable and constraint neighbors via permutation-equivariant functions. Finally, the variable embeddings are decoded to predict the optimal solution X^* .

as VC-2-FWL, is capable of capturing linear SDPs.

Concretely, we contribute the following.

1. **Impossibility results** We prove that standard constraint-variable message passing (VC-WL) fails to represent SDPs. Furthermore, we show that the higher-order architecture VC-2-WL is also insufficient to capture SDPs.
2. **Theoretical sufficiency for solution** We establish that the expressivity of VC-2-FWL is sufficient to represent SDP solutions. Specifically, we prove that the algorithmic operations of the PDHG solver (Chambolle & Pock, 2016; Wang et al., 2024) are subsumed by the stable coloring of VC-2-FWL. This implies that VC-2-FWL possesses the expressivity to simulate the solver’s trajectory and approximate the solution.
3. **Empirical validation** We validate our theory on synthetic and real-world benchmarks, showing that VC-2-FWL-expressive architectures consistently achieve the lowest loss and objective gap. Moreover, we demonstrate that utilizing these high-quality predictions to warm-start a standard solver significantly reduces convergence time.

In summary, we formally establish the expressivity level required to solve linear SDPs, offering the first rigorous theoretical framework to ground the design of neural SDP solvers. By validating that expressive power directly translates to solution quality, our work establishes a principled blueprint for future research in learning-based optimization.

Conflict of Interest Disclosure We declare that we have no specific financial conflicts of interest to disclose related to this work.

1.1. Related work

In the following, we discuss related work.

MPNNs and L2O *Message-passing neural networks* (MPNNs) (Gilmer et al., 2017; Scarselli et al., 2008) have been extensively studied and are broadly categorized into spatial and spectral variants. Spatial MPNNs (Bresson & Laurent, 2017; Duvenaud et al., 2015; Hamilton et al., 2017; Velickovic et al., 2018; Xu et al., 2019) follow the message-passing paradigm introduced by Gilmer et al. (2017). MPNNs have shown strong potential in L2O. A widely adopted approach represents MILPs using constraint-variable bipartite graphs (Gasse et al., 2019; Ding et al., 2020; Chen et al., 2023a; Khalil et al., 2022; Qian et al., 2024). Recent work has also aligned MPNNs with various optimization algorithms, including IPMs (Qian et al., 2024; Qian & Morris, 2025b), PDHG (Li et al., 2024a), and some distributed algorithms (Li et al., 2024b; 2025). From a theoretical perspective, several studies have analyzed the expressivity of MPNNs in approximating solutions to LP, QP, and more general SOCP (Chen et al., 2023a;b; 2024; 2025; Qian et al., 2024; Wu et al., 2025; Qian & Morris, 2025a; Li et al., 2026). In contrast, there has been relatively little work on applying machine learning to SDP. Early approaches leveraged the relation between KKT conditions and dynamical system equilibria to model SDP solving via ODEs (Jiang & Wang, 1999; Gao, 2004; Nikseresht & Nazemi, 2019; Kriváchy et al., 2021). However, these methods are empirically limited to small-scale instances with few variables. Closely related is Yau et al. (2024), which aligns GNNs with low-rank SDP relaxations for maximum constraint satisfaction problems (Max-CSP). However, a crucial distinction lies in the problem objective: Yau et al. (2024) analyzes GNNs as approximation algorithms for CO problems, using the algorithms of their SDP relaxations as a tool. In contrast, our work treats general linear SDPs as the primary object of

study, focusing on the GNN’s ability to recover the optimal SDP solution itself.

Expressivity of GNNs MPNNs are limited by the expressivity of the 1-WL test (Xu et al., 2019; Morris et al., 2019). To address this, various approaches have been proposed, including, among others, higher-order GNNs (Morris et al., 2019; 2020; Maron et al., 2019a; Morris et al., 2022), subgraph-based methods (Bevilacqua et al., 2022; Papp et al., 2021; Qian et al., 2022; Bevilacqua et al., 2024; Frasca et al., 2022; Bar-Shalom et al., 2024; Zhang et al., 2023; Southern et al., 2025), feature augmentation (Sato et al., 2021; You et al., 2021; Brasoveanu et al., 2023; Eliasof et al., 2023), and techniques probing local structure (Zhang & Li, 2021; Chen et al., 2022; Huang et al., 2023; Zhao et al., 2022). Graph transformers (Dwivedi & Bresson, 2020; Rampásek et al., 2022; Kim et al., 2022; Müller et al., 2024) have also been analyzed through the lens of the WL hierarchy (Müller & Morris, 2024; Müller et al., 2024). Notably, Hu et al. (2022) study 2-WL and 2-FWL for link prediction using second-order features, similar in form to our approach. However, our work differs in principle and focuses on semidefinite programming. For a survey on graph expressivity, see Morris et al. (2023).

1.2. Background

Here, we provide the necessary background; see Appendix B for an extended background.

SDP Let $\mathbb{S}^n \subset \mathbb{R}^{n \times n}$ denote the set of symmetric matrices, and $\mathbb{S}_+^n \subset \mathbb{S}^n$ denote the cone of PSD matrices. We consider the standard *linear SDP* problem with primal variable $\mathbf{X} \in \mathbb{S}_+^n$ of the form:¹

$$\begin{aligned} \min_{\mathbf{X} \in \mathbb{S}_+^n} \quad & \langle \mathbf{C}, \mathbf{X} \rangle \\ \text{s.t.} \quad & \langle \mathbf{A}_k, \mathbf{X} \rangle = b_k, \quad k \in [m]. \end{aligned} \quad (1)$$

Here, $\mathbf{A}_k \in \mathbb{S}^n$ represents the k -th constraint matrix, whose entry indexed at (i, j) is $A_{k,ij}$, and we denote the stack of them as a tensor $\mathbf{A} \in \mathbb{R}^{m \times n \times n}$. Here, the dot product on matrices is defined as $\langle \mathbf{C}, \mathbf{X} \rangle := \sum_{i=1}^n \sum_{j=1}^n C_{ij} X_{ij}$. Further, we define the linear operator

$$\mathcal{A}: \mathbb{S}^n \rightarrow \mathbb{R}^m, \quad \mathcal{A}(\mathbf{X}) = (\langle \mathbf{A}_1, \mathbf{X} \rangle, \dots, \langle \mathbf{A}_m, \mathbf{X} \rangle)^\top,$$

and its adjoint operator

$$\mathcal{A}^*: \mathbb{R}^m \rightarrow \mathbb{S}^n, \quad \mathcal{A}^*(\mathbf{y}) = \sum_{k=1}^m y_k \mathbf{A}_k$$

for brevity. While the primal variable $\mathbf{X}$ contains n^2 entries, the symmetry requirement restricts the problem to $\frac{n(n+1)}{2}$

¹In some literature, the min is replaced by inf, as the minimum may not be reached. We exclude this case.

free variables. We assume the constraints are linearly independent; otherwise, some of them can be reduced, or the problem is infeasible, thus m is bounded by $\mathcal{O}(n^2)$. Without loss of generality, we assume the coefficient matrices $\mathbf{C}$ and $\{\mathbf{A}_k\}_{k=1}^m$ are symmetric.² SDP problems may admit multiple optimal solutions with various ranks (Han et al., 2025), making the solution set diverse and intractable to characterize. To ensure a well-defined learning target, we follow Chen et al. (2023a) in the LP case and focus on the unique optimal solution with the *minimum Frobenius norm*.

Proposition 1.1. *The SDP problem defined in Equation (1) has a unique primal solution $\mathbf{X}^*$ with the minimum Frobenius norm $\|\mathbf{X}^*\|_F^2$.*

WL hierarchy The WL hierarchy is one of the heuristics for the graph isomorphism problem (Cai et al., 1992), and a measurement for the expressive power of GNNs (Morris et al., 2023). Let $[n] := \{1, \dots, n\}$ and let $\{\{\dots\}\}$ denote multisets. A graph $G := (V, E)$ consists of a finite set of nodes V and edges $E \subseteq \{(u, v) \mid u, v \in V\}$.³⁴ The 1-WL (Weisfeiler & Leman, 1968) (or *color refinement*) iteratively updates the color c_v^t , for iteration $t > 0$, of a node v based on the colors of its neighbors $N(v) := \{u \in V \mid (u, v) \in E\}$. The node colors are initialized with labeling function $l: V \rightarrow \mathbb{N}$ as $c_v^0 := l(v)$, and updated with

$$c_v^t := \text{hash}\left(c_v^{t-1}, \left\{\left\{c_u^{t-1} \mid u \in N(v)\right\}\right\}\right),$$

for $t > 0$. To distinguish more graphs, the hierarchy extends to k -tuples of nodes ($k \geq 2$), leading to the k -WL. In this work, we focus on the case $k = 2$, where colors are assigned to node pairs $(u, v) \in V^2$. The initialization $c_{uv}^0 := \text{atp}(u, v)$ is the *atomic type*, which encodes the isomorphism type of the pair, i.e., whether $u = v$ and whether $(u, v) \in E$. The *folklore* (2-FWL) and *standard* (2-WL) variants differ in how they aggregate information from a third node $w \in V$. That is, the 2-FWL aggregates the *joint* configuration of w with respect to both u and v , i.e.,

$$c_{uv}^t := \text{hash}\left(c_{uv}^{t-1}, \left\{\left\{(c_{uw}^{t-1}, c_{vw}^{t-1}) \mid w \in V\right\}\right\}\right).$$

In contrast, 2-WL aggregates the interaction of w with each node independently:

$$\text{hash}\left(c_{uv}^{t-1}, \left(\left\{\left\{c_{uw}^{t-1} \mid w \in V\right\}\right\}, \left\{\left\{c_{vw}^{t-1} \mid w \in V\right\}\right\}\right)\right).$$

Crucially, 2-FWL is strictly more expressive than 2-WL (Grohe, 2017). Both algorithms run until stabilization and

²If $\mathbf{C}$ is not symmetric, it can be replaced by $(\mathbf{C} + \mathbf{C}^\top)/2$ without affecting the objective value, as $\mathbf{X}$ is symmetric.

³Sometimes denoted as $V(G)$ and $E(G)$ for specific G .

⁴For notational convenience, we usually write (u, v) for the undirected edge $\{u, v\}$.

yield a unique *stable coloring* c^∞ . Finally, we say algorithm $\mathcal{A}$ *refines* $\mathcal{B}$, denoted $\mathcal{A} \sqsubseteq \mathcal{B}$, if $\mathcal{A}$ leads to a finer partitioning of (higher-order) nodes than $\mathcal{B}$. The corresponding strict relation is denoted by $\sqsubset$.

MPNNs MPNNs learn a d -dimensional real-valued vector of each node in a graph by aggregating information from neighboring nodes. Following Gilmer et al. (2017), let G be an attributed, edge-weighted graph with edge-weights $w: E(G) \rightarrow \mathbb{R}$ with initial node-feature $\mathbf{h}_v^0 \in \mathbb{R}^{d_0}$, $d_0 \in \mathbb{N}$, for $v \in V(G)$. An *MPNN architecture* consists of a composition of L neural network layers for some $L > 0$. In each *layer*, $t \in \mathbb{N}$, we compute a node feature

$$\begin{aligned} \mathbf{m}_v^t &= \text{AGG}^t \left(\left\{ \left(\mathbf{h}_v^{t-1}, \mathbf{h}_u^{t-1}, w_{vu} \right) \mid u \in N(v) \right\} \right) \\ \mathbf{h}_v^t &= \text{UPD}^t \left(\mathbf{h}_v^{t-1}, \mathbf{m}_v^t \right) \in \mathbb{R}^{d_t} \end{aligned}$$

where UPD^t and AGG^t may be parameterized functions, e.g., neural networks. In the case of graph-level tasks, one uses a READOUT function

$$\mathbf{h}_G := \text{READOUT} \left(\left\{ \mathbf{h}_v^L \mid v \in V(G) \right\} \right) \in \mathbb{R}^d,$$

to compute a single vectorial representation based on learned node features after iteration L . Again, READOUT may be a parameterized function.

2. Representing SDP instances

Here, we outline our graph representation for faithfully encoding SDP problems.

2.1. From LP to SDP

We first revisit the standard graph representation used for LP. Given an LP instance $I: \min_{\mathbf{x} \geq 0} \mathbf{c}^\top \mathbf{x}$ s.t. $\mathbf{A}\mathbf{x} = \mathbf{b}$, we can construct a bipartite *variable-constraint* (V-C) graph with constraint nodes $C(I)$ and variable nodes $V(I)$ (Chen et al., 2023a). Edges between $C(I)$ and $V(I)$ are defined by nonzero entries of $\mathbf{A}$ with weights A_{cv} , for $v \in V(I)$, $c \in C(I)$. Since LP is a special case of SDP where all matrices are diagonal, it is natural to extend this representation to the general SDP setting. Specifically, we treat the matrix-shaped variable $\mathbf{X}$ as a collection of n^2 individual variables, initializing node features with coefficients from the objective matrix $\mathbf{C}$; and create m constraint nodes initialized with $\mathbf{b}$. A variable node indexed by (i, j) is connected to a constraint node k if $A_{k,ij} \neq 0$, with edge weight $A_{k,ij}$. We hereby define the neighbors of variable nodes $N(ij) := \{k \in [m] \mid A_{k,ij} \neq 0\}$, corresponding to the variable X_{ij} , and of constraint nodes $N(k) := \{(i, j) \in [n]^2 \mid A_{k,ij} \neq 0\}$, corresponding to the constraint indexed k .

To study the expressive power of MPNNs operating on V-C graphs for SDPs, we define a variant VC-WL of the 1-WL.

To that, let $\mathbf{v}_{ij}^t$ and $\mathbf{c}_k^t$ denote the colors of variable node (i, j) and constraint node k at iteration t , respectively. We define the initialization and color update as follows,

$$\begin{aligned} \mathbf{v}_{ij}^0 &:= \text{init}_v(C_{ij}, \mathbb{I}_{i=j}), \text{ for } i, j \in [n], \\ \mathbf{c}_k^0 &:= \text{init}_c(b_k), \text{ for } k \in [m], \\ \mathbf{v}_{ij}^t &:= \text{hash}(\mathbf{v}_{ij}^{t-1}, \left\{ (A_{k,ij}, \mathbf{c}_k^{t-1}) \mid k \in N(ij) \right\}), \\ \mathbf{c}_k^t &:= \text{hash}(\mathbf{c}_k^{t-1}, \left\{ (A_{k,ij}, \mathbf{v}_{ij}^{t-1}) \mid (i, j) \in N(k) \right\}), \end{aligned} \quad (2)$$

where init and hash are injective functions, $\mathbb{I}_{i=j}$ is a diagonal indicator which takes 1 if $i = j$ otherwise 0. While this representation captures the sparsity of the problem and inherits the invariance and equivariance of MPNNs, we show it fundamentally lacks the expressivity required for SDPs.

Proposition 2.1. *The VC-WL fails to represent linear SDP solutions. That is, there exist instances where the stable coloring of the VC-WL satisfies $\mathbf{v}_{ij}^\infty = \mathbf{v}_{pq}^\infty$ for distinct variable indices (i, j) and (p, q) , yet the entries in the unique optimal solution differ, i.e., $X_{ij}^* \neq X_{pq}^*$.*

Hence, in the following, we investigate more powerful algorithms.

2.2. Leveraging higher-order information

The limitation of standard bipartite graph representation stems from treating SDP variables as independent entities, effectively flattening the problem and discarding the crucial intrinsic matrix geometry. Unlike standard LPs, the primal variables X_{ij} of an SDP form a PSD matrix, characterized by spectral properties and global correlations. Consequently, an effective representation must move beyond bipartite graph representation and instead explicitly model the global interactions among X_{ij} .

We seek a neural architecture $\text{NN}: \mathbb{S}^n \times (\mathbb{S}^n)^m \times \mathbb{R}^m \rightarrow \mathbb{S}^n$ that maps an SDP instance to a solution $\mathbf{X}$ while strictly respecting the problem’s underlying symmetries. Specifically, we enforce three design principles.

- Symmetry** Since the solution resides in $\mathbb{S}_+^n$, for any symmetric inputs $\mathbf{C}$ and $\mathbf{A}$, the solution $\mathbf{X} := \text{NN}(\mathbf{C}, \mathbf{A}, \mathbf{b})$ must satisfy $\mathbf{X} = \mathbf{X}^\top$.
- Equivariance** For any permutation matrix $\mathbf{P} \in \{0, 1\}^{n \times n}$, transforming the inputs via $\mathbf{PCP}^\top$ and $\mathbf{PA}_k\mathbf{P}^\top$ for all k must result in the equivalently permuted output $\mathbf{PXP}^\top$.
- Invariance** For any permutation matrix $\mathbf{Q} \in \{0, 1\}^{m \times m}$ applied to the constraints, the output must remain unchanged: $\text{NN}(\mathbf{C}, \mathbf{QA}, \mathbf{Qb}) = \text{NN}(\mathbf{C}, \mathbf{A}, \mathbf{b})$.

Since the primal variables are indexed by pairs (i, j) , they are structurally analogous to the colored 2-tuples in a higher-order graph. Guided by these principles, 2-WL and 2-FWL

emerge not merely as heuristics, but also as the natural inductive bias for this domain. Consequently, we can adapt the iterative color refinement to distinguish the variables of SDP instances. We formally introduce two variants, the VC-2-WL and VC-2-FWL, by extending the standard 2-WL and 2-FWL with slight modifications and the incorporation of constraint nodes. We define both variants because of the established hierarchy that $2\text{-FWL} \sqsubset 2\text{-WL}$ in graph theory. This comparative approach allows us to theoretically *pinpoint* the exact level of expressivity required, i.e., whether the VC-2-WL suffices or whether joint structural modeling of VC-2-FWL is necessary.

Both variants initialize variable and constraint features according to Equation (2). The VC-2-WL update rule extends the standard 2-WL by integrating constraint node information alongside the structural aggregation, i.e.,

$$\begin{aligned} \mathbf{v}_{ij}^t &:= \text{hash}\left(\mathbf{v}_{ij}^{t-1}, \{\{\mathbf{v}_{uj}^{t-1} \mid u \in [n]\}\}, \{\{\mathbf{v}_{iu}^{t-1} \mid u \in [n]\}\}, \right. \\ &\quad \left. \{\{(A_{k,ij}, \mathbf{c}_k^{t-1}) \mid k \in N(ij)\}\}\right) \quad (3) \\ \mathbf{c}_k^t &:= \text{hash}\left(\mathbf{c}_k^{t-1}, \{\{(A_{k,ij}, \mathbf{v}_{ij}^{t-1}) \mid (i,j) \in N(k)\}\}\right). \end{aligned}$$

The variable update aggregates row and column neighborhoods independently, reflecting standard 2-WL. However, this creates an asymmetry, as the hash function treats the row- and column-neighbor multisets as an *ordered* tuple, meaning the updated $\mathbf{v}_{ij}$ and $\mathbf{v}_{ji}$ are not guaranteed to be identical. To strictly enforce symmetry in output, we explicitly symmetrize the states: $\mathbf{v}_{ji}^t \leftarrow \mathbf{v}_{ij}^t$ for all $i < j$.

In contrast, the VC-2-FWL update couples the neighbor indices to capture joint interactions, i.e.,

$$\begin{aligned} \mathbf{v}_{ij}^t &:= \text{hash}\left(\mathbf{v}_{ij}^{t-1}, \{\{\{\mathbf{v}_{uj}^{t-1}, \mathbf{v}_{iu}^{t-1}\} \mid u \in [n]\}\}, \right. \\ &\quad \left. \{\{(A_{k,ij}, \mathbf{c}_k^{t-1}) \mid k \in N(ij)\}\}\right) \quad (4) \\ \mathbf{c}_k^t &:= \text{hash}\left(\mathbf{c}_k^{t-1}, \{\{(A_{k,ij}, \mathbf{v}_{ij}^{t-1}) \mid (i,j) \in N(k)\}\}\right). \end{aligned}$$

Unlike standard 2-FWL, which aggregates a multiset of *ordered tuples*, our design aggregates a multiset of *unordered multisets* $\{\{\mathbf{v}_{uj}^t, \mathbf{v}_{iu}^t\}\}$. This modification ensures the updated $\mathbf{v}_{ij}^t = \mathbf{v}_{ji}^t$ without the need for manual symmetrization. For theoretical comparison, we define a tuple-based version, VC-2-FWL+, in Appendix E.1. We establish the following strict hierarchy of expressivity, see Appendix C.2 for the detailed proof, i.e.,

$$\text{VC-2-FWL+} \sqsubset \{\text{VC-2-FWL} \not\sqsubset \text{VC-2-WL}\} \sqsubset \text{VC-WL}.$$

That is, VC-2-FWL+ strictly refines VC-2-FWL and VC-2-WL, while VC-2-FWL and VC-2-WL are incomparable, and they both strictly refine VC-WL. Guided by this hierarchy, we first examine the limitations of the intermediate variant, VC-2-WL. Despite strictly refining the VC-WL, we demonstrate that it remains insufficient.

Proposition 2.2. *The VC-2-WL fails to represent linear SDP solutions. That is, there exist instances where the stable colors under VC-2-WL satisfy $\mathbf{v}_{ij}^\infty = \mathbf{v}_{pq}^\infty$ for distinct indices (i, j) and (p, q) , yet the unique optimal solution entries differ, i.e., $X_{ij}^* \neq X_{pq}^*$.*

However, we argue that VC-2-FWL+ is *overly expressive* by proving in Appendix C.4 that VC-2-FWL is sufficiently expressive for this domain. Formally,

Theorem 2.3. *Let $\mathbf{X}^* \in \mathbb{S}_+^n$ be the primal optimal solution to a given SDP instance and given indices $(i, j), (p, q)$. If the stable colorings of VC-2-FWL satisfy $\mathbf{v}_{ij}^\infty = \mathbf{v}_{pq}^\infty$, then the solution values satisfy $X_{ij}^* = X_{pq}^*$.*

In practice, excessive expressivity can introduce unnecessarily too many colors or features, potentially causing the neural network to overfit. On the contrary, by discarding unnecessary directional information, our design yields a smaller set of colors which facilitates faster convergence, and encourages the neural network to learn more robust and generalizable embeddings.

In summary, both VC-2-WL and VC-2-FWL satisfy our three design principles: symmetric outputs, permutation equivariance regarding $\mathbf{C}, \mathbf{A}$, and invariance to constraint ordering. Crucially, VC-2-FWL provides the expressivity level required to solve the problem.

2.3. Complexity

We analyze the computational complexity of VC-2-FWL by mapping it to the standard 1-WL operating on an implicit auxiliary graph. By leveraging tight complexity bounds from Berkholz et al. (2017), we have the following proposition (see Appendix C.5 for proof):

Proposition 2.4. *Given an SDP of $n \times n$ variables and m constraints, $\text{nnz}(\mathbf{A})$ denotes the number of non-zero entries in $\mathbf{A}$, then VC-2-FWL converges in time $\mathcal{O}((n^3 + \text{nnz}(\mathbf{A})) \log n)$.*

The derived complexity is highly competitive for practical applications. In many applications, such as max-cut, the constraint tensor is *sparse*, rendering the $\text{nnz}(\mathbf{A})$ term negligible. Notably, our total complexity is comparable to the fastest implementation of IPM for SDPs (Jiang et al., 2020), which has complexity $\mathcal{O}(n^{3.5} \log \frac{1}{\epsilon})$. Besides, the per-iteration of our VC-2-FWL is $\mathcal{O}(n^3 + \text{nnz}(\mathbf{A}))$, which is comparable to any SDP solver per iteration (Wen et al., 2010; Jiang et al., 2020; Wang et al., 2024), as they all require spectral decomposition of complexity $\mathcal{O}(n^3)$. A critical advantage of our approach, however, is that the neuralized VC-2-FWL being highly compatible for parallelization on GPUs.

2.4. Expressive neural architecture

Based on the initialization functions of Equation (2) and the update functions of Equation (4), we can neuralize the VC-2-FWL using multi-layer perceptrons (MLPs) and instantiate a neural network, which we name VC-2-FMPNN.

The feature initialization is

$$\begin{aligned} \mathbf{h}_k^0 &:= \text{INIT}_c(b_k) \in \mathbb{R}^d \\ \mathbf{h}_{ij}^0 &:= \text{INIT}_v(C_{ij}, \mathbb{I}_{i=j}) \in \mathbb{R}^d \end{aligned} \quad (5)$$

and the update function

$$\begin{aligned} \mathbf{m}_{ij,v \rightarrow v}^t &:= \sum_{u \in [n]} \text{MSG}_{v \rightarrow v}^t(\text{MAP}^t(\mathbf{h}_{uj}^{t-1}) + \text{MAP}^t(\mathbf{h}_{iu}^{t-1})) \\ \mathbf{m}_{ij,c \rightarrow v}^t &:= \sum_{k \in N(ij)} \text{MSG}_{c \rightarrow v}^t(A_{k,ij}, \mathbf{h}_k^{t-1}) \\ \mathbf{m}_{k,v \rightarrow c}^t &:= \sum_{(i,j) \in N(k)} \text{MSG}_{v \rightarrow c}^t(A_{k,ij}, \mathbf{h}_{ij}^{t-1}) \\ \mathbf{h}_{ij}^t &:= \text{UPD}_v^t(\mathbf{h}_{ij}^{t-1}, \mathbf{m}_{ij,v \rightarrow v}^t, \mathbf{m}_{ij,c \rightarrow v}^t) \\ \mathbf{h}_k^t &:= \text{UPD}_c^t(\mathbf{h}_k^{t-1}, \mathbf{m}_{k,v \rightarrow c}^t) \end{aligned} \quad (6)$$

where INIT, MSG, MAP, UPD are MLPs. We now formally establish that this neural architecture possesses the capacity to fully simulate the VC-2-FWL algorithm, thereby inheriting its theoretical expressivity.

Proposition 2.5. *There exists a set of parameters for the functions INIT, UPD, MSG, MAP, such that VC-2-FMPNN has maximal expressivity equal to the VC-2-FWL.*

See to Appendix C.6 for a proof and how we also neuralize VC-WL and VC-2-WL to VC-MPNN and VC-2-MPNN.

3. Empirical results

In the following, we aim to assess the extent to which our theoretical results translate to practice. Concretely, we answer the following questions.

Q1 Expressivity Do the theoretical hierarchies of expressivity translate into approximation performance on SDP?

Q2 Efficiency Can the proposed neural architecture accelerate the solving process for state-of-the-art solvers?

The implementation of our neural methods and baselines can be accessed at <https://github.com/chendiqian/GNN4SDP>.

3.1. Experimental protocol

Here, we outline the architectural configuration and details on training and evaluation. Before presenting our empirical

findings, it is important to clarify the scope of our evaluation. The primary objective of this work is to develop neural architectures capable of approximating solutions to general linear SDPs, rather than serving as an end-to-end solver for CO problems. Consequently, our main experiments evaluate the quality of the predicted SDP continuous solutions against exact SDP solvers. While mapping these continuous embeddings back to discrete integer solutions is beyond our core focus, we have included downstream evaluations comparing our approximated SDP solutions to exact CO targets in Appendix X for completeness.

Neural architecture We evaluate the approximation performance of VC-2-FMPNN for solving SDPs against theoretically weaker baselines, i.e., VC-MPNN and VC-2-MPNN, to verify whether empirical results align with our expressivity hierarchy. Besides, there are a few interesting neural architectures. The δ -VC-2-MPNN and VC-2-IGN, adapted from δ -2-WL (Morris et al., 2020) and 2-IGN (Maron et al., 2019b) respectively, are similar in form to VC-2-MPNN, see Appendix E.2 and Appendix E.3 for their definition and expressivity analysis. Furthermore, we benchmark against the VC-ET, a variant of Edge Transformer (ET) with expressivity equivalent to 2-FWL (Müller et al., 2024), see Appendix E.4 for details.

Datasets We employ synthetic SDP problems as relaxations of classical CO problems, i.e., max-cut, max-clique, max independent set (MIS), vertex cover, and max 2-SAT, as well as linear matrix inequality (LMI) problems from control theory. Detailed problem formulations are provided in Appendix D. For each problem class, we generate 10 000 instances and partition them into training, validation, and test sets with 8:1:1 ratio. For the graph-based tasks, i.e., max-cut, max-clique, MIS, and vertex cover, we construct instances from Erdős–Rényi graphs (Erdős & Rényi, 1959) with 100 nodes and edge probability of 0.1, resulting in SDP problems with 10 000 variables. To specifically challenge the expressivity of weaker baselines and expose approximation gaps, we also curate a more difficult max-cut dataset comprising 10-regular graphs with 100 nodes. Beyond the graph problems, the suite includes max 2-SAT instances with 100 variables and 1000 clauses, as well as LMI control problems with 2500 variables and 500 constraints. We emphasize that our experimental design prioritizes learnability and structural expressivity over scalability. While the instance sizes are moderate, they are sufficient to: (i) reveal the limitations of weaker architectures and (ii) demonstrate significant acceleration for classical SDP solvers.

To demonstrate real-world applicability beyond synthetic benchmarks, we also evaluate our method on real-world instances from the SDPLIB (Borchers, 1999). We select the class of max-cut problems, which consists of 13 instances

Table 1. Loss and relative objective gap (%) on test set. The best results across all are highlighted.

Target	Model	Problems					
		Max-Cut	Max-Cut (reg)	Max-Clique	MIS	Vertex Cover	Max 2-SAT
Test loss	VC-MPNN	0.216±0.000	0.221±0.000	1.016e-5±0.000	9.945e-6±0.000	0.099±0.000	0.217±0.000
	VC-2-MPNN	0.119±0.018	0.221±0.000	5.108e-6±0.000	5.032e-6±0.000	0.029±0.001	0.053±0.009
	δ -VC-2-MPNN	0.014±0.015	0.026±0.003	5.395e-6±0.000	5.058e-6±0.000	0.028±0.001	0.038±0.009
	VC-2-IGN	0.215±0.000	0.221±0.000	6.310e-6±0.000	6.120e-6±0.000	0.030±0.002	0.082±0.007
	VC-2-FMPNN	5.515e-5±0.000	5.140e-5±0.000	5.197e-7±0.000	4.772e-7±0.000	0.001±0.000	0.001±0.000
	VC-ET	0.0001±0.000	0.0002±0.000	2.906e-6±0.000	1.895e-6±0.000	0.013±0.008	0.017±0.011
Raw obj gap (%)	VC-MPNN	1.297±0.003	1.226±0.001	1.228±0.005	1.148±0.006	1.048±0.039	2.044±0.166
	VC-2-MPNN	1.046±0.073	1.225±0.001	0.889±0.023	0.858±0.063	1.235±0.050	0.882±0.053
	δ -VC-2-MPNN	0.835±0.041	0.841±0.002	0.915±0.014	0.882±0.035	1.203±0.031	0.823±0.093
	VC-2-IGN	1.282±0.005	1.225±0.001	1.305±0.071	1.224±0.121	1.274±0.067	0.918±0.084
	VC-2-FMPNN	0.126±0.003	0.111±0.009	0.438±0.036	0.412±0.026	0.429±0.044	0.901±0.201
	VC-ET	0.142±0.043	0.125±0.038	0.659±0.104	0.525±0.124	0.503±0.019	1.149±0.262
Proj. obj gap (%)	VC-MPNN	26.027±0.144	24.722±0.018	1.217±0.006	1.139±0.005	4.948±0.108	29.341±0.254
	VC-2-MPNN	13.662±2.078	24.729±0.019	0.884±0.020	0.852±0.059	1.478±0.265	7.173±0.128
	δ -VC-2-MPNN	1.929±0.021	3.781±0.045	0.906±0.016	0.874±0.033	1.321±0.062	4.208±1.222
	VC-2-IGN	24.796±0.007	24.737±0.008	1.344±0.035	1.242±0.145	1.677±0.447	10.607±0.927
	VC-2-FMPNN	0.111±0.017	0.092±0.004	0.436±0.035	0.411±0.025	0.469±0.051	0.862±0.211
	VC-ET	0.159±0.043	0.128±0.032	0.647±0.102	0.523±0.135	0.510±0.015	1.104±0.146

that are homogeneous in problem structure but vary significantly in scale, as detailed in Table 9. Due to the limited number of available instances, the standard train-validation split is infeasible. Instead, we treat this as a direct test of *trainability*, i.e., assessing whether the architectures can overfit and effectively predict the near-optimal solution.

Training and evaluation All models were trained using the Adam optimizer (Kingma & Ba, 2015) with default hyperparameters and batch size of 256. The training objective is to minimize the supervised mean squared error (MSE) between the predicted and ground-truth solution matrices. Training for synthetic datasets were conducted on a compute cluster with four NVIDIA L40S GPUs, whereas SDPLIB utilized a single GPU. We trained for at most 1000 epochs, using a learning rate scheduler that decays the learning rate after 100 consecutive epochs without improvement in the relative objective gap on the validation set. Additionally, early stopping was applied if this metric failed to improve for 200 consecutive epochs.

To quantify the computational efficiency of neural SDP solvers, we benchmarked inference time against classical solvers using the max-cut problem as a representative case study. We compared against SDPLR, a solver based on Burer-Monteiro method (Burer & Monteiro, 2003), MOSEK (ApS, 2022), an IPM solver, and SCS (O’Donoghue, 2021), a first-order solver based on the ADMM algorithm (Wen et al., 2010). SDPLR and MOSEK were executed on an Intel® Xeon® Silver 4510 CPU, as they lack native GPU support. Both the SCS solver and all the neural architectures were executed on a single NVIDIA L40S GPU. Additionally, we investigate the practical utility of our approach by using the VC-2-FMPNN prediction to warm-start the SCS solver.

For all experiments, including model training and solver

timing, we compute evaluation metrics, specifically loss and objective gap, by averaging over the full dataset. We repeat this process using five different random seeds and report the resulting mean and standard deviation.

3.2. Results and discussion

In the following, we present results and discuss them.

Approximation performance The results of test loss and relative objective gap are reported in Table 1. The optimal objective values for LMI are 0, so we report the predicted objective in Table 6. We first examine the test loss. Consistent with the theoretical hierarchy, our VC-2-FMPNN and equally powerful VC-ET achieve the lowest losses across all problems, suggesting that 2-FWL-like updates are essential for SDP approximation. This advantage is most significant on max-cut, where VC-2-FMPNN outperforms VC-MPNN, VC-2-MPNN, δ -VC-2-MPNN, and VC-2-IGN by orders of magnitude, revealing a clear expressivity hierarchy and the incapacity in those weaker baselines. To further evaluate the quality of the predicted solutions, we compute the relative objective gap over the test set $\mathcal{D}$, defined as

$$\frac{1}{|\mathcal{D}|} \sum_{I \in \mathcal{D}} \left| \frac{\text{obj}(I) - \text{obj}^*(I)}{\text{obj}^*(I)} \right| \cdot 100\%,$$

where $\text{obj}(I)$ denotes the predicted objective value and $\text{obj}^*(I)$ the optimal value for instance I . Furthermore, the predicted solution matrix X can be projected onto the PSD cone, as shown in Algorithm 1. The results in Table 1 mirror the test loss trends. For the raw objective gap, VC-2-FMPNN outperforms almost all baselines across all tasks, except slightly worse than δ -VC-2-MPNN on max 2-SAT. However, the objective gaps for VC-2-FMPNN and VC-ET after PSD projection remain low, indicating that

their predictions are not only accurate in objective value but also geometrically close to the PSD cone. In sharp contrast, models with weaker expressivity often produce solutions far from that. This is evidenced by the drastic degradation in their performance after projection. For instance, the gap for VC-MPNN on max-cut rockets from $\sim 1.3\%$ to over 26%, with similar collapses observed on max 2-SAT. Interestingly, VC-MPNN, VC-2-MPNN and VC-2-IGN fail on the regular graph dataset, producing identical losses and gaps, but δ -VC-2-MPNN shows significant advantage over them, reflecting our theory of their hierarchy in Appendix E.2.

Table 2. Mean absolute residuals on constraints of max-cut problems on test set, repeated with 5 seeds.

Model	Cons. vio.	Proj. cons. vio.
VC-MPNN	0.001$\pm$0.000	0.298 $\pm$ 0.001
VC-2-MPNN	0.016 $\pm$ 0.006	0.180 $\pm$ 0.001
δ -VC-2-MPNN	0.006 $\pm$ 0.001	0.069 $\pm$ 0.019
VC-2-IGN	0.001$\pm$0.000	0.304 $\pm$ 0.001
VC-2-FMPNN	0.003 $\pm$ 0.001	0.003$\pm$0.001
VC-ET	0.001$\pm$0.000	0.003$\pm$0.001

Besides loss and objective, we also probe the structural feasibility of the predictions by measuring the mean absolute residuals on constraints for each problem instance:

$$\frac{1}{m} \sum_{k \in [m]} |\langle \mathbf{A}_k, \mathbf{X} \rangle - b_k|$$

and averaged over the dataset. We show the results on max-cut problem as representative in Table 2. While some weaker methods show low level of constraint violation, such as VC-MPNN and VC-2-IGN, the projected solutions exhibit a huge gap, indicating their predicted solutions are far from feasible region. In comparison, VC-2-FMPNN maintains negligible violation (~ 0.003) both before and after projection, demonstrating that it successfully approximates the true geometry of the feasible solution. For more results on constraint violation, see Table 11.

We compare VC-MPNN, VC-2-MPNN and VC-2-FMPNN on SDPLIB. Table 3 reports the training MSE loss. For more complete results, see Table 10. The results reveal the dramatic gap in trainability: VC-MPNN and VC-2-MPNN fail to even overfit the training data, observing from the high losses. In contrast, VC-2-FMPNN achieves negligible training loss around 10^{-4} magnitude.

Overall, these results align closely with our theory: VC-2-FMPNN and VC-ET, with VC-2-FWL expressive power, not only approximate objective values more accurately but also produce solutions closer to the feasible region, showing their ability to learn high-quality SDP solutions.

Table 3. Training loss on SDPLIB instances. The best results across all are highlighted.

Name	Loss		
	VC-MPNN	VC-2-MPNN	VC-2-FMPNN
MCP100	0.203 $\pm$ 0.000	0.199 $\pm$ 0.004	1.974e-4$\pm$0.000
MCP124-1	0.225 $\pm$ 0.000	0.224 $\pm$ 0.000	1.546e-4$\pm$0.000
MCP124-2	0.220 $\pm$ 0.000	0.218 $\pm$ 0.002	2.303e-4$\pm$0.000
MCP124-3	0.215 $\pm$ 0.000	0.214 $\pm$ 0.002	3.076e-4$\pm$0.000
MCP124-4	0.270 $\pm$ 0.000	0.272 $\pm$ 0.007	3.374e-4$\pm$0.000
MCP250-1	0.215 $\pm$ 0.000	0.215 $\pm$ 0.000	3.034e-4$\pm$0.000
MCP250-2	0.158 $\pm$ 0.015	0.160 $\pm$ 0.007	4.176e-4$\pm$0.000
MCP250-3	0.155 $\pm$ 0.000	0.156 $\pm$ 0.000	6.266e-4$\pm$0.000
MCP250-4	0.182 $\pm$ 0.013	0.186 $\pm$ 0.004	5.892e-4$\pm$0.000
MCP500-1	0.118 $\pm$ 0.000	0.118 $\pm$ 0.001	4.536e-4$\pm$0.000
MCP500-2	0.123 $\pm$ 0.000	0.124 $\pm$ 0.001	4.948e-4$\pm$0.000
MCP500-3	0.121 $\pm$ 0.000	0.122 $\pm$ 0.000	6.745e-4$\pm$0.000
MCP500-4	0.126 $\pm$ 0.000	0.130 $\pm$ 0.003	9.761e-4$\pm$0.000

Table 4. Timing (in seconds) on various sizes of max-cut problem.

Model	Sizes		
	50 $\times$ 50	100 $\times$ 100	150 $\times$ 150
SDPLR	0.111 $\pm$ 0.050	0.195 $\pm$ 0.041	0.285 $\pm$ 0.062
MOSEK	0.206 $\pm$ 0.008	4.206 $\pm$ 0.151	28.839 $\pm$ 1.814
SCS	0.061 $\pm$ 0.004	0.386 $\pm$ 0.028	3.357 $\pm$ 0.240
SCS (Warm s.)	0.042 $\pm$ 0.005	0.280 $\pm$ 0.041	2.092 $\pm$ 0.298
VC-MPNN	0.006$\pm$0.000	0.006$\pm$0.000	0.008$\pm$0.000
VC-2-MPNN	0.009 $\pm$ 0.000	0.009 $\pm$ 0.000	0.012 $\pm$ 0.000
δ -VC-2-MPNN	0.011 $\pm$ 0.000	0.012 $\pm$ 0.002	0.017 $\pm$ 0.003
VC-2-IGN	0.010 $\pm$ 0.000	0.010 $\pm$ 0.000	0.015 $\pm$ 0.000
VC-2-FMPNN	0.009 $\pm$ 0.000	0.010 $\pm$ 0.000	0.013 $\pm$ 0.000
VC-ET	0.010 $\pm$ 0.000	0.020 $\pm$ 0.000	0.050 $\pm$ 0.001

Timing As detailed in Table 4, classical solvers’ runtime scales aggressively with problem size, especially MOSEK. In contrast, neural inference time is negligible and exhibit minimal growth, due to GPU parallelism. Among the neural architectures, VC-2-FMPNN exhibits an optimal balance between efficiency and expressivity. It is only marginally slower than VC-MPNN, yet significantly faster than VC-ET, particularly as problem size increases. Finally, the warm-start experiments demonstrate the practical value of VC-2-FMPNN: initializing SCS with its prediction consistently reduces convergence time, e.g., from 3.35s to 2.09s on the largest instances, highlighting the potential application of neural approximation into classical optimization. For more timing results on SCS and our neural approach, see Table 11.

We extend our timing analysis to the SDPLIB, where problem difficulty varies significantly. Table 5 shows the timing of the SCS solver versus our VC-2-FMPNN inference, as well as the performance gains achieved by using the neural prediction to warm-start the solver. The results underscore the scalability of our approach. While the classical solver’s runtime grows rapidly over 5 minutes for the

Table 5. Timing (in seconds) and warm start the solver on SDPLIB.

Name	SCS	VC-2-FMPNN	Warm start	Improvement
MCP100	0.560±0.005	0.015±0.001	0.287±0.005	48.7%
MCP124-1	3.243±0.006	0.016±0.000	0.623±0.067	80.7%
MCP124-2	1.152±0.003	0.016±0.000	0.532±0.002	53.8%
MCP124-3	0.759±0.002	0.016±0.000	0.467±0.050	38.4%
MCP124-4	0.546±0.005	0.016±0.000	0.484±0.056	11.3%
MCP250-1	17.274±0.044	0.024±0.000	4.717±0.035	72.6%
MCP250-2	12.308±0.035	0.024±0.000	5.971±1.029	51.5%
MCP250-3	6.460±0.017	0.024±0.000	3.302±0.013	48.8%
MCP250-4	4.556±0.011	0.024±0.000	3.316±0.035	27.2%
MCP500-1	328.471±1.236	0.147±0.000	77.471±0.796	76.4%
MCP500-2	217.164±1.629	0.151±0.005	68.822±0.849	68.3%
MCP500-3	134.705±0.198	0.147±0.000	40.586±0.542	69.8%
MCP500-4	81.779±0.944	0.148±0.000	39.425±0.242	51.8%

hardest instance (MCP500-1), the VC-2-FMPNN inference time remains negligible, completing under 0.15 seconds across all cases. More importantly, initializing SCS with the VC-2-FMPNN prediction consistently accelerates convergence, yielding runtime reductions up to 80.7%. This confirms that VC-2-FMPNN is able to capture high-quality approximation of the global optimum, significantly reducing the workload for the iterative solver.

4. Conclusion

We studied the expressive power required of GNNs to predict optimal solutions of linear SDPs, using the unique minimum-Frobenius-norm solution as the learning target. We proved that VC-WL and higher-order VC-2-WL can fail on SDPs, whereas VC-2-FWL-equivalent architectures are sufficient because they can emulate PDHG solver updates until convergence. Empirically, these more expressive architectures achieve the lowest prediction error and the smallest objective gaps across synthetic and real-world benchmarks. Moreover, their predictions can warm-start a first-order solver (SCS), yielding substantial speedups on SDPLIB instances. *Overall, our results sharpen the understanding of the expressivity required to learn solutions for a large, practically important class of convex optimization problems.*

Impact Statement

This paper presents work aimed at advancing the field of machine learning. There are many potential societal consequences of our work, none of which we feel must be specifically highlighted here.

Acknowledgements

Christopher Morris and Chendi Qian are partially funded by a DFG Emmy Noether grant (468502433) and RWTH Junior Principal Investigator Fellowship under Germany’s Excellence Strategy. We thank Erik Müller for crafting the figures.

References

- ApS, M. Mosek optimizer api for python. *Version*, 9(17): 6–4, 2022. 7
- Ba, L. J., Kiros, J. R., and Hinton, G. E. Layer normalization. *CoRR*, 2016. 42, 43
- Babai, L. Lectures on graph isomorphism. *University of Toronto, Department of Computer Science. Mimeographed lecture notes*, 3:15, 1979. 14
- Bar-Shalom, G., Eitan, Y., Frasca, F., and Maron, H. A flexible, equivariant framework for subgraph gnn’s via graph products and graph coarsening. *Neural Information Processing Systems*, 2024. 3
- Bauschke, H. H. and Combettes, P. L. Correction to: convex analysis and monotone operator theory in hilbert spaces. In *Convex analysis and monotone operator theory in Hilbert spaces*, pp. C1–C4. Springer, 2020. 21
- Berkholz, C., Bonsma, P., and Grohe, M. Tight lower and upper bounds for the complexity of canonical colour refinement. *Theory of Computing Systems*, 60(4):581–614, 2017. 5, 27
- Bevilacqua, B., Frasca, F., Lim, D., Srinivasan, B., Cai, C., Balamurugan, G., Bronstein, M. M., and Maron, H. Equivariant subgraph aggregation networks. In *International Conference on Learning Representations*, 2022. 3
- Bevilacqua, B., Eliasof, M., Meirom, E. A., Ribeiro, B., and Maron, H. Efficient subgraph gnn’s by learning effective selection policies. In *International Conference on Learning Representations*, 2024. 3
- Borchers, B. Sdplib 1.2, a library of semidefinite programming test problems. *Optimization Methods and Software*, 11(1-4):683–690, 1999. URL <https://doi.org/10.1080/10556789908805769>. 6
- Boyd, S. and Vandenberghe, L. *Convex Optimization*. Cambridge University Press, 2004. 1, 21
- Boyd, S., El Ghaoui, L., Feron, E., and Balakrishnan, V. *Linear matrix inequalities in system and control theory*. SIAM, 1994. 36
- Brasoveanu, A. D., Jögl, F., Welke, P., and Thiessen, M. Extending graph neural networks with global features. In *The Second Learning on Graphs Conference*, 2023. 3
- Bresson, X. and Laurent, T. Residual gated graph convnets. *CoRR*, 2017. 2

- Burer, S. and Monteiro, R. D. A nonlinear programming algorithm for solving semidefinite programs via low-rank factorization. *Mathematical programming*, 95(2):329–357, 2003. [7](#)
- Cai, J.-Y., Fürer, M., and Immerman, N. An optimal lower bound on the number of variables for graph identification. *Combinatorica*, 12(4):389–410, 1992. [1](#), [3](#), [14](#), [15](#)
- Cai, T., Luo, S., Xu, K., He, D., Liu, T.-y., and Wang, L. Graphnorm: A principled approach to accelerating graph neural network training. In *International Conference on Machine Learning*, 2021. [43](#)
- Chambolle, A. and Pock, T. On the ergodic convergence rates of a first-order primal–dual algorithm. *Mathematical Programming*, 159(1):253–287, 2016. [2](#), [21](#), [25](#)
- Charikar, M. On semidefinite programming relaxations for graph coloring and vertex cover. In *SODA*, 2002. [1](#), [33](#)
- Chen, D., O’Bray, L., and Borgwardt, K. Structure-aware transformer for graph representation learning. In *International Conference on Machine Learning*, 2022. [3](#)
- Chen, Z., Liu, J., Wang, X., and Yin, W. On representing linear programs by graph neural networks. In *International Conference on Learning Representations*, 2023a. [2](#), [3](#), [4](#)
- Chen, Z., Liu, J., Wang, X., and Yin, W. On representing mixed-integer linear programs by graph neural networks. In *International Conference on Learning Representations*, 2023b. [1](#), [2](#)
- Chen, Z., Liu, J., Chen, X., Wang, W., and Yin, W. Rethinking the capacity of graph neural networks for branching strategy. *Neural Information Processing Systems*, 2024. [2](#)
- Chen, Z., Chen, X., Liu, J., Wang, X., and Yin, W. Expressive power of graph neural networks for (mixed-integer) quadratic programs. In *International Conference on Machine Learning*. PMLR, 2025. [2](#)
- Dattorro, J. *Convex optimization & Euclidean distance geometry*. Lulu.com, 2010. [1](#), [35](#)
- de Klerk, E. and Warners, J. Semidefinite programming approaches for max-2-sat and max-3-sat: computational perspectives. In *Combinatorial and global optimization*, pp. 161–176. World Scientific, 2002. [34](#)
- Ding, J.-Y., Zhang, C., Shen, L., Li, S., Wang, B., Xu, Y., and Song, L. Accelerating primal solution findings for mixed integer programs based on solution prediction. In *AAAI*, 2020. [2](#)
- Duvenaud, D., Maclaurin, D., Aguilera-Iparraguirre, J., Gómez-Bombarelli, R., Hirzel, T., Aspuru-Guzik, A., and Adams, R. P. Convolutional networks on graphs for learning molecular fingerprints. In *Neural Information Processing Systems*, 2015. [2](#)
- Dwivedi, V. P. and Bresson, X. A generalization of transformer networks to graphs. *arXiv preprint arXiv:2012.09699*, 2020. [3](#)
- Eckstein, J. and Bertsekas, D. P. On the douglas–rachford splitting method and the proximal point algorithm for maximal monotone operators. *Mathematical programming*, 55(1):293–318, 1992. [20](#)
- Eliasof, M., Frasca, F., Bevilacqua, B., Treister, E., Chechik, G., and Maron, H. Graph positional encoding via random feature propagation. In *International Conference on Machine Learning*, 2023. [3](#)
- Erdős, P. and Rényi, A. On random graphs i. *Publ. math. debrecen*, 6(290-297):18, 1959. [6](#)
- Frasca, F., Bevilacqua, B., Bronstein, M. M., and Maron, H. Understanding and extending subgraph gnn’s by rethinking their symmetries. In *Neural Information Processing Systems*, 2022. [3](#)
- Fürer, M. Graph isomorphism testing without numerics for graphs of bounded eigenvalue multiplicity. In *SODA*, 1995. [24](#)
- Fürer, M. On the power of combinatorial and spectral invariants. *Linear algebra and its applications*, 432(9): 2373–2380, 2010. [24](#)
- Galli, L. and Letchford, A. N. On the Lovász theta function and some variants. *Discrete Optimization*, 25:159–174, 2017. [1](#), [33](#)
- Gao, X.-B. A novel neural network for nonlinear convex programming. *IEEE Transactions on Neural Networks*, 15(3):613–621, 2004. [2](#)
- Gasse, M., Chételat, D., Ferroni, N., Charlin, L., and Lodi, A. Exact combinatorial optimization with graph convolutional neural networks. *Neural Information Processing Systems*, 2019. [1](#), [2](#), [44](#)
- Gilmer, J., Schoenholz, S. S., Riley, P. F., Vinyals, O., and Dahl, G. E. Neural message passing for quantum chemistry. In *International Conference on Machine Learning*, 2017. [1](#), [2](#), [4](#)
- Goemans, M. X. and Williamson, D. P. Improved approximation algorithms for maximum cut and satisfiability problems using semidefinite programming. *Journal of the ACM (JACM)*, 42(6):1115–1145, 1995. [1](#), [32](#)

- Grohe, M. *Descriptive complexity, canonisation, and definable graph structure theory*. Cambridge University Press, 2017. 3, 14, 15
- Grohe, M. The logic of graph neural networks. In *Symposium on Logic in Computer Science*, 2021. 14
- Hamilton, W., Ying, Z., and Leskovec, J. Inductive representation learning on large graphs. *Neural Information Processing Systems*, 2017. 2
- Han, Q., Li, C., Lin, Z., Chen, C., Deng, Q., Ge, D., Liu, H., and Ye, Y. A low-rank admm splitting approach for semidefinite programming. *INFORMS Journal on Computing*, 2025. 3
- Hatami, H., Magen, A., and Markakis, E. Integrality gaps of semidefinite programs for vertex cover and relations to ℓ_1 embeddability of negative type metrics. *SIAM Journal on Discrete Mathematics*, 23(1):178–194, 2009. 34
- Helmberg, C., Rendl, F., Vanderbei, R. J., and Wolkowicz, H. An interior-point method for semidefinite programming. *SIAM Journal on optimization*, 6(2):342–361, 1996. 1
- Horn, R. A. and Johnson, C. R. *Topics in matrix analysis*. Cambridge university press, 1994. 24, 25
- Hrga, T. and Povh, J. Madam: a parallel exact solver for max-cut based on semidefinite programming and admm. *Computational Optimization and Applications*, 80(2):347–375, 2021. 44
- Hu, Y., Wang, X., Lin, Z., Li, P., and Zhang, M. Two-dimensional weisfeiler-lehman graph neural networks for link prediction. *arXiv preprint arXiv:2206.09567*, 2022. 3
- Huang, Y., Peng, X., Ma, J., and Zhang, M. Boosting the cycle counting power of graph neural networks with ℓ_2 -gnn. In *International Conference on Learning Representations*, 2023. 3
- Immerman, N. and Lander, E. Describing graphs: A first-order approach to graph canonization. In *Complexity Theory Retrospective: In Honor of Juris Hartmanis on the Occasion of His Sixtieth Birthday, July 5, 1988*, pp. 59–81, 1990. 14
- Jiang, D. and Wang, J. A recurrent neural network for real-time semidefinite programming. *IEEE transactions on neural networks*, 10(1):81–93, 1999. 2
- Jiang, H., Kathuria, T., Lee, Y. T., Padmanabhan, S., and Song, Z. A faster interior point method for semidefinite programming. In *FOCS. IEEE*, 2020. 1, 5
- Khalil, E. B., Morris, C., and Lodi, A. MIP-GNN: A data-driven framework for guiding combinatorial solvers. In *AAAI*, 2022. 2
- Kim, J., Nguyen, D., Min, S., Cho, S., Lee, M., Lee, H., and Hong, S. Pure transformers are powerful graph learners. In *Advances in Neural Information Processing Systems*, 2022. 3
- Kingma, D. P. and Ba, J. Adam: A method for stochastic optimization. In *International Conference on Learning Representations*, 2015. 7
- Kleinberg, J. and Goemans, M. X. The Lovász theta function and a semidefinite programming relaxation of vertex cover. *SIAM Journal on Discrete Mathematics*, 11(2): 196–204, 1998. 34
- Kriváchy, T., Cai, Y., Bowles, J., Cavalcanti, D., and Brunner, N. High-speed batch processing of semidefinite programs with feedforward neural networks. *New Journal of Physics*, 23(10):103034, 2021. 2
- Levin, E., Ma, Y., Díaz, M., and Villar, S. On transferring transferability: Towards a theory for size generalization. *arXiv preprint arXiv:2505.23599*, 2025. 14
- Li, B., Yang, L., Chen, Y., Wang, S., Mao, H., Chen, Q., Ma, Y., Wang, A., Ding, T., Tang, J., and Sun, R. PDHG-unrolled learning-to-optimize method for large-scale linear programming. In *International Conference on Machine Learning*, PMLR, 2024a. 1, 2
- Li, Q., Ding, T., Yang, L., Ouyang, M., Shi, Q., and Sun, R. On the power of small-size graph neural networks for linear programming. In *Neural Information Processing Systems*, 2024b. 2
- Li, Q., Ouyang, M., Ding, T., Wang, Y., Shi, Q., and Sun, R. Towards explaining the power of constant-depth graph neural networks for structured linear programming. In *International Conference on Learning Representations*, 2025. 2
- Li, R., Liang, E., and Chen, M. On the universality and complexity of gnn for solving second-order cone programs. In *The Fourteenth International Conference on Learning Representations*, 2026. 2
- Lobo, M. S., Vandenberghe, L., Boyd, S., and Lebret, H. Applications of second-order cone programming. *Linear algebra and its applications*, 284(1-3):193–228, 1998. 35
- Lorenz, D. A. and Tran-Dinh, Q. Non-stationary douglas-rachford and alternating direction method of multipliers: adaptive step-sizes and convergence. *Computational Optimization and Applications*, 74(1):67–92, 2019. 23

- Maron, H., Ben-Hamu, H., Serviansky, H., and Lipman, Y. Provably powerful graph networks. In *Neural Information Processing Systems*, 2019a. 1, 3, 29, 43
- Maron, H., Ben-Hamu, H., Shamir, N., and Lipman, Y. Invariant and equivariant graph networks. In *International Conference on Learning Representations*, 2019b. 6, 40
- Morris, C., Ritzert, M., Fey, M., Hamilton, W. L., Lenssen, J. E., Rattan, G., and Grohe, M. Weisfeiler and Leman go neural: Higher-order graph neural networks. In *AAAI*, 2019. 1, 3, 14, 29
- Morris, C., Rattan, G., and Mutzel, P. Weisfeiler and Leman go sparse: Towards higher-order graph embeddings. In *Neural Information Processing Systems*, 2020. 1, 3, 6, 38
- Morris, C., Rattan, G., Kiefer, S., and Ravanbakhsh, S. SpeqNets: Sparsity-aware permutation-equivariant graph networks. In *International Conference on Machine Learning*, 2022. 3
- Morris, C., Lipman, Y., Maron, H., Rieck, B., Kriege, N. M., Grohe, M., Fey, M., and Borgwardt, K. M. Weisfeiler and leman go machine learning: The story so far. *JMLR*, 2023. 3
- Müller, L. and Morris, C. Aligning transformers with weisfeiler-leman. In *International Conference on Machine Learning*, PMLR, 2024. 3
- Müller, L., Galkin, M., Morris, C., and Rampásek, L. Attending to graph transformers. *TMLR*, 2024. 3
- Müller, L., Kusuma, D., Bonet, B., and Morris, C. Towards principled graph transformers. *Advances in Neural Information Processing Systems*, 2024. 3, 6, 42
- Nikseresht, A. and Nazemi, A. A novel neural network for solving semidefinite programming problems with some applications. *Journal of Computational and Applied Mathematics*, 350:309–323, 2019. 2
- Nocedal, J. and Wright, S. J. *Numerical optimization*. Springer, 1999. 1
- O’Donoghue, B. Operator splitting for a homogeneous embedding of the linear complementarity problem. *SIAM Journal on Optimization*, 31(3):1999–2023, 2021. 7
- Oliveira, D. E., Wolkowicz, H., and Xu, Y. Admm for the sdp relaxation of the qap. *Mathematical Programming Computation*, 10(4):631–658, 2018. 35
- O’Connor, D. and Vandenberghe, L. On the equivalence of the primal-dual hybrid gradient method and douglas-rachford splitting. *Mathematical Programming*, 179(1): 85–108, 2020. 20
- Papp, P. A., Martinkus, K., Faber, L., and Wattenhofer, R. Dropgnn: Random dropouts increase the expressiveness of graph neural networks. *Neural Information Processing Systems*, 2021. 3
- Park, S. and Van Hentenryck, P. Self-supervised primal-dual learning for constrained optimization. In *AAAI*, 2023. 14
- Povh, J. and Rendl, F. Copositive and semidefinite relaxations of the quadratic assignment problem. *Discrete Optimization*, 6(3):231–241, 2009. 35
- Qian, C. and Morris, C. Principled data augmentation for learning to solve quadratic programming problems. *arXiv preprint arXiv:2506.01728*, 2025a. 2
- Qian, C. and Morris, C. Towards graph neural networks for provably solving convex optimization problems. *arXiv preprint arXiv:2502.02446*, 2025b. 2
- Qian, C., Rattan, G., Geerts, F., Morris, C., and Niepert, M. Ordered subgraph aggregation networks. In *Neural Information Processing Systems*, 2022. 3
- Qian, C., Chételat, D., and Morris, C. Exploring the power of graph neural networks in solving linear optimization problems. In *AISTATS*, 2024. 1, 2
- Rampásek, L., Galkin, M., Dwivedi, V. P., Luu, A. T., Wolf, G., and Beaini, D. Recipe for a general, powerful, scalable graph transformer. *Neural Information Processing Systems*, 2022. 3
- Rontsis, N., Goulart, P., and Nakatsukasa, Y. Efficient semidefinite programming with approximate admm. *Journal of Optimization Theory and Applications*, 192(1): 292–320, 2022. 21
- Sato, R., Yamada, M., and Kashima, H. Random features strengthen graph neural networks. In *Proceedings of the 2021 SIAM international conference on data mining (SDM)*, pp. 333–341. SIAM, 2021. 3
- Scarselli, F., Gori, M., Tsoi, A. C., Hagenbuchner, M., and Monfardini, G. The graph neural network model. *IEEE Transactions on Neural Networks*, 20(1):61–80, 2008. 1, 2
- Southern, J., Eitan, Y., Bar-Shalom, G., Bronstein, M. M., Maron, H., and Frasca, F. Balancing efficiency and expressiveness: Subgraph gnns with walk-based centrality. In *International Conference on Machine Learning*. PMLR, 2025. 3
- Tanneau, M. and Van Hentenryck, P. Dual lagrangian learning for conic optimization. *Neural Information Processing Systems*, 2024. 14

- Vandenberghe, L. and Boyd, S. P. Semidefinite programming. *SIAM Rev.*, 38(1):49–95, 1996. 1
- Velickovic, P., Cucurull, G., Casanova, A., Romero, A., Liò, P., and Bengio, Y. Graph attention networks. In *International Conference on Learning Representations*, 2018. 2
- Wang, Y., Lan, H., and Ye, Y. A tuning-free primal-dual splitting algorithm for large-scale semidefinite programming. *arXiv preprint arXiv:2402.00311*, 2024. 2, 5, 20, 21, 22, 23, 25
- Weisfeiler, B. and Leman, A. The reduction of a graph to canonical form and the algebra which appears therein. *Nauchno-Tekhnicheskaya Informatsia, Series*, 2(9):12–16, 1968. 3, 14
- Wen, Z., Goldfarb, D., and Yin, W. Alternating direction augmented lagrangian methods for semidefinite programming. *Mathematical Programming Computation*, 2(3): 203–230, 2010. 5, 7
- Wright, S. Optimization for data analysis, 2022. 21
- Wu, C., Chen, Q., Wang, A., Ding, T., Sun, R., Yang, W., and Shi, Q. On representing convex quadratically constrained quadratic programs via graph neural networks. *TMLR*, 2025. 2
- Xu, K., Hu, W., Leskovec, J., and Jegelka, S. How powerful are graph neural networks? In *International Conference on Learning Representations*, 2019. 1, 2, 3, 29
- Yau, M., Karalias, N., Lu, E., Xu, J., and Jegelka, S. Are graph neural networks optimal approximation algorithms? *Neural Information Processing Systems*, 2024. 2, 33, 44
- You, J., Gomes-Selman, J. M., Ying, R., and Leskovec, J. Identity-aware graph neural networks. In *AAAI*, volume 35, 2021. 3
- Zhang, B., Feng, G., Du, Y., He, D., and Wang, L. A complete expressiveness hierarchy for subgraph gnns via subgraph weisfeiler-lehman tests. In *International Conference on Machine Learning*. PMLR, 2023. 3
- Zhang, M. and Li, P. Nested graph neural networks. In *Advances in Neural Information Processing Systems*, 2021. 3
- Zhao, L., Jin, W., Akoglu, L., and Shah, N. From stars to subgraphs: Uplifting any GNN with local structure awareness. In *International Conference on Learning Representations*, 2022. 3

A. Limitations and future directions

A key limitation of our study is that the required 2-FWL-level expressivity comes at a nontrivial computational cost, as it requires considering every variable pair. In addition, our theoretical results demonstrate only that there exist parameter assignments that allow recovery of the optimal low-norm solution, and do not guarantee that gradient-descent-based methods will converge. In addition, our expressivity guarantees are non-uniform, in the sense that we only show the existence of a parameter for a given SDP problem instance size and do not shed light on the architectures’ size-generalization capabilities (Levin et al., 2025).

Looking forward, we identify three key directions. First, we propose designing efficient architectures for specific SDP subclasses where lower-order expressivity suffices. Second, the rigorous alignment between our architecture and the PDHG algorithm opens a promising avenue for self-supervised primal-dual learning (Park & Van Hentenryck, 2023; Tanneau & Van Hentenryck, 2024). Rather than relying on supervised learning, future models could be trained by directly minimizing algorithmic residuals or duality gaps. Finally, we aim to extend this framework to broader conic programs and integrate these learned components into combinatorial optimization pipelines, such as branch-and-cut frameworks.

B. More background

B.1. Higher order Weisfeiler–Leman

We introduce some standard notations first. For $n \geq 1$, let $[n] := \{1, \dots, n\} \subset \mathbb{N}$. We use $\{\{\dots\}\}$ to denote multisets, i.e., the generalization of sets allowing for multiple instances of each of its elements. A *graph* G is a pair $(V(G), E(G))$ with *finite* sets of *vertices* $V(G)$ and *edges* $E(G) \subseteq \{\{u, v\} \subseteq V(G) \mid u \neq v\}$. For ease of notation, we denote the edge $\{u, v\}$ in $E(G)$ by (u, v) or (v, u) . A *labeled graph* G is a triple (V, E, l) with *(node) coloring* or *label function* $l: V(G) \rightarrow \mathbb{N}$. The *neighborhood* of v in G is denoted by $N(v) := \{u \in V(G) \mid \{v, u\} \in E(G)\}$.

Two graphs G and H are *isomorphic* and we write $G \simeq H$ if there exists a bijection $\varphi: V(G) \rightarrow V(H)$ that preserves the adjacency relation, i.e., $(u, v) \in E(G) \iff (\varphi(u), \varphi(v)) \in E(H)$. Then φ is an *isomorphism* between G and H . In the case of labeled graphs, we additionally require that $l(v) = l(\varphi(v))$ for v in $V(G)$. We further define the atomic type $\text{atp}: V(G)^k \rightarrow \mathbb{N}$ such that $\text{atp}(\mathbf{v}) = \text{atp}(\mathbf{w})$ for $\mathbf{v}, \mathbf{w} \in V(G)^k$ if and only if the mapping $\varphi: V(G)^k \rightarrow V(G)^k$ where $v_i \rightarrow w_i$ induces a partial isomorphism, i.e., we have $v_i = v_j \iff w_i = w_j$ and $(v_i, v_j) \in E(G) \iff (\varphi(v_i), \varphi(v_j)) \in E(G)$.

The 1-WL or color refinement is a simple heuristic for the graph isomorphism problem, originally proposed by Weisfeiler & Leman (1968). Intuitively, the algorithm determines if two graphs are non-isomorphic by iteratively coloring or labeling vertices. Given an initial coloring or labeling of the vertices of both graphs, e.g., their degree or application-specific information, in each iteration, two vertices with the same label get different labels if the number of identically labeled neighbors is not equal. If, after some iteration, the number of vertices annotated with a specific label is different in both graphs, the algorithm terminates and a stable coloring (partition) is obtained. We can then conclude that the two graphs are not isomorphic. It is easy to see that the algorithm cannot distinguish all non-isomorphic graphs (Cai et al., 1992).

Formally, let $G = (V, E, l)$ be a labeled graph. In each iteration $t + 1$, the 1-WL computes a node coloring for node v as $\mathbf{c}_v^t$, which depends on the coloring of the neighbors. That is,

$$\mathbf{c}_v^{t+1} := \text{hash}(\mathbf{c}_v^t, \{\{\mathbf{c}_u^t \mid u \in N(v)\}\}),$$

where hash injectively maps the above pair to a unique natural number, which has not been used in previous iterations. In iteration 0, the coloring $\mathbf{c}_v^0 := l(v)$.

To test if two graphs G and H are non-isomorphic, we run the above algorithm in “parallel” on both graphs. If the two graphs have a different number of vertices colored c in $\mathbb{N}$ at some iteration, the 1-WL *distinguishes* the graphs as non-isomorphic. Moreover, if

$$\mathbf{c}_v^t = \mathbf{c}_w^t \iff \mathbf{c}_v^{t+1} = \mathbf{c}_w^{t+1},$$

for all vertices v and w in $V(G)$, the algorithm terminates. For such t , we define the *stable coloring* $\mathbf{c}_v^\infty = \mathbf{c}_v^t$ for v in $V(G)$.

Due to the shortcomings of the 1-WL or color refinement in distinguishing non-isomorphic graphs, several researchers (Babai, 1979; Immerman & Lander, 1990; Cai et al., 1992; Grohe, 2017; Morris et al., 2019; Grohe, 2021) devised a more powerful generalization of the former, today known as the *k-dimensional Weisfeiler-Leman algorithm* (*k*-WL). There are two versions

of k -WL algorithm, known as *Folklore Weisfeiler-Leman* (k -FWL) and *Oblivious Weisfeiler-Leman* that is often used in the graph learning community, both will be discussed in this section.

Intuitively, to surpass the limitations of the 1-WL, the k -FWL algorithm colors subgraphs instead of a single node. More precisely, given a graph G , it colors the tuples from $V(G)^k$ for $k \geq 2$ instead of the vertices. By defining a neighborhood between these tuples, we can define a coloring similar to the 1-WL. Formally, let G be a graph, and let $k \geq 2$. In each iteration $t \geq 0$, the algorithm, similarly to the 1-WL, computes a *coloring* c_v^t for a tuple v . In the first iteration, $t = 0$, the tuples $v, w \in V(G)^k$ get the same color if they have the same atomic type, i.e., $c_v^0 := \text{atp}(v)$. The update of the coloring is defined by

$$c_v^{t+1} := \text{hash} \left(c_v^t, \left\{ \left(c_{\phi_1(v,w)}^t, \dots, c_{\phi_k(v,w)}^t \right) \mid w \in V(G) \right\} \right),$$

where

$$\phi_j(v, w) := (v_1, \dots, v_{j-1}, w, v_{j+1}, \dots, v_k).$$

That is, $\phi_j(v, w)$ replaces the j -th component of the tuple v with the node w . Hence, two tuples are *adjacent* or j -*neighbors*, with respect to a node w , if they are different in the j th component (or equal, in the case of self-loops).

k -WL differs from k -FWL in the way the j -neighbors are collected and aggregated. Specifically, the update of k -WL is

$$c_v^{t+1} := \text{hash} \left(c_v^t, \left(\{c_{\phi_1(v,w)}^t \mid w \in V(G)\}, \dots, \{c_{\phi_k(v,w)}^t \mid w \in V(G)\} \right) \right).$$

Again, we run the k -WL or k -FWL algorithm until convergence, i.e.,

$$c_v^t = c_w^t \iff c_v^{t+1} = c_w^{t+1},$$

for all v and w in $V(G)^k$ holds, and call the partition of $V(G)^k$ induced by c^t the stable partition. For such t , we define the stable coloring $c_v^\infty = c_v^t$ for v in $V(G)^k$. Hence, two tuples v and w with the same color in iteration $(t-1)$ get different colors in iteration t if there exists a j in $[k]$ such that the number of j -neighbors of v and w , respectively, colored with a certain color is different.

To test whether two graphs G and H are non-isomorphic, we run the k -FWL or k -WL in “parallel” on both graphs. Then, if the two graphs have a different number of vertices colored c in $\mathbb{N}$, the k -WL *distinguishes* the graphs as non-isomorphic.

Finally, we characterize the relative expressive power of these algorithms. We say that an algorithm $\mathcal{A}$ *refines* $\mathcal{B}$, denoted $\mathcal{A} \sqsubseteq \mathcal{B}$, if every pair of graphs distinguished by $\mathcal{B}$ is also distinguished by $\mathcal{A}$. And we say an algorithm $\mathcal{A}$ *strictly refines* $\mathcal{B}$, denoted $\mathcal{A} \sqsubset \mathcal{B}$ if there exists a pair of graphs that cannot be distinguished by $\mathcal{B}$ but $\mathcal{A}$. Observing the update rules, k -FWL aggregates the joint structure of neighbors across all tuple positions, whereas k -WL aggregates each position independently. Consequently, k -FWL is strictly more expressive than k -WL for $k \geq 2$. In fact, it has been established that k -FWL possesses the same expressive power as $(k+1)$ -WL (Cai et al., 1992; Grohe, 2017). Since increasing the dimension k strictly increases discriminative power (Cai et al., 1992), we obtain the following hierarchy of expressivity:

$$\dots \equiv (k+1)\text{-FWL} \sqsubset (k+1)\text{-WL} \equiv k\text{-FWL} \sqsubset k\text{-WL} \equiv (k-1)\text{-FWL} \sqsubset \dots$$

C. Omitted proofs

Proposition C.1 (Restatement of Proposition 1.1). *The SDP problem defined in Equation (1) has a unique primal solution X^* with the minimum Frobenius norm $\|X^*\|_F^2$.*

Proof for Proposition 1.1. We prove by contradiction. Suppose there exists another feasible solution $X' \neq X^* \in \mathbb{S}_+^n$, such that $\mathcal{A}(X') = b$, and $\langle C, X' \rangle = \langle C, X^* \rangle$, and $\|X'\|_F^2 = \|X^*\|_F^2$. Note that we are not assuming $\|X'\|_F^2 < \|X^*\|_F^2$ or $\langle C, X' \rangle < \langle C, X^* \rangle$, otherwise the X' will replace X^* to be the optimal solution with least Frobenius norm. Assuming the existence of X' , we can find a third solution $X'' = \frac{1}{2}(X^* + X')$, which is feasible and optimal, in that:

- $X'' \in \mathbb{S}_+^n$, as $X^* \in \mathbb{S}_+^n$ and $X' \in \mathbb{S}_+^n$, and the PSD cone is a convex cone.
- $\mathcal{A}(X'') = \frac{1}{2}\mathcal{A}(X^*) + \frac{1}{2}\mathcal{A}(X') = b$, i.e., X'' is feasible w.r.t. the constraints.

- $\langle C, X'' \rangle = \frac{1}{2} \langle C, X^* \rangle + \frac{1}{2} \langle C, X' \rangle = \langle C, X^* \rangle = \langle C, X' \rangle$, i.e., X'' has the same objective value.

Besides, $\|X''\|_F^2 = \frac{1}{4} \|X^* + X'\|_F^2 \leq \frac{\|X^*\|_F^2 + \|X'\|_F^2}{2} = \|X^*\|_F^2 = \|X'\|_F^2$, and equality holds only when $X^* = X'$. Since we assume $X^* \neq X'$, this implies X'' has a strictly lower Frobenius norm than X^* and X' , contradicting the assumption that X^* and X' are minimum norm solutions. Thus, the minimum norm solution must be unique. $\square$

C.1. Proof for failure of VC-WL and VC-2-WL

Proposition C.2 (Restatement of Proposition 2.1). *The VC-WL fails to represent linear SDP solutions. That is, there exist instances where the stable coloring of the VC-WL satisfies $v_{ij}^\infty = v_{pq}^\infty$ for distinct variable indices (i, j) and (p, q) , yet the entries in the unique optimal solution differ; i.e., $X_{ij}^* \neq X_{pq}^*$.*

Proof for Proposition 2.1. We prove by providing the following counter-example

$$C = \begin{bmatrix} 1 & 1 & 0 \\ 1 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \quad A_1 = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}, \quad A_2 = \begin{bmatrix} 0 & 0 & 1 \\ 0 & 0 & 0 \\ 1 & 0 & 0 \end{bmatrix}, \quad b = [1 \quad 1]$$

on which VC-WL converges to colors $\begin{bmatrix} a & b & c \\ b & a & d \\ c & d & a \end{bmatrix}$, where $\{a, b, c, \dots\}$ are from an alphabet to represent colors. Note that the diagonal entries $(1, 1)$ and $(3, 3)$ ⁵ are assigned the same color a . However, the unique optimal solution yields distinct values $X_{11}^* \approx 0.707$ and $X_{33}^* \approx 0.354$. VC-WL fails because it treats variables independently and ignores the matrix structure. $\square$

Remark C.3. From a measure-theoretic perspective, one might argue that a specific counterexample could merely reside on a set of measure zero, thereby still allowing a model to maintain universal approximation *almost everywhere*. However, this defense is inadequate for SDP relaxations of CO problems. First, the input distributions of CO problems are not uniformly drawn from $\mathbb{R}^{n \times n}$. Rather, they are heavily concentrated on specific discrete topological structures. Second, our identified failure cases are not isolated points. As established in Lemma C.24, scaling any failing instance by a scalar $\alpha > 0$ generates an uncountable infinite set (a ray) of failures. This specific construction represents just one of many systematic methods to expose these representational blind spots. Ultimately, the empirical collapse of the less expressive models observed in Table 1 confirms that this theoretical failure translates into a severe, practical expressivity limitation, rather than a negligible mathematical anomaly.

Proposition C.4 (Restatement of Proposition 2.2). *The VC-2-WL fails to represent linear SDP solutions. That is, there exist instances where the stable colors under VC-2-WL satisfy $v_{ij}^\infty = v_{pq}^\infty$ for distinct indices (i, j) and (p, q) , yet the unique optimal solution entries differ $X_{ij}^* \neq X_{pq}^*$.*

Proof for Proposition 2.2. We prove this by construction, using a counterexample in which the objective matrix C forms a Latin square. Consider the following SDP instance with $n = 6$,

$$C = \begin{bmatrix} 1 & 2 & 3 & 4 & 5 & 6 \\ 2 & 1 & 4 & 5 & 6 & 3 \\ 3 & 4 & 1 & 6 & 2 & 5 \\ 4 & 5 & 6 & 1 & 3 & 2 \\ 5 & 6 & 2 & 3 & 1 & 4 \\ 6 & 3 & 5 & 2 & 4 & 1 \end{bmatrix}, \quad A_1 = I_6, \quad b = [1].$$

We focus on the entries $(1, 5)$ and $(2, 4)$. First, observe that their initial features are identical, i.e., $C_{15} = C_{24} = 5$, and both are non-diagonal entries. Secondly, observe that every row and column in C contains the set of values $\{1, \dots, 6\}$. Consequently, for any index (u, v) , the row and column multisets are identical, i.e.,

$$\{\{v_{uk}^0 \mid k \in [n]\}\} = \{\{v_{kv}^0 \mid k \in [n]\}\} = \{\{1, 2, 3, 4, 5, 6\}\}.$$

⁵All matrix indices in this paper are 1-indexed.

Since VC-2-WL aggregates row and column multisets independently as in Equation (3), and both the node features and their neighborhoods are identical, the algorithm cannot distinguish $(1, 5)$ from $(2, 4)$ at initialization or any subsequent iteration. Thus, $v_{15}^\infty = v_{24}^\infty$ under VC-2-WL algorithm. However, the unique minimum-norm solution yields $X_{15}^* \approx -0.115$ and $X_{24}^* \approx -0.172$. Hence, the inability of VC-2-WL to distinguish these variables indicates it is insufficient for SDPs. $\square$

C.2. Proof for expressivity hierarchy

We first compare VC-WL and VC-2-WL by showing the following result.

Proposition C.5. *The VC-2-WL algorithm strictly refines VC-WL, denoted as*

$$\text{VC-2-WL} \sqsubset \text{VC-WL}.$$

Proof. Observing from the update functions of Equation (2) and Equation (3), we notice that the update of constraint nodes is exactly the same. Now, consider the variable nodes. Observe that the information aggregated by the VC-WL hash function for a variable v_{ij} consists solely of its constraint neighbors. By definition, the corresponding input for VC-2-WL includes this constraint information as a subset, strictly augmented by the structural row and column multisets.

We now formally prove $\text{VC-2-WL} \sqsubseteq \text{VC-WL}$, that is, for any iteration t , if VC-2-WL fails to distinguish v_{ij}^t and v_{pq}^t , so cannot VC-WL. First, for $t = 0$, the color v_{ij}^0 is basically the encoding of the problem C_{ij} , therefore if for VC-2-WL: $v_{ij}^0 = v_{pq}^0$, it also holds for VC-WL. Then, for all $t > 0$, we have, by definition of VC-2-WL:

$$\begin{aligned} v_{ij}^t &= v_{pq}^t \\ \implies &\text{hash}\left(v_{ij}^{t-1}, \{\{v_{uj}^{t-1} \mid u \in [n]\}, \{v_{iu}^{t-1} \mid u \in [n]\}, \{(A_{k,ij}, c_k^{t-1}) \mid k \in N(ij)\}\}\right) \\ &= \text{hash}\left(v_{pq}^{t-1}, \{\{v_{uq}^{t-1} \mid u \in [n]\}, \{v_{pu}^{t-1} \mid u \in [n]\}, \{(A_{k,pq}, c_k^{t-1}) \mid k \in N(pq)\}\}\right) \\ \implies &\text{hash}\left(v_{ij}^{t-1}, \{\{(A_{k,ij}, c_k^{t-1}) \mid k \in N(ij)\}\}\right) \\ &= \text{hash}\left(v_{pq}^{t-1}, \{\{(A_{k,pq}, c_k^{t-1}) \mid k \in N(pq)\}\}\right). \end{aligned}$$

Since this corresponds exactly to the update rule of VC-WL, it follows that VC-WL also cannot distinguish them. Thus, $\text{VC-2-WL} \sqsubseteq \text{VC-WL}$.

To show the strict refinement, we pick the example in the proof of Proposition 2.1 where VC-WL fails to distinguish the variables X_{11}^* and X_{33}^* . In contrast, VC-2-WL converges to $\begin{bmatrix} a & b & c \\ b & e & d \\ c & d & f \end{bmatrix}$, where $\{a, b, c, \dots\}$ are from an alphabet to represent colors. Here, the diagonal entries are distinguished ($a \neq f$), allowing the VC-2-WL algorithm to distinguish the solutions.

Therefore, $\text{VC-2-WL} \sqsubset \text{VC-WL}$. $\square$

Similarly, $\text{VC-2-FWL} \sqsubset \text{VC-WL}$ can be proven in exactly the same way with the same example; we omit the proof.

Next, we show the hierarchy between VC-2-FWL+ and VC-2-WL.

Proposition C.6. *The VC-2-FWL+ algorithm strictly refines VC-2-WL, denoted as*

$$\text{VC-2-FWL+} \sqsubset \text{VC-2-WL}.$$

Proof. The update of constraint nodes is exactly the same for VC-2-WL and VC-2-FWL+. Now, we consider the variable nodes.

We first prove $\text{VC-2-FWL+} \sqsubseteq \text{VC-2-WL}$. First, for $t = 0$, the colors v_{ij}^0 is basically the encoding of the problem C_{ij} , therefore if for VC-2-FWL+: $v_{ij}^0 = v_{pq}^0$, it also holds for VC-2-WL. For any $t > 0$, we want to show that if for VC-2-FWL+:

$v_{ij}^t = v_{pq}^t$ holds, it also holds for VC-2-WL update. Specifically, for VC-2-FWL+:

$$\begin{aligned}
 v_{ij}^t &= v_{pq}^t \\
 &\Rightarrow \text{hash}\left(v_{ij}^{t-1}, \{\{(v_{uj}^{t-1}, v_{iu}^{t-1}) \mid u \in [n]\}\}, \{(A_{k,ij}, c_k^{t-1}) \mid k \in N(ij)\}\}\right) \\
 &= \text{hash}\left(v_{pq}^{t-1}, \{\{(v_{uq}^{t-1}, v_{pu}^{t-1}) \mid u \in [n]\}\}, \{(A_{k,pq}, c_k^{t-1}) \mid k \in N(pq)\}\}\right) \\
 &\Rightarrow v_{ij}^{t-1} = v_{pq}^{t-1} \wedge \\
 &\quad \{\{(v_{uj}^{t-1}, v_{iu}^{t-1}) \mid u \in [n]\}\} = \{\{(v_{uq}^{t-1}, v_{pu}^{t-1}) \mid u \in [n]\}\} \wedge \\
 &\quad \{(A_{k,ij}, c_k^{t-1}) \mid k \in N(ij)\}\} = \{(A_{k,pq}, c_k^{t-1}) \mid k \in N(ij)\}\} \\
 &\Rightarrow v_{ij}^{t-1} = v_{pq}^{t-1} \wedge \\
 &\quad \{\{v_{uj}^{t-1} \mid u \in [n]\}\} = \{\{v_{uq}^{t-1} \mid u \in [n]\}\} \wedge \\
 &\quad \{\{v_{iu}^{t-1} \mid u \in [n]\}\} = \{\{v_{pu}^{t-1} \mid u \in [n]\}\} \wedge \\
 &\quad \{(A_{k,ij}, c_k^{t-1}) \mid k \in N(ij)\}\} = \{(A_{k,pq}, c_k^{t-1}) \mid k \in N(ij)\}\} \\
 &\Rightarrow \text{hash}\left(v_{ij}^{t-1}, \{\{v_{uj}^{t-1} \mid u \in [n]\}\}, \{\{v_{iu}^{t-1} \mid u \in [n]\}\}, \{(A_{k,ij}, c_k^{t-1}) \mid k \in N(ij)\}\}\right) \\
 &= \text{hash}\left(v_{pq}^{t-1}, \{\{v_{uq}^{t-1} \mid u \in [n]\}\}, \{\{v_{pu}^{t-1} \mid u \in [n]\}\}, \{(A_{k,pq}, c_k^{t-1}) \mid k \in N(pq)\}\}\right)
 \end{aligned}$$

thus with VC-2-WL update they also have the same color. The implication holds because the tuples are ordered, which ensures that their marginal entries form multisets that are respectively identical.

We then prove strict refinement by giving an example in the proof of Proposition 2.2 where VC-2-WL fails. In this example, the converged VC-2-FWL+ assigns different colors to every primal variable, except symmetric ones, effectively distinguishing the variables in the solution.

Therefore, VC-2-FWL+ $\sqsubset$ VC-2-WL. □

Proposition C.7. *The VC-2-FWL+ algorithm strictly refines VC-2-FWL, denoted as*

$$\text{VC-2-FWL+} \sqsubset \text{VC-2-FWL}.$$

Proof. We first prove VC-2-FWL+ $\sqsubseteq$ VC-2-FWL.

First, for $t = 0$, the colors initializations are the same, therefore if for VC-2-FWL+: $v_{ij}^0 = v_{pq}^0$, it also holds for VC-2-FWL. For some $t > 0$, we want to show that if for VC-2-FWL+: $v_{ij}^t = v_{pq}^t$ holds, it also holds for VC-2-FWL update. Specifically, for VC-2-FWL+:

$$\begin{aligned}
 v_{ij}^t &= v_{pq}^t \Rightarrow \\
 &\quad \text{hash}\left(v_{ij}^{t-1}, \{\{(v_{uj}^{t-1}, v_{iu}^{t-1}) \mid u \in [n]\}\}, \{(A_{k,ij}, c_k^{t-1}) \mid k \in N(ij)\}\}\right) \\
 &= \text{hash}\left(v_{pq}^{t-1}, \{\{(v_{uq}^{t-1}, v_{pu}^{t-1}) \mid u \in [n]\}\}, \{(A_{k,pq}, c_k^{t-1}) \mid k \in N(pq)\}\}\right) \\
 &\Rightarrow \text{hash}\left(v_{ij}^{t-1}, \{\{\{v_{uj}^{t-1}, v_{iu}^{t-1}\} \mid u \in [n]\}\}, \{(A_{k,ij}, c_k^{t-1}) \mid k \in N(ij)\}\}\right) \\
 &= \text{hash}\left(v_{pq}^{t-1}, \{\{\{v_{uq}^{t-1}, v_{pu}^{t-1}\} \mid u \in [n]\}\}, \{(A_{k,pq}, c_k^{t-1}) \mid k \in N(pq)\}\}\right).
 \end{aligned}$$

The last implication is straightforward because a 2-multiset is an unordered 2-tuple. The last equation represents the VC-2-FWL update, which cannot distinguish them either. This holds for all t , therefore by induction, VC-2-FWL+ $\sqsubseteq$ VC-2-FWL.

Next, we provide the strict refinement by providing an example where VC-2-FWL+ distinguishes a pair, where VC-2-FWL does not. Consider the example

$$C = \begin{bmatrix} 0 & 1 & 2 & 3 \\ 1 & 0 & 4 & 2 \\ 2 & 4 & 0 & 1 \\ 3 & 2 & 1 & 0 \end{bmatrix}, \quad A_1 = \mathbf{1}_{4 \times 4}, \quad b = [1].$$

We simply let $v_{ij}^0 = C_{ij}$. Consider the update for the indices (1, 2) and (3, 4). Under VC-2-FWL:

$$\begin{aligned} v_{12}^1 &:= \text{hash}\left(v_{12}^0, \{\{\{v_{12}^0, v_{11}^0\}, \{v_{22}^0, v_{12}^0\}, \{v_{32}^0, v_{13}^0\}, \{v_{42}^0, v_{14}^0\}\}\}, (1, c_1^0)\right) \\ &= \text{hash}\left(1, \{\{\{1, 0\}, \{0, 1\}, \{4, 2\}, \{2, 3\}\}\}, (1, c_1^0)\right), \end{aligned}$$

and

$$\begin{aligned} v_{34}^1 &:= \text{hash}\left(v_{34}^0, \{\{\{v_{14}^0, v_{31}^0\}, \{v_{24}^0, v_{32}^0\}, \{v_{34}^0, v_{33}^0\}, \{v_{44}^0, v_{34}^0\}\}\}, (1, c_1^0)\right) \\ &= \text{hash}\left(1, \{\{\{3, 2\}, \{2, 4\}, \{1, 0\}, \{0, 1\}\}\}, (1, c_1^0)\right). \end{aligned}$$

If we calculate the bins of 2-multisets, we notice they both have: two $\{0, 1\}$, one $\{2, 3\}$, one $\{2, 4\}$. Therefore, v_{12}^1 and v_{34}^1 take on the same color under VC-2-FWL. VC-2-FWL+ differs in that those 2-multisets become 2-tuples. Therefore, for VC-2-FWL+, v_{12}^1 update contains a tuple (2, 3) and a (4, 2), and v_{34}^1 contains a (3, 2) and a (2, 4), thus $v_{12}^1 \neq v_{34}^1$ under VC-2-FWL+. That means VC-2-FWL+ distinguishes v_{12} and v_{34} at the 1st iteration while VC-2-FWL not.

Therefore, VC-2-FWL+ $\sqsubset$ VC-2-FWL. $\square$

Finally, we show that VC-2-WL and VC-2-FWL are incomparable.

Proposition C.8. VC-2-WL and VC-2-FWL are incomparable in expressive power, denoted as

$$\text{VC-2-WL} \not\equiv \text{VC-2-FWL}.$$

Proof. We first show that there exists an example where VC-2-WL can distinguish a pair of variables and VC-2-FWL cannot. Consider the following example:

$$C = \begin{bmatrix} 1 & 0 & 4 & 2 \\ 0 & 1 & 2 & 3 \\ 4 & 2 & 1 & 0 \\ 2 & 3 & 0 & 1 \end{bmatrix}, \quad A_1 = \mathbf{1}_{4 \times 4}, \quad b = [1].$$

We simply let $v_{ij}^0 = C_{ij}$. Consider the update for the indices (1, 4) and (2, 3). Under VC-2-FWL:

$$\begin{aligned} v_{14}^1 &:= \text{hash}\left(v_{14}^0, \{\{\{v_{14}^0, v_{11}^0\}, \{v_{24}^0, v_{12}^0\}, \{v_{34}^0, v_{13}^0\}, \{v_{44}^0, v_{14}^0\}\}\}, (1, c_1^0)\right) \\ &= \text{hash}\left(2, \{\{\{2, 1\}, \{3, 0\}, \{0, 4\}, \{1, 2\}\}\}, (1, c_1^0)\right), \end{aligned}$$

and

$$\begin{aligned} v_{23}^1 &:= \text{hash}\left(v_{23}^0, \{\{\{v_{13}^0, v_{21}^0\}, \{v_{23}^0, v_{22}^0\}, \{v_{33}^0, v_{23}^0\}, \{v_{43}^0, v_{24}^0\}\}\}, (1, c_1^0)\right) \\ &= \text{hash}\left(2, \{\{\{4, 0\}, \{2, 1\}, \{1, 2\}, \{0, 3\}\}\}, (1, c_1^0)\right). \end{aligned}$$

Therefore $v_{14}^1 = v_{23}^1$ under VC-2-FWL. While for VC-2-WL update:

$$\begin{aligned} v_{14}^1 &:= \text{hash}\left(v_{14}^0, \{\{v_{14}^0, v_{24}^0, v_{34}^0, v_{44}^0\}, \{v_{11}^0, v_{12}^0, v_{13}^0, v_{14}^0\}\}, (1, c_1^0)\right) \\ &= \text{hash}\left(2, \{\{2, 3, 0, 1\}, \{1, 0, 4, 2\}\}, (1, c_1^0)\right), \end{aligned}$$

and

$$\begin{aligned} v_{23}^1 &:= \text{hash}\left(v_{23}^0, \{\{v_{13}^0, v_{23}^0, v_{33}^0, v_{43}^0\}, \{v_{21}^0, v_{22}^0, v_{23}^0, v_{24}^0\}\}, (1, c_1^0)\right) \\ &= \text{hash}\left(2, \{\{4, 2, 1, 0\}\}, \{\{0, 1, 2, 3\}\}, (1, c_1^0)\right) \end{aligned}$$

thus $v_{14}^1 \neq v_{23}^1$ under VC-2-WL.

Next, we show there exists an example where VC-2-FWL can distinguish a pair of variables and VC-2-WL cannot. We reuse the example in the proof of Proposition 2.2. VC-2-FWL effectively distinguishes all the variables except symmetric ones, and is able to distinguish variables (1, 5) and (2, 4) where VC-2-WL fails.

Therefore, VC-2-WL and VC-2-FWL are incomparable in expressive power. $\square$

Remark C.9. The incomparability mainly stems from the directness of VC-2-WL. If we define a variant of VC-2-WL, supposedly VC-2-WL $_{\text{agg}}$, where $\{\{v_{uj}^{t-1} \mid u \in [n]\}\}$ and $\{\{v_{iu}^{t-1} \mid u \in [n]\}\}$ are aggregated in an unordered way then used to update v_{ij} , it will not distinguish the variables v_{14} and v_{23} in the case above. However, it will be less expressive than VC-2-WL, and also fails on the example of Proposition 2.2. Therefore, it is not valuable to investigate such a VC-2-WL variant.

Remark C.10. Although VC-2-WL and VC-2-FWL are theoretically incomparable, with VC-2-WL possessing superior distinguishing power in specific cases, the optimization dynamics of SDPs are still inherently subsumed by VC-2-FWL rather than VC-2-WL. We will later demonstrate that the algorithmic trajectory of an iterative solver, including both intermediate steps and the final converged solution, is strictly refined by the VC-2-FWL process.

C.3. Convergence of PDHG algorithm on SDP with least-norm solution

As a prerequisite to establishing the expressivity of VC-2-FWL, we first rigorously demonstrate the convergence of the PDHG algorithm applied to the SDP problem with Frobenius norm regularization. First, we briefly introduce some conventional notations based on Eckstein & Bertsekas (1992); O'Connor & Vandenberghe (2020); Wang et al. (2024).

Let F be a set-valued operator which maps every $x \in \mathbb{R}^n$ to a subset of $\mathbb{R}^n$. The notation is useful for subgradient methods, as the subgradient of a non-smooth function is usually a set. The *graph* gr of an operator F is the set

$$\text{gr}(F) := \{(x, y) \in \mathbb{R}^n \times \mathbb{R}^n \mid y \in F(x)\}.$$

An operator F is *monotone* if

$$(y - \hat{y})^\top (x - \hat{x}) \geq 0, \quad \text{for all } (x, y), (\hat{x}, \hat{y}) \in \text{gr}(F).$$

A monotone operator is *maximal monotone* if its graph is not contained in the graph of another monotone operator. An operator F is μ -strongly monotone if and only if

$$(y - \hat{y})^\top (x - \hat{x}) \geq \mu(x - \hat{x})^\top (x - \hat{x}), \quad \text{for all } (x, y), (\hat{x}, \hat{y}) \in \text{gr}(F).$$

Given a positive scalar c and an operator F , the *resolvent* of an operator F is defined as $J_{cF} := (I + cF)^{-1}$ where I is identity mapping. $J_{cF}(x)$ is the set of all the values y satisfying

$$x - y \in cF(y). \tag{7}$$

The resolvent of the subdifferential of a *closed, convex and proper* function f is called the *proximal operator* of f , denoted $\text{prox}_{cf} = J_{c\partial f}$. Equation (7) is the optimality condition of the optimization problem

$$\text{prox}_{cf}(x) := \arg \min_y \left(cf(y) + \frac{1}{2} \|y - x\|_2^2 \right).$$

In plain language, the proximal operator finds an optimal solution that minimizes the function f without searching too far

from the input $\mathbf{x}$. The equivalence can be verified,

$$\begin{aligned}
 & \mathbf{0} \in \partial \left(cf(\mathbf{y}) + \frac{1}{2} \|\mathbf{y} - \mathbf{x}\|_2^2 \right) \\
 \implies & \mathbf{0} \in c\partial f(\mathbf{y}) + (\mathbf{y} - \mathbf{x}) \\
 \implies & \mathbf{x} \in c\partial f(\mathbf{y}) + \mathbf{y} \\
 \implies & \mathbf{x} \in (I + c\partial f)(\mathbf{y}) \\
 \implies & \mathbf{y} = (I + c\partial f)^{-1}(\mathbf{x}) = J_{c\partial f}(\mathbf{x}).
 \end{aligned} \tag{8}$$

Now let's state our problem setting. First, it is obvious that finding the primal solution $\mathbf{X}^*$ to

$$\begin{aligned}
 & \min_{\mathbf{X} \in \mathbb{S}_+^n} \langle \mathbf{C}, \mathbf{X} \rangle \\
 & \text{s.t. } \mathcal{A}(\mathbf{X}) = \mathbf{b}
 \end{aligned}$$

with the minimum Frobenius norm is equivalent to finding the primal solution to

$$\begin{aligned}
 & \min_{\mathbf{X} \in \mathbb{S}_+^n} \langle \mathbf{C}, \mathbf{X} \rangle + \frac{\varepsilon}{2} \langle \mathbf{X}, \mathbf{X} \rangle \\
 & \text{s.t. } \mathcal{A}(\mathbf{X}) = \mathbf{b}
 \end{aligned} \tag{9}$$

with arbitrarily small $\varepsilon \rightarrow 0$. The SDP with a Frobenius regularization term has a unique primal solution, as the objective is strictly convex and the feasible set is a convex set (Wright, 2022, p. 21), which aligns with our unique solution assumption Proposition 1.1.

Then, we introduce the PDHG method Chambolle & Pock (2016) on this problem. We extend the PDHG for linear SDP in Wang et al. (2024) by inserting the quadratic term $\frac{\varepsilon}{2} \langle \mathbf{X}, \mathbf{X} \rangle$. Consequently, instead of the PDHG update form in Wang et al. (2024, Algorithm 1), we have the following form

$$\begin{aligned}
 \mathbf{X}^{t+1} &:= \text{Proj}_{\mathbb{S}_+^n} \left[\frac{\mathbf{X}^t - \alpha_t \mathcal{A}^*(\mathbf{y}^t) - \alpha_t \mathbf{C}}{1 + \alpha_t \varepsilon} \right] \\
 \mathbf{y}^{t+1} &:= \mathbf{y}^t + \beta_t \mathcal{A}(\mathbf{X}^{t+1} + \theta_t (\mathbf{X}^{t+1} - \mathbf{X}^t)) - \beta_t \mathbf{b},
 \end{aligned} \tag{10}$$

where $\mathbf{X}^t, \mathbf{y}^t$ are primal-dual solutions at different iterations, and $\alpha_t, \beta_t, \theta_t$ are suitable step lengths.

The $\text{Proj}_{\mathbb{S}_+^n}$ projects the matrix into the PSD cone $\mathbb{S}_+^n$, specifically, by solving

$$\mathbf{X}^{t+1} := \arg \min_{\mathbf{X} \in \mathbb{S}_+^n} \left\| \mathbf{X} - \frac{\mathbf{X}^t - \alpha_t \mathcal{A}^*(\mathbf{y}^t) - \alpha_t \mathbf{C}}{1 + \alpha_t \varepsilon} \right\|_F^2.$$

It can be done by spectral decomposition (Boyd & Vandenberghe, 2004, p. 399) (Rontsis et al., 2022), see Algorithm 1 for detailed process.

Algorithm 1 Projection onto the PSD Cone

Require: A symmetric matrix $\mathbf{X} \in \mathbb{S}^n$

Ensure: The projected matrix $\mathbf{X}_{\mathbb{S}} := \arg \min_{\mathbf{X}' \in \mathbb{S}_+^n} \|\mathbf{X}' - \mathbf{X}\|_F$

- 1: Compute the spectral decomposition $\mathbf{X} = \mathbf{V} \mathbf{\Lambda} \mathbf{V}^\top$
 - 2: $\{\mathbf{\Lambda} := \text{diag}(\lambda_1, \dots, \lambda_n)\}$ is the diagonal matrix of eigenvalues.
 - 3: $\{\mathbf{V}\}$ is the orthonormal matrix of corresponding eigenvectors $\mathbf{v}_i$ as its columns.
 - 4: Construct the projected matrix by dropping the negative eigen-pairs:
 - 5: $\mathbf{X}_{\mathbb{S}} \leftarrow \sum_{i=1}^n \max(\lambda_i, 0) \cdot \mathbf{v}_i \mathbf{v}_i^\top$
 - 6: **return** $\mathbf{X}_{\mathbb{S}}$
-

As the constraints of the SDP problem are equality constraints, there is no projection on $\mathbf{y}^t \in \mathbb{R}^m$ needed.

Wang et al. (2024, Theorem 2.1) showed that the PDHG algorithm on linear SDP (without our Frobenius term) weakly converges (Bauschke & Combettes, 2020, p. 35) to an optimal point. We hereby prove a similar result for Equation (10).

Proposition C.11. *If the adjustment of $\{(\alpha_t, \theta_t, \beta_t)\}_t$ in Equation (10) follow*

$$\alpha_t \in (\alpha_{\min}, \alpha_{\max}), \quad \sum_{t=1}^{\infty} |\alpha_{t+1} - \alpha_t| < \infty, \quad \theta_t = \frac{\alpha_t}{\alpha_{t-1}}, \quad \alpha_t \beta_t = R,$$

where $0 < \alpha_{\min} \leq \alpha_{\max} < \infty$ and $R < \frac{1}{\lambda_{\max}(\mathcal{A}^* \mathcal{A})}$. Then PDHG algorithm in Equation (10) weakly converges to $(\mathbf{X}^*, \mathbf{y}^*)$ such that $0 \in \mathbf{C} + \varepsilon \mathbf{X}^* + \partial_{\mathbf{X}} \mathbb{I}_{\mathbb{S}_+^n}(\mathbf{X}^*) + \partial_{\mathbf{X}} \mathbb{I}_{=\mathbf{b}}(\mathcal{A}(\mathbf{X}^*))$, for any $\varepsilon > 0$.

Proof. Our proof establishes convergence by framing the algorithm as an instance of non-stationary Douglas-Rachford Splitting (DRS), following the methodology of Wang et al. (2024). We adapt their derivation to account for the Frobenius regularization term, which alters the primal variable update.

We rewrite Equation (9) into the following non-smooth optimization problem:

$$\min_{\mathbf{X}} \quad \langle \mathbf{C}, \mathbf{X} \rangle + \frac{\varepsilon}{2} \langle \mathbf{X}, \mathbf{X} \rangle + \mathbb{I}_{\mathbb{S}_+^n}(\mathbf{X}) + \mathbb{I}_{=\mathbf{b}}(\mathcal{A}(\mathbf{X})), \quad (11)$$

with indicator function $\mathbb{I}_{\mathbb{S}_+^n}(\mathbf{X}) = 0$ if $\mathbf{X} \in \mathbb{S}_+^n$ otherwise $+\infty$, and $\mathbb{I}_{=\mathbf{b}}(\mathbf{y}) = 0$ if $\mathbf{y} = \mathbf{b}$ and $+\infty$ otherwise.

To apply DRS, we introduce an auxiliary variable $\hat{\mathbf{X}}$ (constrained to be zero) and a linear operator $\mathcal{T}: \mathbb{S}^n \rightarrow \mathbb{R}^m$. The regularized problem in Equation (11) is reformulated as

$$\min_{\mathbf{X}, \hat{\mathbf{X}}} \quad \langle \mathbf{C}, \mathbf{X} \rangle + \frac{\varepsilon}{2} \langle \mathbf{X}, \mathbf{X} \rangle + \mathbb{I}_{\mathbb{S}_+^n}(\mathbf{X}) + \mathbb{I}_{=0}(\hat{\mathbf{X}}) + \mathbb{I}_{=\mathbf{b}}(\mathcal{A}(\mathbf{X}) + \mathcal{T}(\hat{\mathbf{X}})).$$

As the problem is convex, it is equivalent to finding the first-order optimality condition:

$$\text{Find } \begin{pmatrix} \mathbf{X} \\ \hat{\mathbf{X}} \end{pmatrix} \quad \text{s.t.} \quad 0 \in \underbrace{\begin{pmatrix} \mathbf{C} + \varepsilon \mathbf{X} \\ 0 \end{pmatrix} + \begin{pmatrix} \partial_{\mathbf{X}} \mathbb{I}_{\mathbb{S}_+^n}(\mathbf{X}) \\ 0 \end{pmatrix}}_{B = \partial f(\mathbf{X}, \hat{\mathbf{X}})} + \underbrace{\begin{pmatrix} 0 \\ \partial_{\hat{\mathbf{X}}} \mathbb{I}_{=0}(\hat{\mathbf{X}}) \end{pmatrix} + \begin{pmatrix} \partial_{\mathbf{X}} \mathbb{I}_{=\mathbf{b}}(\mathcal{A}(\mathbf{X}) + \mathcal{T}(\hat{\mathbf{X}})) \\ \partial_{\hat{\mathbf{X}}} \mathbb{I}_{=\mathbf{b}}(\mathcal{A}(\mathbf{X}) + \mathcal{T}(\hat{\mathbf{X}})) \end{pmatrix}}_{A = \partial g(\mathbf{X}, \hat{\mathbf{X}})}, \quad (12)$$

Operators A and B are maximally monotone. Notably, due to the regularization term $\frac{\varepsilon}{2} \langle \mathbf{X}, \mathbf{X} \rangle$, the operator B in our case is ε -strongly monotone with respect to $\mathbf{X}$. Following Wang et al. (2024) and applying non-stationary DRS to Equation (12) yields the following fixed-point iterations:

$$\begin{pmatrix} \mathbf{X}^t \\ \hat{\mathbf{X}}^t \end{pmatrix} = J_{\alpha_{t-1} \partial f} \left(\begin{pmatrix} \mathbf{Z}^t \\ \hat{\mathbf{Z}}^t \end{pmatrix} \right) \quad (13)$$

$$\begin{pmatrix} \mathbf{X}^{t+\frac{1}{2}} \\ \hat{\mathbf{X}}^{t+\frac{1}{2}} \end{pmatrix} = J_{\alpha_t \partial g} \left(\begin{pmatrix} \mathbf{X}^t \\ \hat{\mathbf{X}}^t \end{pmatrix} + \theta_t \left(\begin{pmatrix} \mathbf{X}^t \\ \hat{\mathbf{X}}^t \end{pmatrix} - \begin{pmatrix} \mathbf{Z}^t \\ \hat{\mathbf{Z}}^t \end{pmatrix} \right) \right) \quad (14)$$

$$\begin{pmatrix} \mathbf{Z}^{t+1} \\ \hat{\mathbf{Z}}^{t+1} \end{pmatrix} = \begin{pmatrix} \mathbf{X}^{t+\frac{1}{2}} \\ \hat{\mathbf{X}}^{t+\frac{1}{2}} \end{pmatrix} + \theta_t \left(\begin{pmatrix} \mathbf{Z}^t \\ \hat{\mathbf{Z}}^t \end{pmatrix} - \begin{pmatrix} \mathbf{X}^t \\ \hat{\mathbf{X}}^t \end{pmatrix} \right). \quad (15)$$

While the overall structure matches the standard DRS form in Wang et al. (2024), the specific update in Equation (13) differs. In our case, Equation (13), by definition of the resolvent, can be simplified to

$$\begin{aligned} \mathbf{X}^t &= \arg \min_{\mathbf{X}} \left\{ \langle \mathbf{C}, \mathbf{X} \rangle + \frac{\varepsilon}{2} \langle \mathbf{X}, \mathbf{X} \rangle + \mathbb{I}_{\mathbb{S}_+^n}(\mathbf{X}) + \frac{1}{2\alpha_{t-1}} \|\mathbf{X} - \mathbf{Z}^t\|_F^2 \right\} \\ \hat{\mathbf{X}}^t &= 0 \end{aligned} \quad (16)$$

Equation (14) follows Wang et al. (2024), and we obtain:

$$\begin{aligned} \mathbf{y}^t &= \arg \min_{\mathbf{y}} \left\{ \mathbb{I}_{=\mathbf{b}}(\mathbf{y}) - \left\langle \mathcal{A}(\mathbf{X}^t + \theta_t(\mathbf{X}^t - \mathbf{Z}^t)) - \theta_t \mathcal{T}(\hat{\mathbf{Z}}^t), \mathbf{y} \right\rangle \right. \\ &\quad \left. + \frac{\alpha_t}{2} (\|\mathcal{A}^*(\mathbf{y})\|_F^2 + \|\mathcal{T}^*(\mathbf{y})\|_F^2) \right\} \end{aligned} \quad (17)$$

$$\mathbf{X}^{t+\frac{1}{2}} = \mathbf{X}^t + \theta_t(\mathbf{X}^t - \mathbf{Z}^t) - \alpha_t \mathcal{A}^*(\mathbf{y}^t) \quad (18)$$

$$\hat{\mathbf{X}}^{t+\frac{1}{2}} = -\theta_t \hat{\mathbf{Z}}^t - \alpha_t \mathcal{T}^*(\mathbf{y}^t), \quad (19)$$

Subsequently, Equation (15) can be simplified to

$$\mathbf{Z}^{t+1} = \mathbf{X}^t - \alpha_t \mathcal{A}^*(\mathbf{y}^t) \quad (20)$$

$$\hat{\mathbf{Z}}^{t+1} = -\alpha_t \mathcal{T}^*(\mathbf{y}^t). \quad (21)$$

Substituting Equation (20) into Equation (16), we get

$$\begin{aligned} \mathbf{X}^{t+1} &:= \arg \min_{\mathbf{X}} \left\{ \langle \mathbf{C}, \mathbf{X} \rangle + \frac{\varepsilon}{2} \langle \mathbf{X}, \mathbf{X} \rangle + \mathbb{I}_{\mathbb{S}_+^n}(\mathbf{X}) + \frac{1}{2\alpha_t} \left\| \mathbf{X} - (\mathbf{X}^t - \alpha_t \mathcal{A}^*(\mathbf{y}^t)) \right\|_F^2 \right\} \\ &= \arg \min_{\mathbf{X}} \left\{ \mathbb{I}_{\mathbb{S}_+^n}(\mathbf{X}) + \left(\frac{\varepsilon}{2} + \frac{1}{2\alpha_t} \right) \langle \mathbf{X}, \mathbf{X} \rangle + \left\langle \mathbf{C} - \frac{1}{\alpha_t} (\mathbf{X}^t - \alpha_t \mathcal{A}^*(\mathbf{y}^t)), \mathbf{X} \right\rangle \right\} \\ &= \arg \min_{\mathbf{X}} \left\{ \mathbb{I}_{\mathbb{S}_+^n}(\mathbf{X}) + \left\| \mathbf{X} - \frac{\mathbf{X}^t - \alpha_t \mathcal{A}^*(\mathbf{y}^t) - \alpha_t \mathbf{C}}{1 + \alpha_t \varepsilon} \right\|_F^2 \right\} \\ &= \text{Prox}_{\frac{1}{2} \mathbb{I}_{\mathbb{S}_+^n}}(\cdot) \left[\frac{\mathbf{X}^t - \alpha_t \mathcal{A}^*(\mathbf{y}^t) - \alpha_t \mathbf{C}}{1 + \alpha_t \varepsilon} \right] \\ &= \text{Proj}_{\mathbb{S}_+^n} \left[\frac{\mathbf{X}^t - \alpha_t \mathcal{A}^*(\mathbf{y}^t) - \alpha_t \mathbf{C}}{1 + \alpha_t \varepsilon} \right] \end{aligned} \quad (22)$$

The derivation from $\arg \min$ to Proj holds, as the term

$$\left\| \mathbf{X} - \frac{\mathbf{X}^t - \alpha_t \mathcal{A}^*(\mathbf{y}^t) - \alpha_t \mathbf{C}}{1 + \alpha_t \varepsilon} \right\|_F^2 \quad (23)$$

is isotropic w.r.t. each entry of $\mathbf{X}$ and centered at

$$\frac{\mathbf{X}^t - \alpha_t \mathcal{A}^*(\mathbf{y}^t) - \alpha_t \mathbf{C}}{1 + \alpha_t \varepsilon}. \quad (24)$$

Therefore, finding the minimizer to Equation (22) is equivalent to finding the minimizer to Equation (23), that is Equation (24), then do Euclidean projection to the space of PSD cone, which is exactly the definition of PSD projection.

We notice that when $\varepsilon \rightarrow 0$, the problem reduces to linear SDP and the formulation is exactly the same as in Wang et al. (2024).

The dual update follows the derivation in Wang et al. (2024), the resolvent step Equation (17) simplifies to:

$$\mathbf{y}^{t+1} := \mathbf{y}^t + \beta_t \mathcal{A}(\mathbf{X}^{t+1} + \theta_t (\mathbf{X}^{t+1} - \mathbf{X}^t)) - \beta_t \mathbf{b}, \quad (25)$$

Now we arrive at the same conclusion as Wang et al. (2024). Since our PDHG update Equation (10) is an instance of non-stationary DRS, convergence to the unique solution $(\mathbf{X}^*, \mathbf{y}^*)$ follows immediately from Lorenz & Tran-Dinh (2019). $\square$

Remark C.12 (Notation and indexing). We clarify the indexing convention used in our derivation compared to Wang et al. (2024). While their Algorithm 1 performs the primal update $\mathbf{X}^t$ followed by the dual update $\mathbf{y}^t$, their theoretical proof presents the dual update first. Although these schemes are functionally equivalent due to the cyclic nature of alternating updates, we prioritize consistency between the derivation and the implementation. In the proof of Wang et al. (2024), a variable substitution to $\mathbf{y}^{t+1}$ is employed in their Equation 8 during the derivation of the dual step. We observe that by retaining the index $\mathbf{y}^t$ (i.e., avoiding this forward shift), the resulting fixed-point iteration aligns perfectly with the order of operations in their Algorithm 1. Consequently, our derivation adopts this convention in Equation (17), resulting in the PDHG algorithm in Equation (10) which updates the primal variable $\mathbf{X}^t$ first, thereby unifying the notation of the theory and the algorithm.

Remark C.13 (Strong convergence in finite dimensions). Standard convergence analyses for operator splitting methods, such as those in Lorenz & Tran-Dinh (2019), typically guarantee only weak convergence of the iterates in general Hilbert spaces. However, our problem setting is the space of symmetric matrices $\mathbb{S}^n$, which is a finite-dimensional space. It is a fundamental result that in finite-dimensional spaces, the weak topology coincides with the strong topology. Consequently, the weak convergence established in Proposition Appendix C.3 implies strong convergence in the Frobenius norm. That is, the sequence satisfies $\lim_{t \rightarrow \infty} \|\mathbf{X}^t - \mathbf{X}^*\|_F = 0$.

C.4. Proof for VC-2-FWL expressivity

Theorem C.14 (Restatement of Theorem 2.3). *Let $\mathbf{X}^* \in \mathbb{S}_+^n$ be the primal optimal solution to a given SDP instance and given indices $(i, j), (p, q)$. If the stable colorings of VC-2-FWL satisfy $\mathbf{v}_{ij}^\infty = \mathbf{v}_{pq}^\infty$, then the solution values satisfy $X_{ij}^* = X_{pq}^*$.*

We begin by proving a critical lemma. While Fürer (1995; 2010) have proven that standard 2-FWL refines the spectral decomposition of the graph adjacency matrix, we contribute a similar result specifically for a multiset 2-FWL variant. Notably, we employ a novel proof technique to demonstrate that, despite its reduced expressivity, the multiset 2-FWL retains this crucial spectral refinement property.

Lemma C.15. *Let $\mathbf{M} \in \mathbb{S}^n$ be a symmetric matrix with spectral decomposition*

$$\mathbf{M} = \sum_{k=1}^m \lambda_k \mathbf{P}_k$$

where $\lambda_1 > \lambda_2 > \dots > \lambda_m$ are the non repeating eigenvalues, and $\mathbf{P}_k$ are Frobenius covariants (Horn & Johnson, 1994, p. 437), i.e., the symmetric matrices describing the projection onto the eigenspace of eigenvalue λ_k .

The stable coloring $\mathbf{v}^\infty$ are produced by the multiset 2-FWL algorithm:

$$\begin{aligned} \mathbf{v}_{ij}^0 &= \text{hash}(\mathbf{M}_{ij}, \mathbb{I}_{i=j}) \\ \mathbf{v}_{ij}^t &:= \text{hash}\left(\mathbf{v}_{ij}^{t-1}, \left\{ \left\{ \left\{ \mathbf{v}_{uj}^{t-1}, \mathbf{v}_{iu}^{t-1} \right\} \mid u \in [n] \right\} \right\}, \forall t > 0, \end{aligned}$$

where $\mathbb{I}_{i=j}$ is a diagonal indicator which takes value 1 if $i = j$ otherwise 0. The following result holds:

$$\mathbf{v}_{ij}^\infty = \mathbf{v}_{pq}^\infty \implies (\mathbf{P}_k)_{ij} = (\mathbf{P}_k)_{pq}, \quad \forall k \in [m].$$

Proof. The proof is based on the fact that the spectral projectors $\mathbf{P}_k$ are polynomials in $\mathbf{M}$ via Sylvester's formula. We say a coloring $\mathbf{v}$ refines a matrix $\mathbf{A}$, defined as:

$$\mathbf{v}_{ij} = \mathbf{v}_{pq} \implies A_{ij} = A_{pq}.$$

We proceed by induction to demonstrate that the multiset aggregation effectively simulates matrix multiplication: if the 2-FWL coloring at an iteration refines the entries of $\mathbf{M}^d$, the next iteration refines $\mathbf{M}^{d+1}$. Consequently, the stable coloring refines any polynomial of $\mathbf{M}$, and by extension, naturally refines all the specific polynomials defining the spectral projectors.

At initialization $t = 0$, the coloring of $\mathbf{v}^0$ refines $\mathbf{M}^1$ naturally, as they are injectively defined by $\mathbf{M}$. Besides, $\mathbf{v}^0$ also refines $\mathbf{M}^0 = \mathbf{I}$, because of the diagonal indicator $\mathbb{I}_{i=j}$.

Assume that at iteration t , the coloring $\mathbf{v}^t$ refines the matrix power $\mathbf{M}^d$:

$$\mathbf{v}_{ij}^t = \mathbf{v}_{pq}^t \implies (\mathbf{M}^d)_{ij} = (\mathbf{M}^d)_{pq}.$$

We show that the next iteration $\mathbf{v}^{t+1}$ will refine $\mathbf{M}^{d+1}$:

$$\mathbf{v}_{ij}^{t+1} = \mathbf{v}_{pq}^{t+1} \implies (\mathbf{M}^{d+1})_{ij} = (\mathbf{M}^{d+1})_{pq}.$$

Specifically, by definition of the multiset 2-FWL:

$$\begin{aligned} \mathbf{v}_{ij}^{t+1} &= \mathbf{v}_{pq}^{t+1} \\ \implies \text{hash}\left(\mathbf{v}_{ij}^t, \left\{ \left\{ \left\{ \mathbf{v}_{uj}^t, \mathbf{v}_{iu}^t \right\} \mid u \in [n] \right\} \right\}\right) &= \text{hash}\left(\mathbf{v}_{pq}^t, \left\{ \left\{ \left\{ \mathbf{v}_{uq}^t, \mathbf{v}_{pu}^t \right\} \mid u \in [n] \right\} \right\}\right) \\ \implies \text{hash}\left(\left\{ \left\{ \left\{ \mathbf{v}_{uj}^t, \mathbf{v}_{iu}^t \right\} \mid u \in [n] \right\} \right\}\right) &= \text{hash}\left(\left\{ \left\{ \left\{ \mathbf{v}_{uq}^t, \mathbf{v}_{pu}^t \right\} \mid u \in [n] \right\} \right\}\right) \\ \implies f\left(\left\{ \left\{ g\left(\mathbf{v}_{uj}^t, \mathbf{v}_{iu}^t\right) \mid u \in [n] \right\} \right\}\right) &= f\left(\left\{ \left\{ g\left(\mathbf{v}_{uq}^t, \mathbf{v}_{pu}^t\right) \mid u \in [n] \right\} \right\}\right) \end{aligned}$$

The last implication holds for all choices of functions f, g due to the injectivity of hash function, as long as g is invariant to the order of inputs. In this specific case, we pick $f(\dots)$ to be sum over a multiset. The choice for g is trickier, we introduce

extra functions $h_1(v_{ij}^t) = (M^d)_{ij}$ and $h_2(v_{ij}^t) = (M^1)_{ij}$, and $g(a, b) = h_1(a)h_2(b) + h_2(a)h_1(b)$. The construction for function h_1 is valid because of the induction assumption $v_{ij}^t = v_{pq}^t \implies (M^d)_{ij} = (M^d)_{pq}$, we can find such a function h_1 that maps the colors to the matrix entries of M^d . The construction for function h_2 is also valid because v^0 already refines M , and v^t refines v^0 . Besides, g is invariant to the order of the inputs: $g(a, b) = g(b, a)$, compatible for the multiset.

Further, we have

$$\begin{aligned}
 & f\left(\left\{\left\{g(v_{uj}^t, v_{iu}^t) \mid u \in [n]\right\}\right\}\right) \\
 &= \sum_{u \in [n]} g(v_{uj}^t, v_{iu}^t) \\
 &= \sum_{u \in [n]} h_1(v_{uj}^t) h_2(v_{iu}^t) + h_2(v_{uj}^t) h_1(v_{iu}^t) \\
 &= \sum_{u \in [n]} (M^d)_{uj} (M^1)_{iu} + (M^1)_{uj} (M^d)_{iu} \\
 &= \sum_{u \in [n]} (M^1)_{iu} (M^d)_{uj} + \sum_{u \in [n]} (M^d)_{iu} (M^1)_{uj} \\
 &= (MM^d)_{ij} + (M^dM)_{ij} \\
 &= 2(M^{d+1})_{ij}
 \end{aligned}$$

Likewise, we have $f\left(\left\{\left\{g(v_{uq}^t, v_{pu}^t) \mid u \in [n]\right\}\right\}\right) = 2(M^{d+1})_{pq}$. Therefore, $v_{ij}^{t+1} = v_{pq}^{t+1} \implies (M^{d+1})_{ij} = (M^{d+1})_{pq}$.

By induction, the stable coloring v^∞ refines M^d for all $d \in \mathbb{N}$. Consequently, it also refines any matrix polynomial $p(M) = \sum_{d=0}^\infty a_d M^d$. This holds because the expressivity to distinguish the structural features of individual powers M^d implies the ability to distinguish their linear combinations. The real-valued matrix M is symmetric and diagonalizable, therefore we apply spectral decomposition $M = \sum_{k=1}^m \lambda_k P_k$, where matrices P_k are given by the Sylvester's formula (Horn & Johnson, 1994, p. 437):

$$P_k = \prod_{j \neq k} \frac{M - \lambda_j I}{\lambda_k - \lambda_j}.$$

Because each P_k is a polynomial in M , the stable coloring v^∞ refines each P_k . Thus, it holds for all k that $v_{ij}^\infty = v_{pq}^\infty \implies (P_k)_{ij} = (P_k)_{pq}$. $\square$

Remark C.16. The lemma above analyzes multiset 2-FWL, a simplified VC-2-FWL algorithm acting solely on a symmetric matrix M without constraint nodes. Note that our full VC-2-FWL architecture Equation (4) subsumes this process. The VC-2-FWL variable update aggregates the same multiset of variable neighbors $\{\{\{v_{uj}, v_{iu}\} \mid u \in [n]\}\}$ together with constraint information. Therefore, VC-2-FWL is no less expressive than the simplified process, as it uses strictly more information for hashing, and any refinement made by the simplified process is guaranteed for VC-2-FWL as well. Thus, the spectral refinement properties proven above apply directly to the VC-2-FWL variable colors. Now with the lemma, we formally prove Theorem 2.3.

Proof of Theorem 2.3. We provide a constructive proof by simulating the primal-dual hybrid gradient (PDHG) algorithm (Chambolle & Pock, 2016; Wang et al., 2024). We show that the stable coloring of VC-2-FWL refines every update of PDHG algorithm, which is proven to converge.

As established in Appendix C.3, the PDHG updates for the Frobenius-regularized SDP are given by:

$$\begin{aligned}
 X^{t+1} &:= \text{Proj}_{\mathbb{S}_+^n} \left[\frac{X^t - \alpha_t \mathcal{A}^*(y^t) - \alpha_t C}{1 + \alpha_t \varepsilon} \right] \\
 y^{t+1} &:= y^t + \beta_t \mathcal{A}(X^{t+1} + \theta_t (X^{t+1} - X^t)) - \beta_t b.
 \end{aligned} \tag{26}$$

For any $\varepsilon > 0$, the SDP problem has a unique primal solution $\mathbf{X}^*$. And if $\varepsilon \rightarrow 0$, it reduces to normal linear SDP. We proceed by induction on t , initializing with $\mathbf{X}^0 = \mathbf{0}_{n \times n}$ and $\mathbf{y}^0 = \mathbf{0}_m$. Our goal is to show that for all $t \geq 0$, the stable VC-2-FWL coloring on variable and constraint nodes refines the intermediate primal-dual solutions of PDHG updates:

$$\begin{aligned} \mathbf{v}_{ij}^\infty = \mathbf{v}_{pq}^\infty &\implies X_{ij}^t = X_{pq}^t \\ \mathbf{c}_k^\infty = \mathbf{c}_l^\infty &\implies y_k^t = y_l^t. \end{aligned}$$

Before the inductive step, we establish three key facts derived from the properties of the stable coloring.

Fact 1 Since the stable coloring $\mathbf{v}^\infty$ refines the initial coloring $\mathbf{v}^0$, and $\mathbf{v}^0$ is defined by $\mathbf{C}$, it holds that $\mathbf{v}_{ij}^\infty = \mathbf{v}_{pq}^\infty \implies C_{ij} = C_{pq}$. Furthermore, by applying Lemma C.15 to $\mathbf{C}$, if its spectral decomposition is given by $\mathbf{C} = \sum_h \lambda_h \mathbf{P}_h$, then the projection matrices also respect the coloring: $\forall h: \mathbf{v}_{ij}^\infty = \mathbf{v}_{pq}^\infty \implies (\mathbf{P}_h)_{ij} = (\mathbf{P}_h)_{pq}$. Besides, we know $\mathbf{c}^\infty$ refines $\mathbf{b}$ by its initialization $\mathbf{c}_k^0 := \text{hash}(\mathbf{b}_k)$.

Fact 2 Consider any two pairs $(i, j), (p, q) \in [n]^2$. According to VC-2-FWL definition Equation (4), if they have the same stable color, their hash function entries must be identical. Specifically, for all choices of function f :

$$\begin{aligned} \mathbf{v}_{ij}^\infty = \mathbf{v}_{pq}^\infty &\implies \text{hash}\left(\mathbf{v}_{ij}^\infty, \left\{\left\{\left\{\mathbf{v}_{u_j}^\infty, \mathbf{v}_{u_i}^\infty\right\} \mid u \in [n]\right\}, \left\{(A_{k,ij}, \mathbf{c}_k^\infty) \mid k \in N(ij)\right\}\right\}\right) = \\ &\quad \text{hash}\left(\mathbf{v}_{pq}^\infty, \left\{\left\{\left\{\mathbf{v}_{u_q}^\infty, \mathbf{v}_{u_p}^\infty\right\} \mid u \in [n]\right\}, \left\{(A_{k,pq}, \mathbf{c}_k^\infty) \mid k \in N(pq)\right\}\right\}\right) \\ &\implies \text{hash}\left(\left\{(A_{k,ij}, \mathbf{c}_k^\infty) \mid k \in N(ij)\right\}\right) = \text{hash}\left(\left\{(A_{k,pq}, \mathbf{c}_k^\infty) \mid k \in N(pq)\right\}\right) \\ &\implies \sum_{k \in N(ij)} f(A_{k,ij}, \mathbf{c}_k^\infty) = \sum_{k \in N(pq)} f(A_{k,pq}, \mathbf{c}_k^\infty) \\ &\implies \sum_{k \in N(ij)} A_{k,ij} f(\mathbf{c}_k^\infty) = \sum_{k \in N(pq)} A_{k,pq} f(\mathbf{c}_k^\infty), \text{ (we overload } f \text{ for ease of notation)} \\ &\implies \sum_{k \in [m]} A_{k,ij} f(\mathbf{c}_k^\infty) = \sum_{k \in [m]} A_{k,pq} f(\mathbf{c}_k^\infty) \end{aligned}$$

The last implication holds as $\forall k \notin N(ij): A_{k,ij} = 0$, by definition of neighbors. Consequently, for any function f mapping the $\mathbf{c}^\infty$ to real numbers, the summations over these multisets are identical. In the inductive steps, we pick function f to map stable coloring to intermediate dual variables, $f(\mathbf{c}_k^\infty) = y_k^t$ for some suitable t , as we will show later.

Fact 3 Similarly for constraint nodes, consider $k, l \in [m]$. According to Equation (4), if two constraint nodes have the same stable colors, we have, for any function g :

$$\begin{aligned} \mathbf{c}_k^\infty = \mathbf{c}_l^\infty &\implies \text{hash}\left(\mathbf{c}_k^\infty, \left\{\left\{(A_{k,ij}, \mathbf{v}_{ij}^\infty) \mid (i, j) \in N(k)\right\}\right\}\right) = \text{hash}\left(\mathbf{c}_l^\infty, \left\{\left\{(A_{l,ij}, \mathbf{v}_{ij}^\infty) \mid (i, j) \in N(l)\right\}\right\}\right) \\ &\implies \text{hash}\left(\left\{(A_{k,ij}, \mathbf{v}_{ij}^\infty) \mid (i, j) \in N(k)\right\}\right) = \text{hash}\left(\left\{(A_{l,ij}, \mathbf{v}_{ij}^\infty) \mid (i, j) \in N(l)\right\}\right) \\ &\implies \sum_{(i,j) \in N(k)} g(A_{k,ij}, \mathbf{v}_{ij}^\infty) = \sum_{(i,j) \in N(l)} g(A_{l,ij}, \mathbf{v}_{ij}^\infty) \\ &\implies \sum_{(i,j) \in N(k)} A_{k,ij} g(\mathbf{v}_{ij}^\infty) = \sum_{(i,j) \in N(l)} A_{l,ij} g(\mathbf{v}_{ij}^\infty), \text{ (we overload } g \text{ for ease of notation)} \\ &\implies \sum_{(i,j) \in [n]^2} A_{k,ij} g(\mathbf{v}_{ij}^\infty) = \sum_{(i,j) \in [n]^2} A_{l,ij} g(\mathbf{v}_{ij}^\infty) \end{aligned}$$

In the inductive steps, we will pick function g to map stable coloring to intermediate primal variables, similar to the choice of f .

Proof by induction We now prove the theorem by induction on t . At initialization $t = 0$, we have $\mathbf{X}^0 = \mathbf{0}$ and $\mathbf{y}^0 = \mathbf{0}$. The implications $\mathbf{v}_{ij}^\infty = \mathbf{v}_{pq}^\infty \implies X_{ij}^0 = X_{pq}^0$ and $\mathbf{c}_k^\infty = \mathbf{c}_l^\infty \implies y_k^0 = y_l^0$ hold trivially.

Suppose the hypothesis holds for iteration $t \geq 0$. We show it holds for iteration $t + 1$.

1. Primal Update. In the primal update step, we must evaluate the term $\mathcal{A}^*(\mathbf{y}^t) = \sum_{k=1}^m y_k^t \mathbf{A}_k$. We define the function f in Fact 2 as $f(\mathbf{c}_k^\infty) = y_k^t$. This function is well-defined because, by the inductive hypothesis, y_k^t is refined by the color $\mathbf{c}_k^\infty$, i.e., $\mathbf{c}_k^\infty = \mathbf{c}_l^\infty \implies y_k^t = y_l^t$. Applying Fact 2 with this f , we obtain:

$$\mathbf{v}_{ij}^\infty = \mathbf{v}_{pq}^\infty \implies \sum_{k \in [m]} A_{k,ij} y_k^t = \sum_{k \in [m]} A_{k,pq} y_k^t.$$

The implication above shows that $\mathcal{A}^*(\mathbf{y}^t)$ is refined by the coloring $\mathbf{v}^\infty$. Now consider the full term inside the PSD projection in Equation (26):

$$\mathbf{Z}^t := \frac{\mathbf{X}^t - \alpha_t \mathcal{A}^*(\mathbf{y}^t) - \alpha_t \mathbf{C}}{1 + \alpha_t \epsilon}.$$

Since $\mathbf{X}^t$ (by hypothesis), $\mathbf{C}$ (by Fact 1), and $\mathcal{A}^*(\mathbf{y}^t)$ (by derivation) are all refined by the coloring $\mathbf{v}^\infty$, their linear combination $\mathbf{Z}^t$ also is. Finally, the update is $\mathbf{X}^{t+1} = \text{Proj}_{\mathbb{S}_+^n}(\mathbf{Z}^t)$. The PSD projection depends only on the spectral decomposition as detailed in Algorithm 1. Specifically, spectral decomposition is performed on $\mathbf{Z}^t$, and the Frobenius covariants corresponding to negative eigenvalues are removed. By Lemma C.15, we know $\mathbf{v}_{ij}^\infty = \mathbf{v}_{pq}^\infty \implies X_{ij}^{t+1} = X_{pq}^{t+1}$, that is, $\mathbf{v}^\infty$ refines $\mathbf{X}^{t+1}$.

2. Dual Update. In the dual update step, we evaluate the term inside the operator $\mathcal{A}$:

$$\mathbf{W}^{t+1} := \mathbf{X}^{t+1} + \theta_t (\mathbf{X}^{t+1} - \mathbf{X}^t).$$

Since both $\mathbf{X}^{t+1}$ and $\mathbf{X}^t$ are refined by $\mathbf{v}^\infty$, so is $\mathbf{W}^{t+1}$. We define the function g in Fact 3 as $g(\mathbf{v}_{ij}^\infty) = (\mathbf{W}^{t+1})_{ij}$. This is well-defined because the value of $(\mathbf{W}^{t+1})_{ij}$ is refined by the color $\mathbf{v}_{ij}^\infty$. Applying Fact 3 with this g , we obtain:

$$\mathbf{c}_k^\infty = \mathbf{c}_l^\infty \implies [\mathcal{A}(\mathbf{W}^{t+1})]_k = [\mathcal{A}(\mathbf{W}^{t+1})]_l.$$

Since $\mathbf{y}^t$ (by hypothesis) and $\mathbf{b}$ (by Fact 1) are both refined by the coloring $\mathbf{c}^\infty$, the sum $\mathbf{y}^{t+1}$ is refined by the coloring.

In summary, we have proven by induction that if the stable coloring $\mathbf{v}^\infty$ and $\mathbf{c}^\infty$ refine $\mathbf{X}^t$ and $\mathbf{y}^t$ in PDHG algorithm, they also refine $\mathbf{X}^{t+1}$ and $\mathbf{y}^{t+1}$.

Since PDHG is guaranteed to converge to $\mathbf{X}^*$ and $\mathbf{y}^*$, this completes the proof. $\square$

Remark C.17. Notably, this result extends to comparisons across distinct SDP instances. The pairs (i, j) and (p, q) do not need to belong to the same problem: provided consistent initialization, if their stable VC-2-FWL colorings are identical, their corresponding optimal values are guaranteed to be equal. This implies a form of universality, suggesting that the VC-2-FWL hierarchy captures fundamental structural invariants that generalize across the whole space of SDP problems.

C.5. Proof for complexity analysis

Lemma C.18 (Restatement of Theorem 12 of Berkholz et al. (2017)). *Let $G = (V, E, c)$ be an edge colored digraph with $N = |V|$ and $M = |E|$. In time $O((N + M) \log(N + M))$, a canonical coarsest edge-color stable coloring can be computed for G .*

Proposition C.19 (Restatement of Proposition 2.4). *Given an SDP of $n \times n$ variables and m constraints, $\text{nnz}(\mathbf{A})$ denotes the number of non-zero entries in $\mathbf{A}$, then VC-2-FWL converges in time*

$$\mathcal{O}((n^3 + \text{nnz}(\mathbf{A})) \log n).$$

Proof. We analyze the computational complexity of VC-2-FWL by demonstrating that its update rule is algorithmically equivalent to standard 1-WL (color refinement) operating on a directed, edge-colored auxiliary graph. This equivalence allows us to invoke the tight complexity bounds established in Lemma C.18.

Auxiliary graph construction Let $G_{\text{aux}} = (V, E)$ be the implicit auxiliary graph defined by the update rules in Equation (4). The node set V is the disjoint union of variable nodes and constraint nodes, and a set of higher-order nodes: $V = V_{\text{var}} \cup V_{\text{con}} \cup V_{\text{tri}}$.

1. V_{var} consists of the n^2 variable nodes. These are initialized with colors $\mathbf{a}_{ij}^0 := \alpha_{\text{var}}(C_{ij})$, where α_{var} is a canonical mapping for variables.
2. V_{con} consists of the m constraint nodes. These are initialized with colors $\mathbf{b}_k^0 := \alpha_{\text{con}}(b_k)$, where α_{con} is distinct from α_{var} .
3. We further construct a node set V_{tri} . For every variable node with index (i, j) , we construct n nodes with index (i, u, j) for $u \in [n]$. These are initialized with colors $\mathbf{c}_{iuj}^0 = \alpha_{\text{tri}}(\{C_{iu}, C_{uj}\})$.

Since the initialization functions $\alpha(\cdot)$ have disjoint ranges, the color classes of three node sets remain disjoint throughout the execution. The total number of nodes is $N = n^2 + m + n^3$.

The edge set E decomposes into the following:

1. $E_{\text{tri-v}} \subseteq V_{\text{tri}} \times V_{\text{var}}$: These edges connect nodes from V_{tri} to V_{var} . Specifically, every variable node indexed (i, j) gets links from $\{(i, u, j) \mid u \in [n]\}$. This requires n^3 edges. The edge colors can be a constant.
2. $E_{\text{v-tri}} \subseteq V_{\text{var}} \times V_{\text{tri}}$: This is the other direction of above. Every higher order node (i, u, j) gets links from variable nodes (i, u) and (u, j) , thus $2n^3$ edges. The edge colors can be a constant.
3. $E_{\text{v-c}} \subseteq V_{\text{var}} \times V_{\text{con}}$: These edges encode the $\mathbf{A}$ structure. An edge exists from $(i, j) \in V_{\text{var}}$ to $k \in V_{\text{con}}$ if and only if $A_{k,ij} \neq 0$. The edge colors are defined by the tensor entries: $e_{k,ij} = \alpha_A(A_{k,ij})$. The size of this set is $\text{nnz}(\mathbf{A})$.
4. $E_{\text{c-v}} \subseteq V_{\text{con}} \times V_{\text{var}}$: These edges are the opposite direction from $E_{\text{v-c}}$. The edge colors are the same: $e_{k,ij} = \alpha_A(A_{k,ij})$.

The total number of edges is $M = 3n^3 + 2\text{nnz}(\mathbf{A})$.

Update equivalence We now show that the 1-WL update on this edge-colored graph is equivalent to the VC-2-FWL update.

For a constraint node, it updates in the sense of 1-WL as:

$$\mathbf{b}_k^{t+1} := \text{hash}(\mathbf{b}_k^t, \{(e_{ij,k}^t, \mathbf{a}_{ij}^t) \mid (i, j): A_{k,ij} \neq 0\})$$

which has exactly the same form as in Equation (4).

For a variable node, we first update the higher-order nodes:

$$\mathbf{c}_{iuj}^{t+1/2} := \text{hash}(\mathbf{c}_{iuj}^t, \{\mathbf{v}_{iu}^t, \mathbf{v}_{uj}^t\}),$$

regardless of the edge colors. Then, we update the variable nodes:

$$\mathbf{a}_{ij}^{t+1} := \text{hash}(\mathbf{a}_{ij}^t, \{\{\mathbf{c}_{i0j}^{t+1/2}, \dots, \mathbf{c}_{inj}^{t+1/2}, (e_{k,ij}, \mathbf{b}_k^t) \dots\}\})$$

Since the node types V_{tri} and V_{con} are distinguished by disjoint color spaces, we can partition the multiset into two distinct multisets without ambiguity:

$$\mathbf{a}_{ij}^{t+1} := \text{hash}(\mathbf{a}_{ij}^t, \{\{\mathbf{c}_{iuj}^{t+1/2} \mid u \in [n]\}, \{(e_{k,ij}, \mathbf{b}_k^t) \mid k: A_{k,ij} \neq 0\}\}).$$

This form matches the variable update in Equation (4). Therefore, the variable update in Equation (4) can be implemented with 2 steps of 1-WL.

Hence, we have shown the equivalence of our VC-2-FWL and standard 1-WL. Applying the bound in Lemma C.18 to our setting: $N = |V| = n^2 + m + n^3$ and $M = |E| = \mathcal{O}(n^3 + \text{nnz}(\mathbf{A}))$. Since m is bounded by $\mathcal{O}(n^2)$ as we assume linearly independent constraints, $\mathcal{O}((N + M)) = \mathcal{O}(n^3 + \text{nnz}(\mathbf{A}))$, and $\mathcal{O}(\log(N + M)) = \mathcal{O}(\log n)$. Substituting these values yields the total time complexity: $\mathcal{O}((n^3 + \text{nnz}(\mathbf{A})) \log n)$. $\square$

C.6. Proof for neuralization

Remark C.20. It is important to clarify the terminology regarding the hash function utilized in the WL test aggregation step. In standard computer science contexts, cryptographic or data-structure hash functions are inherently discrete and non-continuous. However, in the theoretical framework of GNN expressivity, a hash function simply denotes an injective mapping that assigns a unique, deterministic representation to any distinct multiset of neighbor features. Crucially, this

injective mapping is not required to be discrete. As established in the foundational GNN expressivity literature (Morris et al., 2019; Xu et al., 2019), there exist continuous, injective functions capable of perfectly resolving bounded multisets. Because these idealized WL aggregation functions can be chosen to be strictly continuous, they satisfy the necessary conditions to be arbitrarily approximated by MLPs via the universal approximation theorem.

Proposition C.21 (Restatement of Proposition 2.5). *There exists a set of parameters for the functions INIT, UPD, MSG, MAP, such that VC-2-FMPNN has maximal expressivity equal to the VC-2-FWL.*

First, we state a lemma.

Lemma C.22. *Let $\mathcal{K} \subset \mathbb{R}^d$ be a compact set, and let $\mathcal{X}$ be the space of finite multisets of bounded maximum size with elements drawn from $\mathcal{K}$. There exists an integer $p \geq 1$ and a continuous, injective function*

$$f: \mathcal{X} \rightarrow \mathbb{R}^p$$

such that for any two multisets $X_1, X_2 \in \mathcal{X}$,

$$f(X_1) = f(X_2) \iff X_1 = X_2.$$

This is exactly a restatement of Lemma 4 of Xu et al. (2019), if the multiset space is discrete and countable; or of Proposition 1 of Maron et al. (2019a), if it is a multiset of real number space, by using the power-sum multi-symmetric polynomial technique. In our case, unlike discrete graphs, we assume that the problem coefficients $\mathbf{C}, \mathbf{A}, \mathbf{b}$ as well as the intermediate outputs of the neural networks lie within a compact subset $\mathcal{K}$, i.e., real space where the coefficients don't go to infinity. This ensures that the inputs are bounded and allows for the application of the Stone-Weierstrass theorem for approximation. So we can reuse the proof technique in Maron et al. (2019a). Now let's prove Proposition 2.5.

Proof for Proposition 2.5. We prove the equality of expressivity by showing both directions of refinement.

VC-2-FWL $\sqsubseteq$ VC-2-FMPNN: First, VC-2-FMPNN is no more expressive than VC-2-FWL. It is easy to verify with the same logic as Theorem 1 of Morris et al. (2019). The VC-2-FMPNN updates are deterministic functions of the unrolling trees defined by the VC-2-FWL color refinement. Therefore, if VC-2-FWL produces identical colors for two variable pairs, VC-2-FMPNN must produce identical embeddings.

VC-2-FMPNN $\sqsubseteq$ VC-2-FWL: We show that VC-2-FMPNN can distinguish any two instances distinguishable by VC-2-FWL at every iteration t .

At initialization $t = 0$, the initial features are directly encoded from the problem data. Since we assume the input space $\mathcal{K}$ is compact, there exist MLPs by the universal approximation theorem that can implement the initialization functions injectively, preserving the distinction between different initial values of C_{ij} or b_k . At iteration $t > 0$, we consider the update for constraint and variable nodes, respectively.

Constraint node Consider the update of a constraint node k . The update requires an injective mapping of the tuple containing the constraint node's previous feature $\mathbf{h}_k^{t-1}$ and the multiset of its variable neighbors' features together with the edge weights: $\{\{(A_{k,ij}, \mathbf{v}_{ij}^{t-1}) \mid (i, j) \in N(k)\}\}$. According to Lemma C.22, there exists an injective function that uniquely encodes this multiset. As shown in the constructive proof of Maron et al. (2019a, Proposition 1), this injective function can be realized as a summation of power-sum polynomials. Consequently, the MLP in the term

$$\mathbf{m}_{k,v \rightarrow c}^t := \sum_{(i,j) \in N(k)} \text{MSG}_{v \rightarrow c}^t(A_{k,ij}, \mathbf{h}_{ij}^{t-1})$$

can be parameterized to approximate these continuous mappings, ensuring that the summation yields a unique representation of the multiset. Furthermore, the injective combination of the node's previous state $\mathbf{h}_k^{t-1}$ and this multiset embedding $\mathbf{m}_{k,v \rightarrow c}^t$ can be approximated arbitrarily well by concatenation of them and application of another MLP:

$$\mathbf{h}_k^t := \text{UPD}_c^t(\mathbf{h}_k^{t-1}, \mathbf{m}_{k,v \rightarrow c}^t).$$

Since the intermediate space $\mathcal{K}$ is compact, there exist MLPs that universally approximate the injective hash function in Equation (4).

Variable node Consider the VC-2-FWL update for a variable index (i, j) . VC-2-FWL updates the variable color based on the variable node's previous feature, the multiset of constraint neighbors and the multiset of variable neighbor pairs. For the multiset of constraint neighbors, it aggregates the multiset $\{(A_{k,ij}, \mathbf{c}_k^{t-1}) \mid k \in N(ij)\}$, we apply Lemma C.22 once, and the term

$$\mathbf{m}_{ij,c \rightarrow v}^t := \sum_{k \in N(ij)} \text{MSG}_{c \rightarrow v}^t(A_{k,ij}, \mathbf{h}_k^{t-1})$$

implements the injective mapping from constraint neighbors to a vector embedding. The multiset of variable pairs are formed by $\{\{\mathbf{v}_{uj}^{t-1}, \mathbf{v}_{iu}^{t-1}\} \mid u \in [n]\}$. To complete this injectively, we invoke Lemma C.22 twice. First, for a specific u , the unordered pair $s_u = \{\{\mathbf{v}_{uj}^{t-1}, \mathbf{v}_{iu}^{t-1}\}\}$ constitutes a 2-multiset. By Lemma C.22, there exists an MLP capturing the injectivity of s_u . The term

$$\text{MAP}^t(\mathbf{h}_{uj}^{t-1}) + \text{MAP}^t(\mathbf{h}_{iu}^{t-1})$$

explicitly implements this. Second, we must aggregate these encoded pairs over all $u \in [n]$. The target structure is the outer multiset $S_{ij} = \{s_u \mid u \in [n]\}$. Applying Lemma C.22 again, there exists an injective mapping with MLP. Thus, the term

$$\mathbf{m}_{ij,v \rightarrow v}^t := \sum_{u \in [n]} \text{MSG}_{v \rightarrow v}^t(\text{MAP}^t(\mathbf{h}_{uj}^{t-1}) + \text{MAP}^t(\mathbf{h}_{iu}^{t-1}))$$

injectively encodes the nested multiset structure for the VC-2-FWL update. Finally, one can simply concatenate the embeddings $(\mathbf{h}_{ij}^{t-1}, \mathbf{m}_{ij,v \rightarrow v}^t, \mathbf{m}_{ij,c \rightarrow v}^t)$ and update it with the MLP UPD_v^t :

$$\mathbf{h}_{ij}^t := \text{UPD}_v^t(\mathbf{h}_{ij}^{t-1}, \mathbf{m}_{ij,v \rightarrow v}^t, \mathbf{m}_{ij,c \rightarrow v}^t).$$

In summary, every component of VC-2-FWL in Equation (4) can be approximated by an MLP. By picking the output approximation error ϵ to be smaller than the separation margin of the distinct VC-2-FWL colors, VC-2-FMPNN maintains the injectivity of the update step.

Thus, VC-2-FMPNN is at least as expressive as VC-2-FWL, which completes the proof. $\square$

VC-MPNN Similar to VC-2-FMPNN, we can neuralize the VC-WL to VC-MPNN, whose initialization is identical to Equation (5), and the update steps:

$$\begin{aligned} \mathbf{m}_{ij,c \rightarrow v}^t &:= \sum_{k \in N(ij)} \text{MSG}_{c \rightarrow v}^t(A_{k,ij}, \mathbf{h}_k^{t-1}) \\ \mathbf{m}_{k,v \rightarrow c}^t &:= \sum_{(i,j) \in N(k)} \text{MSG}_{v \rightarrow c}^t(A_{k,ij}, \mathbf{h}_{ij}^{t-1}) \\ \mathbf{h}_{ij}^t &:= \text{UPD}_v^t(\mathbf{h}_{ij}^{t-1}, \mathbf{m}_{ij,c \rightarrow v}^t) \\ \mathbf{h}_k^t &:= \text{UPD}_c^t(\mathbf{h}_k^{t-1}, \mathbf{m}_{k,v \rightarrow c}^t) \end{aligned} \tag{27}$$

Proposition C.23. *There exists a set of parameters for the functions INIT, UPD, MSG, such that VC-MPNN has maximal expressivity equal to the VC-WL.*

Proof. We apply the same logic as the proof of Proposition 2.5, simplified for the reduced message passing structure.

VC-WL $\sqsubseteq$ VC-MPNN: Since the VC-MPNN update rules are deterministic functions of the VC-WL unrolling tree (which is actually a subset of the VC-2-FWL tree), the neural network cannot distinguish elements that the color refinement algorithm cannot.

VC-MPNN $\sqsubseteq$ VC-WL: We verify that the network components can implement the injective update steps of VC-WL. At initialization $t = 0$, similar to Proposition 2.5, there exist MLPs that perform this mapping injectively. For iteration $t > 0$:

Constraint node The constraint node update simply follows Proposition 2.5.

Variable node The VC-WL variable update aggregates only the constraint neighbors, omitting the variable node interactions. This is a simplified case of the variable update in Equation (6). The message term $\mathbf{m}_{ij,c \rightarrow v}^t$ with an MLP is sufficient to

injectively encode the constraint neighbor multiset $\{(A_{k,ij}, c_k^{t-1}) \mid k \in N(ij)\}$. The UPD_v^t then injectively maps the pair $(h_{ij}^{t-1}, m_{ij,c \rightarrow v}^t)$ to the next state.

Thus, VC-MPNN is capable of simulating VC-WL injectively. $\square$

Building on the limitation of VC-WL established in Proposition 2.2, we demonstrate that the neuralized VC-MPNN architecture cannot universally approximate SDP solutions. We rely on the following homogeneity property of linear SDPs.

Lemma C.24. *Consider a standard form SDP with data $(C, \mathbf{A}, \mathbf{b})$ and a triplet of primal, dual and slack solution $(\mathbf{X}^*, \mathbf{y}^*, \mathbf{S}^*)$. If the vector $\mathbf{b}$ is scaled by a positive scalar $\alpha > 0$ to $\alpha\mathbf{b}$, then $\alpha\mathbf{X}^*$ is an optimal primal solution to the scaled problem.*

Proof. Optimality in linear SDPs is characterized by the Karush-Kuhn-Tucker (KKT) conditions. The original solution satisfies:

$$\begin{aligned} \mathcal{A}(\mathbf{X}^*) &= \mathbf{b} \\ \mathbf{C} + \mathcal{A}^*(\mathbf{y}^*) - \mathbf{S}^* &= \mathbf{0} \\ \mathbf{X}^* &\succeq 0 \\ \mathbf{S}^* &\succeq 0 \\ \langle \mathbf{X}^*, \mathbf{S}^* \rangle &= 0 \end{aligned} \tag{28}$$

It is not hard to find that the following also holds:

$$\begin{aligned} \mathcal{A}(\alpha\mathbf{X}^*) &= \alpha\mathcal{A}(\mathbf{X}^*) = \alpha\mathbf{b} \\ \mathbf{C} + \mathcal{A}^*(\mathbf{y}^*) - \mathbf{S}^* &= \mathbf{0} \\ \alpha\mathbf{X}^* &\succeq 0 \\ \mathbf{S}^* &\succeq 0 \\ \langle \alpha\mathbf{X}^*, \mathbf{S}^* \rangle &= 0. \end{aligned} \tag{29}$$

Therefore, $\alpha\mathbf{X}^*$ is the solution to the scaled SDP problem; thus, it completes the proof. $\square$

Corollary C.25. *The architecture VC-MPNN is not a universal approximator for SDPs. Specifically, for any $\epsilon > 0$, there exists an SDP instance such that for any parameters of a VC-MPNN, the error between the predicted solution $\mathbf{X}'$ and the optimal solution $\mathbf{X}^*$ satisfies*

$$\|\mathbf{X}^* - \mathbf{X}'\|_\infty \geq \epsilon.$$

Proof. According to Proposition C.23, the expressivity of VC-MPNN is bounded by the VC-WL coloring algorithm.

Consider the counterexample from the proof of Proposition 2.1. In this instance, variables X_{11} and X_{33} are structurally indistinguishable under the VC-WL algorithm, and thus VC-MPNN forces the network to predict identical values, i.e., $X'_{11} = X'_{33} = v$. However, in this case, the unique optimal solution satisfies $X_{11}^* \neq X_{33}^*$. Let $\delta = |X_{11}^* - X_{33}^*| > 0$. Now, scale the constraint vector to $\alpha\mathbf{b}$ with $\alpha = 2\epsilon/\delta$. By Lemma C.24, the new optimal solution is $\alpha\mathbf{X}^*$, and the gap between the entries becomes $|\alpha X_{11}^* - \alpha X_{33}^*| = \alpha\delta = 2\epsilon$. Since the scaling of $\mathbf{b}$ preserves the structural symmetry of the graph, the unrolling trees for variable nodes v_{11} and v_{33} remain isomorphic, the VC-MPNN must still predict identical values for these entries, say v' . By the triangle inequality, the approximation error is bounded from below:

$$\begin{aligned} 2\epsilon &= |\alpha X_{11}^* - \alpha X_{33}^*| \\ &\leq |\alpha X_{11}^* - v'| + |v' - \alpha X_{33}^*| \\ &\leq 2 \max(|\alpha X_{11}^* - v'|, |\alpha X_{33}^* - v'|) \\ &\leq 2\|\alpha\mathbf{X}^* - \mathbf{X}'\|_\infty. \end{aligned}$$

Therefore, $\|\alpha\mathbf{X}^* - \mathbf{X}'\|_\infty \geq \epsilon$, proving that the error can be made arbitrarily large. $\square$

VC-2-MPNN We neuralize the VC-2-WL and instantiate a neural network, which we name VC-2-MPNN. The initialization follows Equation (5), and the update is

$$\begin{aligned}
 \mathbf{m}_{ij,\text{row}}^t &:= \sum_{u \in [n]} \text{MSG}_{\text{row}}^t(\mathbf{h}_{iu}^{t-1}) \\
 \mathbf{m}_{ij,\text{col}}^t &:= \sum_{u \in [n]} \text{MSG}_{\text{col}}^t(\mathbf{h}_{uj}^{t-1}) \\
 \mathbf{m}_{ij,\text{c} \rightarrow \text{v}}^t &:= \sum_{k \in N(ij)} \text{MSG}_{\text{c} \rightarrow \text{v}}^t(A_{k,ij}, \mathbf{h}_k^{t-1}) \\
 \mathbf{m}_{k,\text{v} \rightarrow \text{c}}^t &:= \sum_{(i,j) \in N(k)} \text{MSG}_{\text{v} \rightarrow \text{c}}^t(A_{k,ij}, \mathbf{h}_{ij}^{t-1}) \\
 \mathbf{h}_{ij}^t &:= \text{UPD}_{\text{v}}^t(\mathbf{h}_{ij}^{t-1}, \mathbf{m}_{ij,\text{col}}^t, \mathbf{m}_{ij,\text{row}}^t, \mathbf{m}_{ij,\text{c} \rightarrow \text{v}}^t) \\
 \mathbf{h}_k^t &:= \text{UPD}_{\text{c}}^t(\mathbf{h}_k^{t-1}, \mathbf{m}_{k,\text{v} \rightarrow \text{c}}^t)
 \end{aligned} \tag{30}$$

Besides, following Equation (3), we force symmetry by $\mathbf{h}_{ji}^t = \mathbf{h}_{ij}^t$, if $i < j$.

Proposition C.26. *There exists a set of parameters for the functions INIT, UPD, MSG, such that VC-2-MPNN has maximal expressivity equal to the VC-2-WL.*

Proof. We apply the same logic as the proof of Proposition 2.5, tailored for the message passing structure of VC-2-MPNN.

VC-2-WL $\sqsubseteq$ VC-2-MPNN: Since the VC-2-MPNN update rules are deterministic functions of the VC-2-WL unrolling tree, the neural network cannot distinguish elements that the color refinement algorithm cannot.

VC-2-MPNN $\sqsubseteq$ VC-2-WL: We verify that the network components can implement the injective update steps of VC-2-WL. At initialization $t = 0$, similar to Proposition 2.5, there exist MLPs that perform this mapping injectively.

Constraint node The constraint node update simply follows Proposition 2.5.

Variable node The VC-2-WL variable update aggregates the constraint neighbors, as well as the row and column variable neighbors separately. The message from constraint neighbors $\mathbf{m}_{ij,\text{c} \rightarrow \text{v}}^t$ with an MLP is sufficient to injectively encode the constraint neighbor multiset $\{(A_{k,ij}, \mathbf{c}_k^{t-1}) \mid k \in N(ij)\}$. Similarly, the multisets of row and column neighbors $\mathbf{m}_{ij,\text{row}}^t$ and $\mathbf{m}_{ij,\text{col}}^t$ can be encoded injectively with MLPs respectively. Finally, the update function UPD_{v}^t then injectively maps the vectors $(\mathbf{h}_{ij}^{t-1}, \mathbf{m}_{ij,\text{col}}^t, \mathbf{m}_{ij,\text{row}}^t, \mathbf{m}_{ij,\text{c} \rightarrow \text{v}}^t)$ to the next state.

Thus, VC-2-MPNN is capable of simulating VC-2-WL injectively. $\square$

Next, we show a corollary that VC-2-MPNN is not a universal approximator to represent SDP.

Corollary C.27. *VC-2-MPNN is not a universal approximator for SDPs. Specifically, for any $\epsilon > 0$, there exists an SDP instance such that for any parameters of a VC-2-MPNN, the error between the predicted solution $\mathbf{X}'$ and the optimal solution $\mathbf{X}^*$ satisfies*

$$\|\mathbf{X}^* - \mathbf{X}'\|_{\infty} \geq \epsilon.$$

Proof. We follow exactly the proof logic in Corollary C.25. In this case, for VC-2-MPNN, we can pick the example in the proof of Proposition 2.2 where VC-2-WL fails. $\square$

D. SDP application

D.1. Relaxation of CO

In reality, SDP serves as the continuous relaxation of several graph CO problems.

Max-Cut (Goemans & Williamson, 1995). Let G be a graph with nodes V and edges E with edge weights $w_{ij} \geq 0$, $\{i, j\} \in E$. The goal of the max-cut problem is to partition the nodes into two disjoint sets such that the sum of weights of edges

crossing the cut is maximized. The problem formulation is

$$\max_{\mathbf{x} \in \{-1, 1\}^n} \frac{1}{2} \sum_{\{i, j\} \in E} w_{ij}(1 - x_i x_j), \quad (31)$$

where 1 and -1 denote different partitions. Instead of integer domain $\{-1, 1\}$, we relax each variable x_i to a unit vector $\|v_i\| = 1$, which defines $X_{ij} = v_i^\top v_j$ (Yau et al., 2024). Thus, SDP relaxation is

$$\begin{aligned} \max_{\mathbf{X} \in \mathbb{S}_+^n} \quad & \frac{1}{2} \sum_{\{i, j\} \in E} w_{ij}(1 - X_{ij}) \\ \text{s.t.} \quad & X_{ii} = 1, \forall i \in [n] \end{aligned} \quad (32)$$

Max clique problem Let G be a graph with nodes V and edges E , let $x_i \in \{0, 1\}$ denote whether x_i is included in a clique. The max clique problem can be formulated as

$$\begin{aligned} \max_{\mathbf{x} \in \{0, 1\}^n} \quad & \sum_i x_i \\ \text{s.t.} \quad & x_i x_j = 0, \forall \{i, j\} \notin E. \end{aligned} \quad (33)$$

We know that the Lovász function (Galli & Letchford, 2017) is defined as the optimal value of the SDP relaxation of max-clique problem:

$$\begin{aligned} \max_{\mathbf{X} \in \mathbb{S}_+^n} \quad & \langle \mathbf{J}, \mathbf{X} \rangle \\ \text{s.t.} \quad & \text{Tr}(\mathbf{X}) = 1 \\ & X_{ij} = 0, \forall \{i, j\} \notin E, \end{aligned} \quad (34)$$

where $\mathbf{J}$ is a ones matrix.

Max independent set The max independent set problem is the complement of the max clique problem. Formally,

$$\begin{aligned} \max_{\mathbf{x} \in \{0, 1\}^n} \quad & \sum_i x_i \\ \text{s.t.} \quad & x_i x_j = 0, \forall \{i, j\} \in E. \end{aligned} \quad (35)$$

and its relaxation:

$$\begin{aligned} \max_{\mathbf{X} \in \mathbb{S}_+^n} \quad & \langle \mathbf{J}, \mathbf{X} \rangle \\ \text{s.t.} \quad & \text{Tr}(\mathbf{X}) = 1 \\ & X_{ij} = 0, \forall \{i, j\} \in E. \end{aligned} \quad (36)$$

Graph coloring The goal of the graph coloring problem is to assign colors to nodes such that no adjacent nodes share the same color. Let $y_k \in \{0, 1\}, k \in [K]$ indicate whether the color k is used, where K is an upper bound, e.g., $K = |V|$. And let $x_{ik} \in \{0, 1\}$ indicate whether the node i is assigned to color k . The graph coloring problem in ILP form is defined as:

$$\begin{aligned} \min \quad & \sum_k y_k \\ \text{s.t.} \quad & \sum_k x_{ik} = 1 \\ & x_{ik} + x_{jk} \leq y_k, \forall \{i, j\} \in E \\ & x_{ik} \leq y_k. \end{aligned} \quad (37)$$

We have the SDP relaxation (Charikar, 2002):

$$\begin{aligned} \min_{\mathbf{X} \in \mathbb{S}_+^n} \quad & k \\ \text{s.t.} \quad & X_{ij} = -\frac{1}{k-1}, \forall \{i, j\} \in E \\ & X_{ii} = 1. \end{aligned} \quad (38)$$

Min vertex cover A standard formulation of the minimum vertex cover problem as a quadratic integer program is as following:

$$\begin{aligned} \min_{\mathbf{x} \in \{-1, 1\}^n} \quad & \sum_i \frac{1}{2}(1 + x_i) \\ \text{s.t.} \quad & (1 - x_i)(1 - x_j) = 0, \forall \{i, j\} \in E \end{aligned} \quad (39)$$

The SDP relaxation is as following (Kleinberg & Goemans, 1998; Hatami et al., 2009), where we introduce an extra variable:

$$\begin{aligned} \min_{\mathbf{X} \in \mathbb{S}_+^{n+1}} \quad & \sum_i \frac{1}{2}(1 + X_{0i}) \\ \text{s.t.} \quad & X_{ij} + X_{00} - X_{0i} - X_{0j} = 0, \forall \{i, j\} \in E \\ & X_{ii} = 1, i \in \{0\} \cup V. \end{aligned} \quad (40)$$

Max 2-SAT A max SAT problem is given a number of variables n , that takes on the values $\{-1, 1\}^n$, and a collection of clauses $\{C_1, C_2, \dots, C_k\}$, each clause is a disjunction (logical OR) of a subset of literals. A literal is either a variable or its negation. Each clause may be assigned a weight, and the goal of Max SAT is to find an assignment of variables that maximizes the total weight of satisfied clauses. A max k -SAT problem is one in which each clause has at most k literals.

An instance with n variables and k clauses can be represented by a matrix $\mathbf{A} \in \{-1, 0, 1\}^{k \times n}$, where $A_{ij} = 1$ if clause i includes the positive literal of variable j , $A_{ij} = -1$ if it includes the negated literal, and $A_{ij} = 0$ otherwise. We consider the unweighted setting, where all clause weights are equal, and the objective is to maximize the number of satisfied clauses. The quadratic assignment formulation of the max 2-SAT problem is (de Klerk & Warners, 2002)

$$\min_{\mathbf{x} \in \{-1, 1\}^n} \quad \frac{1}{8} (\mathbf{x}^\top \mathbf{A}^\top \mathbf{A} \mathbf{x} - 2 \mathbf{1}^\top \mathbf{A} \mathbf{x}) \quad (41)$$

whose SDP relaxation is

$$\begin{aligned} \min_{\mathbf{X} \in \mathbb{S}_+^{n+1}} \quad & \frac{1}{8} \left\langle \begin{bmatrix} \mathbf{A}^\top \mathbf{A} - \text{diag}(\mathbf{A}^\top \mathbf{A}) & -\mathbf{A}^\top \mathbf{1} \\ -\mathbf{1}^\top \mathbf{A} & 0 \end{bmatrix}, \mathbf{X} \right\rangle \\ \text{s.t.} \quad & X_{ii} = 1, \forall i \in [n]. \end{aligned} \quad (42)$$

D.2. Quadratic assignment problem

Besides, SDP is also a common relaxation for quadratic assignment problems (QAP).

Traveling salesman problem It is already known that the symmetric traveling salesman problem (TSP) is a special case of QAP. We first introduce the notation of circulant matrices on n vertices $\mathcal{C}^n$. A circulant matrix in $\mathbb{R}^{n \times n}$ has the form

$$\begin{bmatrix} c_0 & c_1 & c_2 & c_3 & \cdots & c_{n-1} \\ c_{n-1} & c_0 & c_1 & c_2 & \cdots & c_{n-2} \\ c_{n-2} & c_{n-1} & c_0 & c_1 & \cdots & c_{n-3} \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ c_1 & c_2 & c_3 & c_4 & \cdots & c_0 \end{bmatrix}$$

We have a basis for symmetric circulant matrices consisting of $\{\mathbf{C}_1^n, \mathbf{C}_2^n, \dots, \mathbf{C}_d^n\}$, where $\mathbf{C}_i^n$ is a matrix with $c_i = c_{n-i} = 1$ and 0 elsewhere. Besides, we have a symmetric matrix $\mathbf{D}$ where $D_{ii} = 0$ and $D_{ij} = D_{ji} > 0$ is the cost per edge. The objective of TSP is

$$\min_{\mathbf{X} \in \mathbb{P}^n} \frac{1}{2} \text{Tr}(\mathbf{D} \mathbf{X} \mathbf{C}_1^n \mathbf{X}^\top), \quad (43)$$

where $\mathbb{P}$ is the permutation matrix group. $\mathbf{X} \mathbf{C}_1^n \mathbf{X}$ can be interpreted as the adjacency matrix of the tour.

(Sub)graph isomorphism problem Given adjacency matrices $\mathbf{A}, \mathbf{B}$, supposed to be the same shape, we seek the largest isomorphic subgraphs by

$$\min_{\mathbf{X} \in \mathbb{P}^n} \|\mathbf{X} \mathbf{A} - \mathbf{B} \mathbf{X}\|_F. \quad (44)$$

With some transformations, we have $\|XA - BX\|_F = \|A\|_F - 2\text{Tr}(BXAX^\top) + \|B\|_F$, thus, the problem is equivalent to

$$\max_{X \in \mathbb{P}^n} \text{Tr}(BXAX^\top). \quad (45)$$

The TSP and GIP are special cases of QAP. The relaxation of SDP for QAP has been well-studied (Povh & Rendl, 2009; Oliveira et al., 2018).

D.3. Subsumption of convex optimization

SDP subsumes a lot of convex optimization problems, including second-order conic programming (SOCP), quadratically-constrained quadratic programming (QCQP), linearly-constrained quadratic programming (LCQP), and linear programming (LP) (Dattorro, 2010, p. 220).

LP It is straightforward to show that LP is a special case of SDP. Given an LP instance

$$\begin{aligned} \min_{\mathbf{x} \in \mathbb{R}_{\geq 0}^n} \quad & \mathbf{c}^\top \mathbf{x} \\ \text{s.t.} \quad & \mathbf{A}\mathbf{x} = \mathbf{b}, \end{aligned} \quad (46)$$

we can turn it into an SDP instance by constructing $\mathbf{C} = \text{diag}(\mathbf{c})$ and $\mathbf{A}_i = \text{diag}(\mathbf{A}_i)$, and extra constraints $X_{ij} = 0, i \neq j$. The positive semidefiniteness of SDP variables perfectly captures the positivity of the LP variables, as the SDP variables are forced to form a diagonal matrix.

SOCP The formulation of an SOCP is

$$\begin{aligned} \min \quad & \mathbf{c}^\top \mathbf{x} \\ \text{s.t.} \quad & \|\mathbf{A}_{(i)}\mathbf{x} + \mathbf{b}_{(i)}\|_2 \leq \mathbf{c}_{(i)}^\top \mathbf{x} + d_{(i)}, \forall i \in [m], \end{aligned} \quad (47)$$

where $\mathbf{A}_{(i)} \in \mathbb{R}^{k_i \times n}, \mathbf{b}_{(i)} \in \mathbb{R}^{k_i}, \mathbf{c}_{(i)} \in \mathbb{R}^n, d_{(i)} \in \mathbb{R}$. The positivity constraints of variables can also be captured by such forms. Each constraint is in a second order cone $\mathcal{L}^{k+1} := \{(\mathbf{x}, t) \in \mathbb{R}^{k+1} \mid \|\mathbf{x}\|_2 \leq t\}$, hence the name. Using Schur decomposition, we turn the second order cone $\|\mathbf{x}\|_2 \leq t$ into a PSD cone $\begin{bmatrix} t & \mathbf{x}^\top \\ \mathbf{x} & t\mathbf{I} \end{bmatrix} \succeq 0$. So we can turn the SOCP problem into an SDP problem (Lobo et al., 1998)

$$\begin{aligned} \min \quad & \mathbf{c}^\top \mathbf{x} \\ \text{s.t.} \quad & \begin{bmatrix} \mathbf{c}_{(i)}^\top \mathbf{x} + d_{(i)} & (\mathbf{A}_{(i)}\mathbf{x} + \mathbf{b}_{(i)})^\top \\ \mathbf{A}_{(i)}\mathbf{x} + \mathbf{b}_{(i)} & (\mathbf{c}_{(i)}^\top \mathbf{x} + d_{(i)})\mathbf{I} \end{bmatrix} \succeq 0, \forall i \in [m]. \end{aligned} \quad (48)$$

Note that it is not the standard primal form of Equation (1), but a dual form

$$\begin{aligned} \min \quad & \mathbf{b}^\top \mathbf{y} \\ \text{s.t.} \quad & \mathbf{C} + \sum_{i=1}^m y_i \mathbf{A}_i \succeq 0. \end{aligned} \quad (49)$$

QP To show that LCQPs and QCQPs are also special cases of SDP, we just need to show they are included in the SOCP family. Given a QCQP instance

$$\begin{aligned} \min \quad & \mathbf{x}^\top \mathbf{Q}_{(0)}\mathbf{x} + 2\mathbf{q}_{(0)}^\top \mathbf{x} + q_{(0)} \\ \text{s.t.} \quad & \mathbf{x}^\top \mathbf{Q}_{(i)}\mathbf{x} + 2\mathbf{q}_{(i)}^\top \mathbf{x} + q_{(i)} \leq 0, \forall i \in [m], \end{aligned} \quad (50)$$

for the problem to be convex, we assume $\mathbf{Q}_{(i)}$ to be PSD matrices. This allows us to transform the QCQP in the following form (Lobo et al., 1998):

$$\begin{aligned} \min \quad & \|\mathbf{Q}_{(0)}^{1/2}\mathbf{x} + \mathbf{Q}_{(0)}^{-1/2}\mathbf{q}_{(0)}\|_2^2 + c_{(0)} - \mathbf{q}_{(0)}^\top \mathbf{Q}_{(0)}^{-1}\mathbf{q}_{(0)} \\ \text{s.t.} \quad & \|\mathbf{Q}_{(i)}^{1/2}\mathbf{x} + \mathbf{Q}_{(i)}^{-1/2}\mathbf{q}_{(i)}\|_2^2 + c_{(i)} - \mathbf{q}_{(i)}^\top \mathbf{Q}_{(i)}^{-1}\mathbf{q}_{(i)} \leq 0, \forall i \in [m], \end{aligned} \quad (51)$$

and further in SOCP form

$$\begin{aligned} \min \quad & t \\ \text{s.t.} \quad & \|Q_{(i)}^{1/2} \mathbf{x} + Q_{(i)}^{-1/2} \mathbf{q}_{(i)}\|_2 \leq \left(\mathbf{q}_{(i)}^\top Q_{(i)}^{-1} \mathbf{q}_{(i)} - c_{(i)} \right)^{1/2}, \forall i \in [m] \\ & \|Q_{(0)}^{1/2} \mathbf{x} + Q_{(0)}^{-1/2} \mathbf{q}_{(0)}\|_2 \leq t. \end{aligned} \quad (52)$$

And LCQP is a special case of QCQP.

D.4. Control theory

Linear Matrix Inequality Lyapunov stability analysis is a fundamental problem in control theory used to verify if a dynamical system $\dot{z} = \mathbf{A}_{sys} z$ is stable. A necessary and sufficient condition for stability is the existence of a quadratic Lyapunov function $V(z) = z^\top \mathbf{P} z$ with $\mathbf{P} \succ 0$ such that the derivative $\dot{V}(z) < 0$. Formally, this is the Linear Matrix Inequality (LMI):

$$\begin{aligned} \text{Find} \quad & \mathbf{P} \in \mathbb{S}_+^n \\ \text{s.t.} \quad & \mathbf{A}_{sys}^\top \mathbf{P} + \mathbf{P} \mathbf{A}_{sys} \prec 0. \end{aligned} \quad (53)$$

To optimize numerical properties or check for specific performance bounds, this is cast as a Semidefinite Program. We adopt the formulation from [Boyd et al. \(1994, p. 9\)](#), enforcing a trace constraint to bound the solution scale and discretizing the matrix inequality into m scalar linear constraints:

$$\begin{aligned} \min_{\mathbf{P} \in \mathbb{S}_+^n} \quad & \langle \mathbf{C}, \mathbf{P} \rangle \\ \text{s.t.} \quad & \text{Tr}(\mathbf{P}) = 1 \\ & \mathcal{A}(\mathbf{P}) = \mathbf{b}. \end{aligned} \quad (54)$$

where each constraint matrix $\mathbf{A}_k$ represents the projection of the Lyapunov operator onto a random direction vector v_k :

$$\mathbf{A}_k = \frac{1}{2} (\mathbf{A}_{sys} v_k v_k^\top + v_k v_k^\top \mathbf{A}_{sys}^\top).$$

We employ an inverse strategy to generate feasible problem instances with known stability certificates. We first sample a random positive definite matrix $\mathbf{P}_{true}$ and normalize it such that $\text{Tr}(\mathbf{P}_{true}) = 1$. Instead of sampling a random system $\mathbf{A}_{sys}$ (which might be unstable), we construct a stable system compatible with our certificate. We solve the inverse Lyapunov equation $\mathbf{A}_{sys}^\top \mathbf{P}_{true} + \mathbf{P}_{true} \mathbf{A}_{sys} = -\epsilon \mathbf{I}$ to recover $\mathbf{A}_{sys}$. We sample m random sparse direction vectors v_k . We compute the RHS scalars $\langle \mathbf{A}_k, \mathbf{P}_{true} \rangle = \mathbf{b}$. This ensures that the ground truth $\mathbf{P}_{true}$ lies exactly on the boundary of the feasible region defined by the hyperplanes.

E. Other architectures

E.1. Ablation of VC-2-FWL

VC-2-FWL+ We define a 2-tuple based 2-FWL-like architecture, which we name VC-2-FWL+. The color initialization follows Equation (2), and the update is as defined below:

$$\begin{aligned} \mathbf{v}_{ij}^t &:= \text{hash} \left(\mathbf{v}_{ij}^{t-1}, \left\{ \left(\mathbf{v}_{uj}^{t-1}, \mathbf{v}_{iu}^{t-1} \right) \mid u \in [n] \right\}, \left\{ \left(\mathbf{A}_{k,ij}, \mathbf{c}_k^{t-1} \right) \mid k \in N(ij) \right\} \right) \\ \mathbf{c}_k^t &:= \text{hash} \left(\mathbf{c}_k^{t-1}, \left\{ \left(\mathbf{A}_{k,ij}, \mathbf{v}_{ij}^{t-1} \right) \mid (i,j) \in N(k) \right\} \right). \end{aligned} \quad (55)$$

Similar to VC-2-WL, it requires symmetrization after the update $\mathbf{c}_{ji}^t \leftarrow \mathbf{c}_{ij}^t, \forall i < j$. See Proposition C.7 for the proof of the expressive power between VC-2-FWL and VC-2-FWL+.

Besides, we discuss some interesting yet weaker versions of VC-2-FWL.

Multiset encoding across constraint We can encode the problem instance $I := (C, \mathbf{A}, \mathbf{b})$ with an injective function on a multiset, in the form of

$$\mathbf{v}_{ij}^0 := \text{hash} \left(C_{ij}, \{ \{ (A_{k,ij}, b_k) \mid k \in [m] \} \} \right),$$

such that the constraint elements on entry (i, j) are aggregated in an injective and permutation invariant way, then merged with the objective coefficient C_{ij} . Then we execute the following multiset-based 2-FWL:

$$\mathbf{v}_{ij}^t := \text{hash} \left(\mathbf{v}_{ij}^{t-1}, \{ \{ \{ \mathbf{v}_{uj}^{t-1}, \mathbf{v}_{iu}^{t-1} \} \mid u \in [n] \} \} \right)$$

Unfortunately, this aggregation is not expressive enough; we can construct a problem:

$$\mathbf{C} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \quad \mathbf{A}_1 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}, \quad \mathbf{A}_2 = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \quad \mathbf{b} = [1 \quad 1]$$

whose optimal solution has the form $\mathbf{X}^* = \begin{bmatrix} 1-\lambda & 0 & 0 \\ 0 & \lambda & 0 \\ 0 & 0 & 1 \end{bmatrix}$, where $\lambda \in [0, 1]$. The solution with minimum Frobenius

norm is $\begin{bmatrix} 0.5 & 0 & 0 \\ 0 & 0.5 & 0 \\ 0 & 0 & 1 \end{bmatrix}$.

However, the encoding

$$\begin{aligned} \mathbf{v}_{22}^0 &= \text{hash} \left(1, \{ \{ (1, 1), (0, 1) \} \} \right) \\ \mathbf{v}_{33}^0 &= \text{hash} \left(1, \{ \{ (0, 1), (1, 1) \} \} \right) \end{aligned}$$

are the same. Later, one can easily verify that the converged colors $\mathbf{v}_{22}^\infty = \mathbf{v}_{33}^\infty$.

The failing of such a combination lies in that the encoding cannot distinguish some structural difference, for example, above, b_1 is related with 2 elements in $\mathbf{A}_1$, while b_2 associates with only one in $\mathbf{A}_2$. The encoding with lack of expressivity will falsely encode variables with different structural roles as the same.

VC-WL then 2-FWL Another interesting ablation of Equation (4) is, can we run VC-WL on the V-C bipartite graph until convergence, then run multiset-based 2-FWL until convergence, and obtain the same expressivity as Equation (4)? Therefore, it effectively splits the aggregation of VC-2-FWL. The answer is no. We give a counterexample.

$$\begin{aligned} \mathbf{C} &= \mathbf{1}_{5 \times 5} \\ \mathbf{A}_1 &= \begin{bmatrix} 1 & 2 & 1 & 1 & 1 \\ 2 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 \end{bmatrix}, \quad \mathbf{A}_2 = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}, \quad \mathbf{A}_3 = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \end{bmatrix} \\ \mathbf{A}_4 &= \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 1 & 0 \end{bmatrix}, \quad \mathbf{A}_5 = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix} \\ \mathbf{b} &= [1 \quad 1 \quad 1 \quad 1 \quad 1]. \end{aligned} \tag{56}$$

In this example, after the convergence of VC-WL, the converged colors $\mathbf{v}_{ij}^\infty$ will be $\begin{bmatrix} a & b & e & e & e \\ b & d & c & c & c \\ e & c & d & c & c \\ e & c & c & d & c \\ e & c & c & c & f \end{bmatrix}$, where $\{a, b, c, \dots\}$ are from an alphabet to represent colors.

After the convergence of multiset-based 2-FWL, the converged colors will be $\begin{bmatrix} a & b & c & c & d \\ b & e & f & f & g \\ c & f & k & h & i \\ c & f & h & k & i \\ d & g & i & i & j \end{bmatrix}$. The solution is approximately $\begin{bmatrix} 4.682 & 1.821 & -4.889 & 0.000 & -0.594 \\ 1.821 & 4.327 & -1.664 & 0.000 & -2.060 \\ -4.889 & -1.664 & 5.121 & 0.000 & 0.500 \\ 0.000 & 0.000 & 0.000 & 0.000 & 0.000 \\ -0.594 & -2.060 & 0.500 & 0.000 & 1.000 \end{bmatrix}$. We can see the pair e.g., X_{13} and X_{14} have respectively different values in their solution, yet cannot be distinguished.

The limitation of such a design lies in the message passing cannot distinguish some variables in different constraints, and these variables may have non-equivalent positions in the matrix and play different roles in the matrix.

Therefore, one round of sequential execution of VC-WL and multiset-based 2-FWL does not suffice. Empirically, multiset-based 2-FWL might break the previous VC-WL symmetry. One has to execute both in turn until they both converge. However, this might introduce more steps until convergence.

E.2. Local VC-2-WL

In addition to our VC-2-WL stemmed from standard 2-WL, we evaluate a stronger variant named δ -VC-2-WL based on the δ - k -WL framework of Morris et al. (2020), adapted here for $k = 2$. The color initialization follows Equation (2), and the constraint node update follows Equation (3). But the refinement step of variable nodes is augmented to explicitly incorporate local connectivity:

$$\mathbf{v}_{ij}^t := \text{hash}\left(\mathbf{v}_{ij}^{t-1}, \left\{\left\{\left(\mathbf{v}_{uj}^{t-1}, \text{adj}(C_{ui})\right) \mid u \in [n]\right\}\right\}, \left\{\left\{\left(\mathbf{v}_{iu}^{t-1}, \text{adj}(C_{uj})\right) \mid u \in [n]\right\}\right\}, \left\{\left\{\left(A_{k,ij}, \mathbf{c}_k^{t-1}\right) \mid k \in N(ij)\right\}\right\}\right)$$

where $\text{adj}(C_{ij})$ is an indicator function representing the sparsity pattern of the cost matrix C , i.e., taking the value 1 if $C_{ij} \neq 0$, and 0 otherwise.

In Morris et al. (2020), it is established that δ - k -WL is strictly more expressive than standard k -WL for graph isomorphism testing. In our SDP context, we prove the same result.

Proposition E.1. *The δ -VC-2-WL algorithm strictly refines VC-2-WL, denoted as*

$$\delta\text{-VC-2-WL} \sqsubset \text{VC-2-WL}.$$

Proof. By definition, δ -VC-2-WL initialization and constraint node update are identical to VC-2-WL, therefore, we focus on variable node update. By observation, the information aggregated by δ -VC-2-WL is a superset of VC-2-WL. Therefore, we immediately have $\delta\text{-VC-2-WL} \sqsubseteq \text{VC-2-WL}$, because the variables indistinguishable to δ -VC-2-WL must also be the same under VC-2-WL.

To show the strict refinement, we construct a sparse example where the adjacency information is non-trivial. Consider an SDP instance defined by

$$C = \begin{bmatrix} 0 & 1 & 0 & 0 & 1 & 1 \\ 1 & 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 \\ 0 & 1 & 1 & 0 & 0 & 1 \\ 1 & 1 & 1 & 0 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 & 0 \end{bmatrix}, \quad A_1 = I_6, \quad \mathbf{b} = [1]. \quad (57)$$

The C is constructed as the adjacency matrix of a regular graph. Because all the row and column multisets are identical, the

VC-2-WL converges to the color configuration $\begin{bmatrix} a & b & c & c & b & b \\ b & a & c & b & b & c \\ c & c & a & b & b & b \\ c & b & b & a & c & b \\ b & b & b & c & a & c \\ b & c & b & b & c & a \end{bmatrix}$ immediately after initialization, where $\{a, b, \dots\}$

are colors from an alphabet. Thus, the variables $(1, 6)$ and $(2, 5)$ are not distinguished by VC-2-WL.

For the update of δ -VC-2-WL, we have at initialization $v_{16}^0 = v_{25}^0 = b$, while

$$\begin{aligned} v_{16}^1 &:= \text{hash} \left(v_{16}^0, \{ \{ (v_{16}^0, \text{adj}(C_{11})), (v_{26}^0, \text{adj}(C_{12})), (v_{36}^0, \text{adj}(C_{13})), (v_{46}^0, \text{adj}(C_{14})), (v_{56}^0, \text{adj}(C_{15})), (v_{66}^0, \text{adj}(C_{16})) \} \}, \right. \\ &\quad \left. \{ \{ (v_{11}^0, \text{adj}(C_{16})), (v_{12}^0, \text{adj}(C_{26})), (v_{13}^0, \text{adj}(C_{36})), (v_{14}^0, \text{adj}(C_{46})), (v_{15}^0, \text{adj}(C_{56})), (v_{16}^0, \text{adj}(C_{66})) \} \} \right) \\ &= \left(b, \{ \{ (b, 0), (c, 1), (b, 0), (b, 0), (c, 1)(a, 1) \} \}, \{ \{ (a, 1), (b, 0), (c, 1), (c, 1), (b, 0)(b, 0) \} \} \right) \end{aligned}$$

and

$$\begin{aligned} v_{25}^1 &:= \text{hash} \left(v_{25}^0, \{ \{ (v_{15}^0, \text{adj}(C_{21})), (v_{25}^0, \text{adj}(C_{22})), (v_{35}^0, \text{adj}(C_{23})), (v_{45}^0, \text{adj}(C_{24})), (v_{55}^0, \text{adj}(C_{25})), (v_{65}^0, \text{adj}(C_{26})) \} \}, \right. \\ &\quad \left. \{ \{ (v_{21}^0, \text{adj}(C_{15})), (v_{22}^0, \text{adj}(C_{25})), (v_{23}^0, \text{adj}(C_{35})), (v_{24}^0, \text{adj}(C_{45})), (v_{25}^0, \text{adj}(C_{55})), (v_{26}^0, \text{adj}(C_{56})) \} \} \right) \\ &= \left(b, \{ \{ (b, 1), (b, 0), (b, 0), (c, 1), (a, 1), (c, 0) \} \}, \{ \{ (b, 1), (a, 1), (c, 1), (b, 0), (b, 0), (c, 0), \} \} \right) \end{aligned}$$

We notice that in the first multiset, v_{25}^1 has a $(b, 1)$ that v_{16}^1 doesn't, so $(1, 6)$ and $(2, 5)$ are distinguished by δ -VC-2-WL.

Therefore, there are variables where δ -VC-2-WL can distinguish while VC-2-WL cannot. That completes the proof that δ -VC-2-WL $\sqsupset$ VC-2-WL. $\square$

However, we demonstrate that this increased expressivity remains insufficient for approximating SDP solutions.

Proposition E.2. *The δ -VC-2-WL fails to represent linear SDPs. That is, there exist instances where the stable colors under δ -VC-2-WL satisfy $v_{ij}^\infty = v_{pq}^\infty$ for distinct indices (i, j) and (p, q) , yet the unique optimal solution entries differ $X_{ij}^* \neq X_{pq}^*$.*

Proof. We prove by construction with a counterexample.

Consider the one established in the proof of Proposition 2.2. If the cost matrix C in that instance is fully dense, the indicator term adj outputs a constant value and provides no extra information. Consequently, the δ -VC-2-WL update rule degenerates to the standard VC-2-WL rule, showing the same inability to distinguish the optimal solution entries. $\square$

Similar to the analysis between VC-2-FWL and VC-2-WL, we show that VC-2-FWL and δ -VC-2-WL are incomparable.

Proposition E.3. *δ -VC-2-WL and VC-2-FWL are incomparable in expressive power, denoted as*

$$\delta\text{-VC-2-WL} \not\equiv \text{VC-2-FWL}.$$

Proof. The proof is straightforward with the same logic as Proposition C.8. First, we can reuse the same instance in Proposition C.8, where VC-2-WL can distinguish a pair of variables and VC-2-FWL cannot. Since δ -VC-2-WL strictly refines VC-2-WL, it is also stronger than VC-2-FWL on that example. Second, we reuse the example in Proposition 2.2, where δ -VC-2-WL fails while VC-2-FWL can effectively distinguish all the variables except symmetric ones.

Therefore, δ -VC-2-WL and VC-2-FWL are incomparable in expressive power. $\square$

Similar to VC-2-MPNN, we can neuralize δ -VC-2-WL into δ -VC-2-MPNN:

$$\begin{aligned}
 \mathbf{m}_{ij,\text{row}}^t &:= \sum_{u \in [n]} \text{MSG}_{\text{row}}^t(\mathbf{h}_{iu}^{t-1}, \text{adj}(C_{uj})) \\
 \mathbf{m}_{ij,\text{col}}^t &:= \sum_{u \in [n]} \text{MSG}_{\text{col}}^t(\mathbf{h}_{uj}^{t-1}, \text{adj}(C_{iu})) \\
 \mathbf{m}_{ij,\text{c} \rightarrow \text{v}}^t &:= \sum_{k \in N(ij)} \text{MSG}_{\text{c} \rightarrow \text{v}}^t(A_{k,ij}, \mathbf{h}_k^{t-1}) \\
 \mathbf{m}_{k,\text{v} \rightarrow \text{c}}^t &:= \sum_{(i,j) \in N(k)} \text{MSG}_{\text{v} \rightarrow \text{c}}^t(A_{k,ij}, \mathbf{h}_{ij}^{t-1}) \\
 \mathbf{h}_{ij}^t &:= \text{UPD}_{\text{v}}^t(\mathbf{h}_{ij}^{t-1}, \mathbf{m}_{ij,\text{col}}^t, \mathbf{m}_{ij,\text{row}}^t, \mathbf{m}_{ij,\text{c} \rightarrow \text{v}}^t) \\
 \mathbf{h}_k^t &:= \text{UPD}_{\text{c}}^t(\mathbf{h}_k^{t-1}, \mathbf{m}_{k,\text{v} \rightarrow \text{c}}^t)
 \end{aligned}$$

which, similar to Proposition C.26, is not universal approximating.

E.3. VC-2-IGN

As an alternative instantiation of the variable update mechanism, we consider the 2-order Invariant Graph Network (2-IGN), based on the theoretical characterization of permutation equivariant layers by Maron et al. (2019b).

The 2-IGN operates on second-order tensors. For a general linear layer mapping $\mathbb{R}^{n \times n} \rightarrow \mathbb{R}^{n \times n}$ (feature dimension omitted for simplicity), the space of permutation equivariant operations is spanned by a basis of size $b(2+2) = 15$, where $b(\cdot)$ denotes the Bell number. However, our application to SDPs imposes a strict geometric constraint: both the input feature matrix and the output updated features must lie in the symmetric space $\mathbb{S}^n$. This symmetry constraint reduces the dimension of the equivariant basis from 15 to 9. The reduction occurs through two mechanisms:

1. **Input symmetry:** Since $\mathbf{X}_{ij} = \mathbf{X}_{ji}$, operations that distinguish rows from columns become redundant, e.g., summing over rows yields the same vector as summing over columns.
2. **Output symmetry:** To guarantee the output is symmetric, the corresponding basis operations must share weights. For instance, broadcasting a vector to rows must be paired with broadcasting to columns.

Formally, let $\mathbf{1} \in \mathbb{R}^n$ be the all-ones vector, $\mathbf{I}$ be the identity matrix, $\mathbf{d} = \text{diag}(\mathbf{X}) \in \mathbb{R}^n$ be the vector of diagonal entries, and $\mathbf{s} = \mathbf{X}\mathbf{1} \in \mathbb{R}^n$ be the vector of row sums. We identify the following 9 basis operations $\mathcal{B} = \{\text{Op}_1, \dots, \text{Op}_9\}$ that map a symmetric $\mathbf{X}$ to a symmetric output:

1. $\text{Op}_1(\mathbf{X}) = \mathbf{X}$. Identity.
2. $\text{Op}_2(\mathbf{X}) = \text{Tr}(\mathbf{X})\mathbf{1}\mathbf{1}^\top$. Global trace broadcast.
3. $\text{Op}_3(\mathbf{X}) = (\mathbf{1}^\top \mathbf{X} \mathbf{1})\mathbf{1}\mathbf{1}^\top$. Global sum broadcast.
4. $\text{Op}_4(\mathbf{X}) = \text{diag}(\mathbf{d})$. Copy diagonal.
5. $\text{Op}_5(\mathbf{X}) = \text{diag}(\mathbf{s})$. Row sums to diagonal.
6. $\text{Op}_6(\mathbf{X}) = \text{Tr}(\mathbf{X})\mathbf{I}$. Trace to diagonal.
7. $\text{Op}_7(\mathbf{X}) = (\mathbf{1}^\top \mathbf{X} \mathbf{1})\mathbf{I}$. Global sum to diagonal.
8. $\text{Op}_8(\mathbf{X}) = \mathbf{s}\mathbf{1}^\top + \mathbf{1}\mathbf{s}^\top$. Row/Col sums broadcast.
9. $\text{Op}_9(\mathbf{X}) = \mathbf{d}\mathbf{1}^\top + \mathbf{1}\mathbf{d}^\top$. Diagonal entries broadcast.

In the context of deep learning, we extend these operations to multi-dimensional inputs $\mathbf{X} \in \mathbb{R}^{n \times n \times f}$. For each basis operation $\text{Op}_b \in \mathcal{B}$, we assign a learnable weight matrix $\mathbf{W}_b \in \mathbb{R}^{f \times f'}$. The 2-IGN layer update is defined as the aggregation of these transformed bases:

$$\mathbf{X}' = \sum_{b=1}^9 \text{Op}_b(\mathbf{X})\mathbf{W}_b \in \mathbb{R}^{n \times n \times f'}.$$

This formulation ensures that the neural network remains strictly equivariant and preserves the symmetric matrix structure required by the SDP relaxation.

We wrap this up as a neural network IGN and plug it into our V-C bipartite graph message passing, and we have the following form of VC-2-IGN:

$$\begin{aligned}
 \mathbf{m}_{ij, \text{IGN}}^t &:= \text{IGN}^t(\mathbf{H}^{t-1})_{ij} \\
 \mathbf{m}_{ij, c \rightarrow v}^t &:= \sum_{k \in N(ij)} \text{MSG}_{c \rightarrow v}^t(A_{k, ij}, \mathbf{h}_k^{t-1}) \\
 \mathbf{m}_{k, v \rightarrow c}^t &:= \sum_{(i, j) \in N(k)} \text{MSG}_{v \rightarrow c}^t(A_{k, ij}, \mathbf{h}_{ij}^{t-1}) \\
 \mathbf{h}_{ij}^t &:= \text{UPD}_v^t(\mathbf{h}_{ij}^{t-1}, \mathbf{m}_{ij, \text{IGN}}^t, \mathbf{m}_{ij, c \rightarrow v}^t) \\
 \mathbf{h}_k^t &:= \text{UPD}_c^t(\mathbf{h}_k^{t-1}, \mathbf{m}_{k, v \rightarrow c}^t)
 \end{aligned}$$

where $\mathbf{H}^{t-1} \in \mathbb{R}^{n \times n \times d}$ is the tensor of variable features $\mathbf{h}_{ij}^{t-1}$.

Proposition E.4. *VC-2-IGN is not a universal approximator for SDPs. Given $\forall \epsilon > 0$, there always exists an SDP problem, such that for all VC-2-IGN, the error between the predicted solution $\mathbf{X}'$ and the optimal solution $\mathbf{X}^*$ satisfies*

$$\|\mathbf{X}^* - \mathbf{X}'\|_\infty > \epsilon.$$

We can easily prove in the same way as Proposition 2.2. VC-2-IGN fails on that example too, because the row and column multisets are identical $\{1, 2, 3, 4, 5, 6\}$, and the diagonal elements are the same. We omit the proof here.

While the 2-IGN architecture is defined via continuous linear operations on tensors, comparing its expressivity directly to the discrete VC-2-WL requires a discrete analogue. To this end, we define the VC-2-IGNWL algorithm. This variant adapts some of the 2-IGN basis operations, specifically the row/column aggregations and diagonal broadcasting, because the other operations do not contribute to expressivity.

$$\begin{aligned}
 \mathbf{v}_{ij}^t &:= \text{hash}\left(\mathbf{v}_{ij}^{t-1}, \{\{\mathbf{v}_{uj}^{t-1} \mid u \in [n]\}\}, \{\{\mathbf{v}_{iu}^{t-1} \mid u \in [n]\}\}, \right. \\
 &\quad \left. \{\{(A_{k, ij}, \mathbf{c}_k^{t-1}) \mid k \in N(ij)\}\}, \mathbf{v}_{ii}^t, \mathbf{v}_{jj}^t\right) \\
 \mathbf{c}_k^t &:= \text{hash}\left(\mathbf{c}_k^{t-1}, \{\{(A_{k, ij}, \mathbf{v}_{ij}^{t-1}) \mid (i, j) \in N(k)\}\}\right).
 \end{aligned} \tag{58}$$

Notably, we replace the sum over rows and columns with hash of the multisets $\{\{\mathbf{v}_{uj}^{t-1} \mid u \in [n]\}\}, \{\{\mathbf{v}_{iu}^{t-1} \mid u \in [n]\}\}$, which boosts the expressivity because of the injectivity of hash function. The constraint update is the same as Equation (3).

We now establish the expressivity hierarchy between this variant and the VC-2-WL.

Proposition E.5. *VC-2-WL has expressivity power equivalent to VC-2-IGNWL, denoted as*

$$\text{VC-2-WL} \equiv \text{VC-2-IGNWL}.$$

Proof. First, we show $\text{VC-2-IGNWL} \sqsubseteq \text{VC-2-WL}$. For $t = 0$, since their initialization is the same, VC-2-IGNWL trivially refines VC-2-WL. For any $t > 0$, if the coloring of VC-2-IGNWL cannot distinguish 2 variable, then neither can VC-2-WL, because the color information used by VC-2-IGNWL includes that used by the VC-2-WL.

Then, we show $\text{VC-2-WL} \sqsubseteq \text{VC-2-IGNWL}$. For $t = 0$, since their initialization is the same, it trivially holds. For $t > 0$, if

the color refinement under VC-2-WL cannot distinguish 2 variable (i, j) and (p, q) , we have:

$$\begin{aligned}
 & \mathbf{v}_{ij}^t = \mathbf{v}_{pq}^t \\
 \implies & \text{hash}\left(\mathbf{v}_{ij}^{t-1}, \{\{\mathbf{v}_{uj}^{t-1} \mid u \in [n]\}\}, \{\{\mathbf{v}_{iu}^{t-1} \mid u \in [n]\}\}, \right. \\
 & \quad \left. \{\{(A_{k,ij}, \mathbf{c}_k^{t-1}) \mid k \in N(ij)\}\}\right) \\
 = & \text{hash}\left(\mathbf{v}_{pq}^{t-1}, \{\{\mathbf{v}_{uq}^{t-1} \mid u \in [n]\}\}, \{\{\mathbf{v}_{pu}^{t-1} \mid u \in [n]\}\}, \right. \\
 & \quad \left. \{\{(A_{k,pq}, \mathbf{c}_k^{t-1}) \mid k \in N(pq)\}\}\right) \\
 \implies & \mathbf{v}_{ij}^{t-1} = \mathbf{v}_{pq}^{t-1} \\
 & \wedge \{\{\mathbf{v}_{uj}^{t-1} \mid u \in [n]\}\} = \{\{\mathbf{v}_{uq}^{t-1} \mid u \in [n]\}\} \\
 & \wedge \{\{\mathbf{v}_{iu}^{t-1} \mid u \in [n]\}\} = \{\{\mathbf{v}_{pu}^{t-1} \mid u \in [n]\}\} \\
 & \wedge \{\{(A_{k,ij}, \mathbf{c}_k^{t-1}) \mid k \in N(ij)\}\} = \{\{(A_{k,pq}, \mathbf{c}_k^{t-1}) \mid k \in N(pq)\}\}
 \end{aligned}$$

Recall in Equation (2) that diagonal elements are specially labeled, they receive unique labels that distinguish them from non-diagonal elements. That means the diagonal elements within e.g. $\{\{\mathbf{v}_{uj}^{t-1} \mid u \in [n]\}\}$ and $\{\{\mathbf{v}_{uq}^{t-1} \mid u \in [n]\}\}$ are unique, and they must be the same as well. So we can continue the implication above:

$$\begin{aligned}
 \implies & \mathbf{v}_{ij}^{t-1} = \mathbf{v}_{pq}^{t-1} \\
 & \wedge \{\{\mathbf{v}_{uj}^{t-1} \mid u \in [n]\}\} = \{\{\mathbf{v}_{uq}^{t-1} \mid u \in [n]\}\} \wedge \mathbf{v}_{jj}^{t-1} = \mathbf{v}_{qq}^{t-1} \\
 & \wedge \{\{\mathbf{v}_{iu}^{t-1} \mid u \in [n]\}\} = \{\{\mathbf{v}_{pu}^{t-1} \mid u \in [n]\}\} \wedge \mathbf{v}_{ii}^{t-1} = \mathbf{v}_{pp}^{t-1} \\
 & \wedge \{\{(A_{k,ij}, \mathbf{c}_k^{t-1}) \mid k \in N(ij)\}\} = \{\{(A_{k,pq}, \mathbf{c}_k^{t-1}) \mid k \in N(pq)\}\} \\
 \implies & \text{hash}\left(\mathbf{v}_{ij}^{t-1}, \{\{\mathbf{v}_{uj}^{t-1} \mid u \in [n]\}\}, \{\{\mathbf{v}_{iu}^{t-1} \mid u \in [n]\}\}, \right. \\
 & \quad \left. \{\{(A_{k,ij}, \mathbf{c}_k^{t-1}) \mid k \in N(ij)\}\}, \mathbf{v}_{ii}^{t-1}, \mathbf{v}_{jj}^{t-1}\right) \\
 = & \text{hash}\left(\mathbf{v}_{pq}^{t-1}, \{\{\mathbf{v}_{uq}^{t-1} \mid u \in [n]\}\}, \{\{\mathbf{v}_{pu}^{t-1} \mid u \in [n]\}\}, \right. \\
 & \quad \left. \{\{(A_{k,pq}, \mathbf{c}_k^{t-1}) \mid k \in N(pq)\}\}, \mathbf{v}_{pp}^{t-1}, \mathbf{v}_{qq}^{t-1}\right)
 \end{aligned}$$

the last step corresponds to the update of VC-2-IGNWL, showing VC-2-IGNWL also cannot distinguish them.

Therefore, VC-2-WL and VC-2-IGNWL have the same expressive power. $\square$

Following Proposition C.8, we also conclude that VC-2-IGNWL and VC-2-FWL are incomparable in expressive power.

E.4. Edge Transformer

We adapt the Edge Transformer (ET) module introduced by Müller et al. (2024). The ET architecture is structurally analogous to standard 2-FWL. While the original ET operates on graph node pairs, our framework operates on variables in a matrix, sharing the same pairwise form. Furthermore, the ET has been rigorously proven to possess the expressive power of the 2-FWL algorithm. These properties make it a perfect candidate to compare with our VC-2-FMPNN.

The ET operates on a second-order tensor $\mathbf{X} \in \mathbb{R}^{n \times n \times d}$, where d is the embedding dimension and $\mathbf{X}_{ij}$ denotes the feature vector of the variable indexed at (i, j) . Formally, the layer update is defined as:

$$\mathbf{X}_{ij}^t := \text{FFN}^t(\mathbf{X}_{ij}^{t-1} + \text{TriAttn}^t(\text{LN}^t(\mathbf{X}_{ij}^{t-1}))),$$

where FFN is a feed-forward neural network, LN denotes layer normalization (Ba et al., 2016) and TriAttn is defined as

$$\text{TriAttn}(\mathbf{X}_{ij}) := \sum_{l=1}^n \alpha_{ilj} \mathbf{V}_{ilj}. \quad (59)$$

This operation computes a linear combination of a *value vector* V_{ilj} , with weights given by scalar attention scores α_{ilj} , over the summation index l . The attention scores are computed via a scaled dot-product mechanism:

$$\alpha_{ilj} := \text{softmax}_{l \in [n]} \left(\frac{1}{\sqrt{d}} X_{il} W^Q (X_{lj} W^K)^\top \right) \in \mathbb{R} \quad (60)$$

which captures the relevance of the triplet (i, l, j) , and the value vector is computed via the *value fusion* of the intermediate representations

$$V_{ilj} := X_{il} W^{V_1} \odot X_{lj} W^{V_2}, \quad (61)$$

where $\odot$ denotes element-wise multiplication, and $W^Q, W^K, W^{V_1}, W^{V_2} \in \mathbb{R}^{d \times d}$ are learnable projection matrices.

Integrating this module into our framework yields the VC-ET architecture. The initialization follows Equation (5). For the update step, we substitute the variable-to-variable message passing with the ET layer. Let $H^{t-1} \in \mathbb{R}^{n \times n \times d}$ denote the tensor of variable features h_{ij}^{t-1} . The update rules are:

$$\begin{aligned} m_{ij, \text{ET}}^t &:= \text{ET}^t (H^{t-1})_{ij} \\ m_{ij, c \rightarrow v}^t &:= \sum_{k \in N(ij)} \text{MSG}_{c \rightarrow v}^t (A_{k, ij}, h_k^{t-1}) \\ m_{k, v \rightarrow c}^t &:= \sum_{(i, j) \in N(k)} \text{MSG}_{v \rightarrow c}^t (A_{k, ij}, h_{ij}^{t-1}) \\ h_{ij}^t &:= \text{UPD}_v^t (h_{ij}^{t-1}, m_{ij, \text{ET}}^t, m_{ij, c \rightarrow v}^t) \\ h_k^t &:= \text{UPD}_c^t (h_k^{t-1}, m_{k, v \rightarrow c}^t). \end{aligned} \quad (62)$$

Thus, VC-ET effectively replaces the explicit 2-FWL-based aggregation with the learnable, triangular attention of the Edge Transformer.

F. More experiment details

Neural architecture and hyperparameters For the VC-MPNN, the initialization follows Equation (5), implemented via two 2-layer MLPs with ReLU activation. The message passing MSG and update UPD functions, defined in Equation (27), are parameterized by a single linear layer followed by ReLU activation and GraphNorm (Cai et al., 2021). The final prediction head is a 3-layer MLP with ReLU activation, mapping the latent features to a scalar prediction per variable.

Similarly, for VC-2-MPNN and VC-2-FMPNN, all intermediate neural functions are implemented as single-layer MLPs. For VC-2-FMPNN, the aggregation function for $\text{MAP}^t(h_{uj}^{t-1})$ and $\text{MAP}^t(h_{iu}^{t-1})$ are replaced by multiplication, therefore it can be implemented in a way similar to Maron et al. (2019a). The δ -VC-2-MPNN differs from the VC-2-MPNN architecture, in that a matrix multiplication is performed between the embedding matrix and a matrix comprised by adj indicators, effectively aggregating the neighborhood indicator. For VC-2-IGN, we employ the basis formulation described in Appendix E.3, parameterizing each basis operation with a single linear layer. For VC-ET, the feed-forward network FFN consists of a single linear layer, and the TriAttn module of 1 attention head is implemented with learnable weight matrices. For all neural architectures, we insert a LayerNorm (Ba et al., 2016) between the GNN convolutional layers.

To ensure fair comparison, we maintain consistent hidden dimensions and layer counts across models for each dataset. We fix the hidden dimension to 96 for all architectures and all datasets. Regarding number of GNN layers, we employ 6 layers for the Max Clique, MIS, and LMI datasets; and 10 layers for the max-cut (both ER and regular graphs, and SDPLIB), Vertex Cover, and max 2-SAT datasets.

Results on LMI problems Since the optimal objective value for LMI problems is 0, we report the predicted objective values directly in Table 6. As shown, performance varies minimally across different neural architectures. This is likely because the LMI instance coefficients are randomly generated, lacking the complex structural symmetries found in graph-based problems that typically expose the limitations of weaker methods.

Compare to CO solution To directly illustrate the comparison regarding how well our predictions translate to discrete CO solutions, we evaluated relative objective gap for Max-Cut, MIS and Vertex Cover.

Table 6. Loss and objective of LMI problems on test set, repeated with 5 seeds. The best results across all are highlighted.

Model	Test loss	Obj.	Proj. obj.
VC-MPNN	4.520e-5 $\pm$ 0.000	-0.091 $\pm$ 0.275	0.124 $\pm$ 0.126
VC-2-MPNN	4.516e-5 $\pm$ 0.000	0.031 $\pm$ 0.044	0.074 $\pm$ 0.028
δ -VC-2-MPNN	4.563e-5 $\pm$ 0.000	-0.163 $\pm$ 0.066	0.021$\pm$0.001
VC-2-IGN	4.506e-5 $\pm$ 0.000	0.017$\pm$0.103	0.090 $\pm$ 0.053
VC-2-FMPNN	4.305e-5$\pm$0.000	-0.063 $\pm$ 0.042	0.066 $\pm$ 0.012
VC-ET	4.357e-5 $\pm$ 0.000	-0.036 $\pm$ 0.000	0.122 $\pm$ 0.097

Table 7. The relative objective gap (%) between our predicted SDP solution and CO solution.

Candidate	Max-Cut	MIS	Vertex Cover
SCS - CO	4.547 $\pm$ 0.858	199.229 $\pm$ 2.465	44.622 $\pm$ 3.699
VC-2-FMPNN - CO	4.610 $\pm$ 0.855	199.358 $\pm$ 3.426	44.955 $\pm$ 3.897

As Table 7 shows, the downstream CO gaps produced by our neural solver perfectly mirror the inherent gaps of the exact SDP relaxations. For Max-Cut, the SDP solution of our GNN performs almost identically to exact SDP solver outputs, with a negligible 0.06% difference. The observations have validated our core claim: the neural approach successfully learns the underlying continuous SDP geometry, rather than directly learning the discrete integer solutions.

However, the neural solver’s high quality solution and fast inference time enable massive potential for CO solvers. Modern exact branch-and-bound methods for Max-Cut (Hrga & Povh, 2021) require solving thousands of intermediate SDPs, which is the primary computational bottleneck. Replacing these expensive exact solvers with our cheap neural proxy to guide branching or pruning decisions, similar to the seminal work Gasse et al. (2019) for LPs, is a highly promising future direction.

Compare to OptGNN We adapted OptGNN (Yau et al., 2024) to our evaluation framework for comparison in relative objective gap on Max-Cut SDP, and the other datasets are incompatible due to formulation differences or unavailable source code.

Table 8. Comparison to OptGNN on Max-Cut problems.

Model	Target	Max-Cut ER	Max-Cut Regular
OptGNN	obj. gap	1.304 $\pm$ 0.014	2.030 $\pm$ 0.019
OptGNN	cons. vio.	0.0	0.0
VC-2-FMPNN	obj. gap	0.111 $\pm$ 0.017	0.092 $\pm$ 0.004
VC-2-FMPNN	cons. vio.	0.003 $\pm$ 0.001	0.003 $\pm$ 0.001

OptGNN achieves zero constraint violations because it optimizes a low-rank latent embedding where the Max-Cut constraint is trivially satisfied via row-normalization. While we could also adapt VC-2-FMPNN to predict the latent to achieve feasibility, we stick to our formulation in the paper to ensure consistency with general SDP. As shown in Table 8, our architecture achieves an obj. gap one order of magnitude tighter. However, we still highlight our different scopes: OptGNN targets discrete CO problems using SDP relaxations as a tool, whereas our framework serves as a continuous surrogate for solving the general linear SDP.

More SDPLIB results We exhibit the statistics of SDPLIB datasets in Table 9.

We show the complete approximation performance of VC-MPNN, VC-2-MPNN and VC-2-FMPNN on SDPLIB, in Table 10.

Constraint violation As demonstrated in Table 11, our VC-2-FMPNN achieves remarkably low constraint violation residues consistently across all tasks, on the order of 10^{-3} to 10^{-5} . While not perfectly feasible, these violations are very low and highly competitive for practical applications.

Table 9. Statistics of selected SDPLIB problems.

Name	#vars	#cons.
MCP100	10000	100
MCP124-{1-4}	15376	124
MCP250-{1-4}	62500	250
MCP500-{1-4}	250000	500

Table 10. Loss and objective gap (%) on SDPLIB instances, repeated with 5 seeds. The best results are highlighted.

Name	Loss			Obj gap (%)			Proj. obj gap (%)		
	VC-MPNN	VC-2-MPNN	VC-2-FMPNN	VC-MPNN	VC-2-MPNN	VC-2-FMPNN	VC-MPNN	VC-2-MPNN	VC-2-FMPNN
MCP100	0.203±0.000	0.199±0.004	1.974e-4±0.000	0.132±0.083	0.145±0.052	0.002±0.000	15.336±0.030	16.332±0.438	1.839±0.050
MCP124-1	0.225±0.000	0.224±0.000	1.546e-4±0.000	0.298±0.199	0.377±0.023	0.039±0.001	12.255±0.157	12.770±0.162	1.802±0.605
MCP124-2	0.220±0.000	0.218±0.002	2.303e-4±0.000	0.166±0.105	0.267±0.023	0.047±0.005	15.184±0.213	15.998±0.324	1.976±0.785
MCP124-3	0.215±0.000	0.214±0.002	3.076e-4±0.000	0.019±0.012	0.325±0.176	0.053±0.017	17.424±0.094	17.959±0.542	2.092±0.452
MCP124-4	0.270±0.000	0.272±0.007	3.374e-4±0.000	0.139±0.067	0.174±0.012	0.069±0.005	20.246±0.013	20.966±0.828	2.967±0.881
MCP250-1	0.215±0.000	0.215±0.000	3.034e-4±0.000	0.284±0.022	0.232±0.032	0.018±0.001	13.171±0.188	12.794±0.045	2.393±1.103
MCP250-2	0.158±0.015	0.160±0.007	4.176e-4±0.000	0.113±0.017	0.274±0.134	0.013±0.008	15.320±0.129	15.269±0.533	2.525±0.778
MCP250-3	0.155±0.000	0.156±0.000	6.266e-4±0.000	0.111±0.049	0.523±0.176	0.391±0.076	18.913±0.087	17.242±0.129	2.892±0.948
MCP250-4	0.182±0.013	0.186±0.004	5.892e-4±0.000	0.137±0.017	0.166±0.088	0.041±0.008	20.134±0.144	20.405±0.592	3.428±1.321
MCP500-1	0.118±0.000	0.118±0.001	4.536e-4±0.000	0.310±0.022	0.237±0.020	0.247±0.011	13.054±0.176	12.842±0.175	3.265±0.912
MCP500-2	0.123±0.000	0.124±0.001	4.948e-4±0.000	0.337±0.103	0.431±0.027	0.345±0.117	16.303±0.121	16.307±0.671	3.240±0.431
MCP500-3	0.121±0.000	0.122±0.000	6.745e-4±0.000	0.060±0.059	0.157±0.011	0.322±0.034	18.428±0.114	18.136±0.090	4.143±1.323
MCP500-4	0.126±0.000	0.130±0.003	9.761e-4±0.000	0.437±0.032	0.074±0.058	0.407±0.010	21.048±0.154	20.610±0.051	4.505±1.201

Table 11. Mean absolute residuals on constraints of different CO problems on test set, repeated with 5 seeds.

Candidate	Target	Problems					
		Max-Cut	Max-Cut (reg)	Max-Clique	MIS	Vertex Cover	Max 2-SAT
SCS	Time (sec.)	0.386±0.028	0.385±0.190	0.642±0.308	0.672±0.340	0.089±0.004	0.266±0.081
VC-2-FMPNN	Time (sec.)	0.010±0.000	0.010±0.000	0.019±0.001	0.019±0.001	0.019±0.001	0.011±0.001
	Proj. Obj. gap (%)	0.111±0.017	0.092±0.004	0.436±0.035	0.411±0.025	0.469±0.051	0.862±0.211
	Proj. cons. vio	0.003±0.001	0.003±0.001	3.082e-5±0.000	3.067e-5±0.000	0.009±0.001	0.008±0.001

SDPLR quality Burer-Monteiro method is not guaranteed to solve the SDP exactly. In Table 12, SDPLR yields near-zero objective gaps compared to SCS, except for 0.02% on Max-Clique/MIS, and almost zero constraint violation up to numerical precisions. While it is highly competitive on Max-Cut problems, it fails to converge on Vertex Cover, and scales poorly in runtime on MIS.

Table 12. Mean absolute residuals on constraints of different CO problems on test set, repeated with 5 seeds.

Candidate	Target	Problems					
		Max-Cut	Max-Cut (reg)	Max-Clique	MIS	Vertex Cover	Max 2-SAT
SCS	Time (sec.)	0.386±0.028	0.385±0.190	0.642±0.308	0.672±0.340	0.089±0.004	0.266±0.081
SDPLR	Time (sec.)	0.195±0.041	0.235±0.052	1.803±0.184	3.301±0.285	Not converging	0.391±0.145
	Proj. Obj. gap (%)	3.672e-6±0.000	4.434e-6±0.000	0.020±0.001	0.022±0.002	–	2.341e-5±0.000
	Proj. cons. vio	1.059e-6±0.000	1.193e-6±0.000	4.269e-7±0.000	2.696e-7±0.000	–	8.223e-7±0.000

Effect of depth To directly observe the effect of depth of VC-2-FMPNN, we conducted a sensitivity analysis on smaller Max-Cut SDPs (ER graphs, 50 nodes, $p = 0.1$) with varying network depths.

Remarkably, we observe no performance drop even at 30 layers. Performance consistently improves with depth, though the gains are marginal after 10 layers, which motivated our default setting with 10 layers in the paper.

Table 13. The loss, relative objective gap and constraint violation on test set, with different number of layers.

Layers	Loss	Obj. gap	Proj. obj. gap	Cons. vio.	Proj. cons. vio.
5	3.103e-3±0.000	0.276±0.001	0.254±0.001	0.004±0.001	0.039±0.002
10	1.088e-4±0.000	0.076±0.005	0.073±0.005	0.001±0.000	0.004±0.001
15	3.448e-5±0.000	0.054±0.002	0.077±0.005	0.001±0.000	0.002±0.001
20	2.594e-5±0.000	0.049±0.005	0.051±0.005	0.001±0.000	0.002±0.000
30	1.141e-5±0.000	0.047±0.005	0.054±0.017	0.001±0.000	0.001±0.000